TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA ĐIỆN – ĐIỆN TỬ



LÊ VIỆT CƯỜNG - 42100549

THIẾT KẾ VÀ THI CÔNG MÔ HÌNH ĐIỀU HƯỚNG PIN NĂNG LƯỢNG MẶT TRỜI

ĐỒ ÁN TỔNG HỢP
KỸ THUẬT CƠ ĐIỆN TỬ

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA ĐIỆN – ĐIỆN TỬ



LÊ VIỆT CƯỜNG – 42100549

THIẾT KẾ VÀ THI CÔNG MÔ HÌNH ĐIỀU HƯỚNG PIN NĂNG LƯỢNG MẶT TRỜI

ĐỒ ÁN TỔNG HỢP
KỸ THUẬT CƠ ĐIỆN TỬ

Người hướng dẫn
TS. Nguyễn Hoàng Nam

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

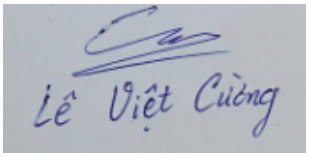
Đầu tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến quý thầy/cô giảng viên Khoa Điện – Điện tử của Trường Đại học Tôn Đức Thắng đã tận tình giảng dạy, truyền đạt những kiến thức quý báu trong suốt quá trình học tập và rèn luyện tại trường. Những kiến thức và kinh nghiệm đó chính là nền tảng quan trọng giúp em hoàn thành đồ án tốt nghiệp này.

Đặc biệt, em xin gửi lời cảm ơn sâu sắc đến thầy Nguyễn Hoàng Nam người đã luôn tận tình hướng dẫn, góp ý chi tiết và động viên em trong suốt quá trình thực hiện đề tài "Thiết kế và thi công mô hình điều hướng pin năng lượng mặt trời" . Sự chỉ bảo tận tâm của thầy đã giúp em từng bước hoàn thiện nội dung nghiên cứu, vượt qua những khó khăn về mặt kỹ thuật cũng như phương pháp thực hiện. Với sự tận tình chỉ bảo, góp ý chi tiết từ giai đoạn hình thành ý tưởng đến hoàn thiện sản phẩm, thầy đã giúp em mở rộng tư duy thiết kế và nâng cao khả năng vận dụng phần mềm, kiến thức thực tiễn một cách hiệu quả hơn. Những kỹ năng quý báu đó sẽ là hành trang vững chắc cho em sau này.

Mặc dù đây không phải đồ án đầu tiên em thực hiện, những mỗi đề tài đều có những khó khăn, thử thách mới. Trong quá trình triển khai chắc chắn sẽ không tránh khỏi những thiếu sót nhất định về nội dung và cách trình bày. Em rất mong nhận được sự góp ý quý báu từ quý thấy/cô để em có thể hoàn thiện bản thân và học hỏi thêm nhiều kinh nghiệm, kiến thức làm hành trang cho các dự án sau này.

TP. Hồ Chí Minh, ngày 20 tháng 07 năm 2026

Tác giả



Công trình được hoàn thành tại Trường Đại học Tôn Đức Thắng

Cán bộ hướng dẫn khoa học: ....TS Nguyễn Hoàng Nam

.....

(Ghi rõ học hàm, học vị, họ tên và chữ ký)

Đồ án tốt tổng hợp được bảo vệ tại **Hội đồng đánh giá Đồ án tổng hợp của Trường Đại học Tôn Đức Thắng** vào ngày 7 /8/2026

Xác nhận của Chủ tịch Hội đồng đánh giá Đồ án tổng hợp và Trưởng khoa quản lý chuyên ngành sau khi nhận Đồ án tổng hợp đã được sửa chữa (nếu có).

CHỦ TỊCH HỘI ĐỒNG	**TRƯỞNG KHOA**
.....	.....

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH
TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

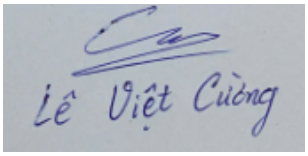
Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi và được sự hướng dẫn khoa học của TS. Nguyễn Hoàng Nam. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Đồ án tổng hợp còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung Đồ án tốt nghiệp của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày 20 tháng 07 năm 2026

Tác giả



TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA ĐIỆN –ĐIỆN TỬ

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc

TP.HCM, ngày 23 tháng 05 năm 2026

NHIỆM VỤ ĐỒ ÁN TỔNG HỢP

(Bản nhiệm vụ này được đóng vào trang thứ nhất của đồ án)

Họ tên sinh viên : *Lê Việt Cường*

Ngành: *Kỹ thuật cơ điện tử* Lớp: *21040401* MSSV: 42100549

1. Tên đề tài: *Thiết kế và thi công mô hình điều hướng pin năng lượng mặt trời*

Design and Implementation of a Solar Panel Tracking Model

2. Nhiệm vụ (yêu cầu về nội dung và số liệu ban đầu):

- Xác định yêu cầu công nghệ của đề tài.
- Lựa chọn các thiết bị, vẽ sơ đồ nguyên lý, lưu đồ giải thuật chương trình và nêu nguyên tắc hoạt động của hệ thống.
- Thiết kế bản vẽ cơ khí và thi công mô hình.
- Lập trình điều khiển và giám sát hệ thống.
- Thí nghiệm và khảo sát chất lượng hệ thống.
- Viết báo cáo và soạn Slide để thuyết trình.
- Trình bày poster theo mẫu.

3. Ngày giao nhiệm vụ: 22/03/2026

4. Ngày bảo vệ 50%: 11/05/2026 - 17/05/2026

5. Ngày hoàn thành và nộp về khoa: 19/07/2026

6. Giáo viên hướng dẫn: TS. Nguyễn Hoàng Nam Phần hướng dẫn: 100%

Nội dung và yêu cầu đã được thông qua Khoa và Bộ môn.

Ngày 29 tháng 05 năm 2026

CHỦ NHIỆM BỘ MÔN

TS. Vũ Trí Viễn

GIẢNG VIÊN HƯỚNG DẪN

TS. Nguyễn Hoàng Nam

TRƯỞNG KHOA

PGS. TS. Trần Thanh Phương

TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA ĐIỆN –ĐIỆN TỬ

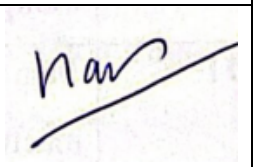
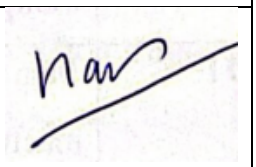
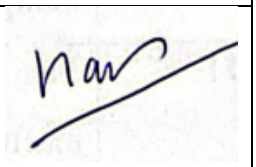
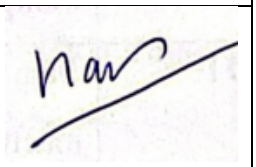
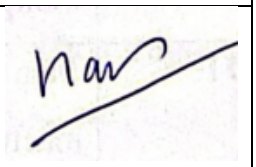
CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc

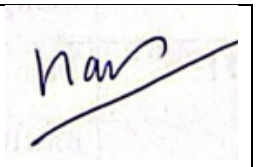
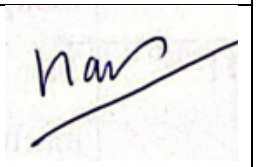
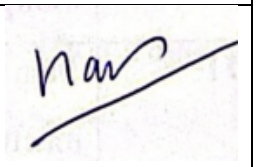
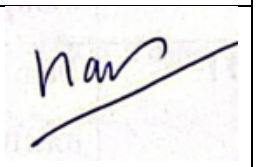
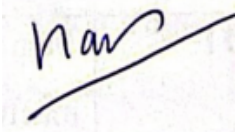
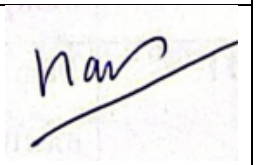
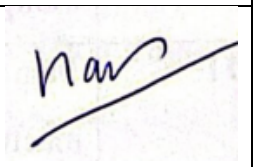
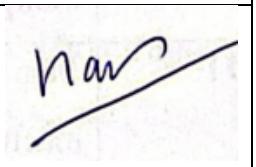
LỊCH TRÌNH LÀM ĐỒ ÁN TỔNG HỢP

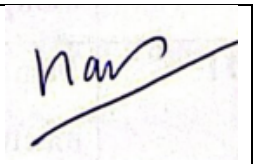
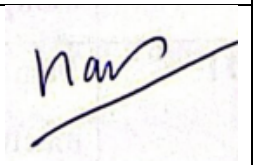
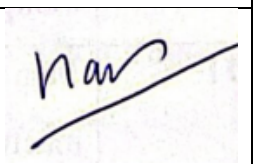
Họ tên sinh viên: Lê Việt Cường

Lớp: 21040401 MSSV: 42100549

Tên đề tài: Thiết kế và thi công mô hình điều hướng pin năng lượng mặt trời

Tuần	**Khối lượng**		**GVHD ký**
	Đã thực hiện	**Tiếp tục thực hiện**	
1	Định hướng và chọn đề tài, tìm hiểu các chủ đề liên quan	Hoàn thành chọn đề tài	
2	Thiết kế mô hình 3D trên SolidWorks	Tiếp tục thiết kế mô hình 3d	
3	Thiết kế mô hình 3D trên SolidWorks	Tiếp tục thiết kế mô hình 3d	
4	Thiết kế mô hình 3D trên SolidWorks	Tiếp tục thiết kế mô hình 3d	
5	Hoàn thành thiết kế 3D	Chỉnh sửa lại và hoàn thiện các thiết kế	

6	Gia công các chi tiết của máy	Mua linh kiện, gia công chi tiết	
7	Gia công các chi tiết của máy	Lắp ghép máy	
Kiểm tra giữa kỳ	Đánh giá khối lượng hoàn thành…50…..% được tiếp tục thực hiện ĐATH		
8	Lắp ráp phần cứng của máy	Lắp ghép máy	
9	Hoàn thành lắp ráp phần cứng của máy	Chỉnh sửa, bổ xung các phần còn thiếu chưa hoàn thiện của máy	
10	Tìm hiểu nguyên lý hoạt động của tấm pin mặt trời và các module cần cho hệ thống	Tìm hiểu và thiết kế lưu đồ giải thuật và sơ đồ khối hệ thống	
11	Tiến hành lắp ráp mạch điện	Chỉnh sửa mạch điện và test các thiết bị	
12	Chạy mô hình và thu thập dữ liệu	Tiếp tục thu thập dữ liệu	
13	Tiếp tục thu thập dữ liệu	Thu thập dữ liệu và chỉnh sửa	

14	Hoàn thành dữ liệu trong 10 ngày	Kiểm tra tất cả các dữ liệu.	
15	Kiểm tra lại tất cả hoạt động của hệ thống	Viết báo cáo và in báo cáo	
16	Viết báo cáo và in báo cáo	Chuẩn bị bảo vệ đồ án	
Nộp Đồ án tổng hợp	Đã hoàn thành…100…..% Đồ án tổng hợp được bảo vệ ĐATH		

THIẾT KẾ VÀ THI CÔNG MÔ HÌNH ĐIỀU HƯỚNG PIN NĂNG LƯỢNG MẶT TRỜI

TÓM TẮT

Đề tài hướng đến việc xây dựng một hệ thống điều khiển bám theo hướng chiếu của mặt trời kết hợp với giám sát hiệu suất hoạt động của tấm pin năng lượng mặt trời trên nền tảng Internet of Things (IoT). Hệ thống được thiết kế dựa trên sự kết hợp giữa vi điều khiển ESP32 và Arduino Mega cùng thuật toán điều khiển PID nhằm nâng cao độ chính xác và tính ổn định trong quá trình vận hành. Mô hình sử dụng 4 cảm biến ánh sáng (LDR) và mạch đo công suất INA219 nhằm thu thập các dữ liệu cần thiết. Dữ liệu sau khi được xử lí được hiển thị trên trang web nhờ lập trình HTML, C++. Đồng thời hệ thống điều khiển động cơ STEP Motor và các công tắc hành trình nhằm để điều chỉnh vị trí của tấm pin luôn luôn hướng về phía có cường đồ ánh sáng mặt trời lớn nhất.

Ngoài chức năng điều khiển cơ khí, toàn bộ dữ liệu từ các cảm biến còn được truyền lên nền tảng IoT, cho phép người dùng giám sát và theo dõi tình trạng hoạt động của hệ thống từ xa thông qua kết nối Internet. Việc ứng dụng bộ điều khiển PID giúp tối ưu quá trình quay của tấm pin, giảm sai số, tăng độ ổn định và duy trì hiệu quả hoạt động trong các điều kiện thời tiết khác nhau. Bên cạnh đó, đề tài tiến hành đo đạc, phân tích và so sánh các thông số điện áp và dòng điện giữa mô hình tấm pin có cơ cấu bám mặt trời (tracking) và tấm pin lắp đặt cố định (static), từ đó đánh giá mức độ cải thiện hiệu suất thu nhận năng lượng. Kết quả nghiên cứu góp phần nâng cao hiệu quả khai thác nguồn năng lượng mặt trời, đồng thời tạo tiền đề phát triển các hệ thống năng lượng tái tạo thông minh có khả năng mở rộng và đáp ứng tốt nhu cầu sử dụng trong thực tế.

Design and Implementation of a Solar Panel Tracking Model

This project aims to develop a solar tracking and performance monitoring system for photovoltaic (PV) panels based on the Internet of Things (IoT). The system integrates an ESP32 microcontroller, an Arduino Mega, and a PID control algorithm to improve tracking accuracy and ensure stable operation. The proposed model employs four Light Dependent Resistor (LDR) sensors and an INA219 power monitoring module to collect the required operating data. After processing, the collected data are displayed on a web-based interface developed using HTML and C++. At the same time, the system controls a stepper motor and limit switches to continuously adjust the orientation of the solar panel so that it always faces the direction of maximum sunlight intensity.

In addition to its mechanical tracking function, all sensor data are transmitted to an IoT platform, enabling users to remotely monitor the operating status of the system through an Internet connection in real time. The implementation of the PID controller optimizes the panel's rotational movement by minimizing tracking errors, enhancing system stability, and maintaining high performance under various weather conditions. Furthermore, the study measures, analyzes, and compares the voltage and current characteristics of a solar tracking panel with those of a conventional fixed (static) solar panel to evaluate the improvement in energy harvesting efficiency. The research findings contribute to enhancing the utilization efficiency of solar energy while providing a foundation for the development of intelligent, user-friendly, and scalable renewable energy systems suitable for future practical applications.

MỤC LỤC

LỜI CẢM ƠN ii
MỤC LỤC xi
DANH MỤC HÌNH ẢNH xiv
DANH MỤC BẢNG BIỂU xvi
DANH MỤC CHỮ VIẾT TẮT xvii
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI 1
1.1 Tổng quan về hệ thống 1
1.2 Lí do chọn đề tài 3
1.3 Mục tiêu của phần thiết kế và thi công 4
1.4 Phạm vi nghiên cứu của đề tài 6
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 8
2.1 Pin năng lượng mặt trời 8
2.1.1 Khái niệm 8
2.1.2 Cấu tạo của tấm pin năng lượng mặt trời 10
2.1.3 Nguyên lí hoạt động của tấm pin năng lượng mặt trời 12
2.2 Hệ thống hướng nắng 14
2.3 Khối xử lí trung tâm 16
2.3.1 ESP 32 16
2.3.2 Mạch đo công suất INA219 18
2.4 Khối chấp hành 19
2.4.1 Rotary Encoder HN3806-AB-600G 19

2.4.2 Động cơ Step 57 22
2.5 Cảm biến ánh sáng TEMT6000 (LDR) 24
2.6 Khối nguồn: 25
2.6.1 Nguồn tổ ong 24V 25
2.6.2 Driver TB6600 27
2.7 Khối hiển thị 29
2.7.1 Ngôn ngữ lập trình 29
2.7.2 Bộ điều khiển sạc năng lượng mặt trời 12V 32
CHƯƠNG 3. THIẾT KẾ VÀ TÍNH TOÁN 35
3.1 Thiết kế hệ thống 35
3.1.1 Sơ đồ khối hệ thống 35
3.1.2 Lưu đồ giải thuật 38
3.1.3 Thuật toán điều khiển PID 40
3.1.4 PID vòng ngoài (PID quang học/vị trí) 43
3.1.5 PID vòng trong (PID tốc độ cơ khí của Encoder) 44
3.1.6 Thiết kế mạch điện 47
3.2 Tính toán cơ cấu 50
CHƯƠNG 4. THỰC NGHIỆM 54
4.1 Giao diện từ xa IoT: 54
4.2 Kết quả 56
CHƯƠNG 5. KẾT LUẬN 58
5.1 Kết luận 58
5.2 Ưu điểm của hệ thống 59
5.3 Nhược điểm của hệ thống 59

5.4 Định hướng phát triển trong tương lai 60
TÀI LIỆU THAM KHẢO 61
PHỤ LỤC A 1
PHỤ LỤC B 1

DANH MỤC HÌNH ẢNH

Hình 1.1 Mô hình điều hướng pin năng lượng mặt trời 1
Hình 2.1 Tấm pin năng lượng mặt trời 8
Hình 2.2 Thông số của tấm pin năng lượng mặt trời 10
Hình 2.3 Cấu tạo của tấm pin năng lượng mặt trời 10
Hình 2.4 Nguyên lí hoạt động của tấm pin năng lượng mặt trời 12
Hình 2.5 Mô hình điều hướng pin năng lượng mặt trời 14
Hình 2.6 Sơ đồ khối 15
Hình 2.7 ESP32 DevKit V1 16
Bảng 2.1 Thông số kỹ thuật ESP32 DevKit V1 17
Hình 2.8 Mạch đo công suất INA219 18
Hình 2.9 Rotary Encoder 20
Hình 2.10 Động cơ Step Nema57 23
Hình 2.11 Cảm biến ánh sáng TEMT6000 (LDR) 24
Hình 2.11 Nguồn tổ ong 24V-5A 26
Hình 2.12 Driver TB6600 27
Hình 2.13 Sơ đồ chân của Driver TB6600 29
Hình 2.14 Giao diện điều khiển trên điện thoại 30
Hình 2.15 Ngôn ngữ lập trình C++ 31
Hình 2.16 Bộ điều khiển sạc pin năng lượng mặt trời 12V 32
Hình 3.1 Sơ đồ khối tổng quát của hệ thống 35
Hình 3.2 Lưu đồ giải thuật của hệ thống 38
Hình 3.3 Sơ đồ mạch điện 47

Hình 4.1 Giao diện biểu đồ đo điện áo, công suất và dòng điện 54
Hình 4.2 Dữ liệu trên DataSheet 55
Hình 4.3 Biểu đồ ngày 1 xuất các giá trị trên DataSheet 56
Hình 5.1 Mô hình sau khi hoàn thiện 59

DANH MỤC BẢNG BIỂU

Bảng 2.2 Thông số kỹ thuật của mạch đo công suất INA219 19

Bảng 2.3 Thông số kỹ thuật của Rotary Encoder HN3806-AB-600g..... 21

Bảng 2.4 Thông số kỹ thuật của động cơ Step Nema57 23

Bảng 2.5 Thông số kỹ thuật của cảm biến ánh sáng (LDR)..... 25

Bảng 2.6 Thông số kỹ thuật nguồn tổ ong 24V-5A 26

Bảng 2.7 Thông số kỹ thuật của Driver TB6600..... 28

Bảng 2.8 Thông số kĩ thuật của bộ điều khiển sạc năng lượng mặt trời 33

DANH MỤC CHỮ VIẾT TẮT

Iot - Internet of Things

I Current – Dòng điện

U Voltage – Điện áp

P Power – Công suất

PID - Proportional-Integral-Derivative

EVA - Ethylene Vinyl Acetate

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Tổng quan về hệ thống

Trong những năm gần đây, nhu cầu khai thác và sử dụng các nguồn năng lượng tái tạo ngày càng tăng nhằm đáp ứng yêu cầu phát triển bền vững và giảm sự phụ thuộc vào nhiên liệu hóa thạch. Trong đó, năng lượng mặt trời được xem là một trong những nguồn năng lượng sạch có tiềm năng lớn. Để nâng cao hiệu quả khai thác nguồn năng lượng này, việc ứng dụng các công nghệ hiện đại, đặc biệt là Internet of Things (IoT), vào hệ thống điện mặt trời đang trở thành xu hướng tất yếu. IoT cho phép các thiết bị được kết nối, giám sát và điều khiển từ xa, góp phần cải thiện hiệu quả vận hành cũng như công tác quản lý hệ thống. Tuy nhiên, tại Việt Nam, việc áp dụng công nghệ IoT vào các hệ thống pin năng lượng mặt trời vẫn còn ở giai đoạn đầu, chưa được triển khai rộng rãi và chưa phát huy hết tiềm năng trong thực tiễn.

Bên cạnh đó, sự phát triển nhanh chóng của các hệ thống điện mặt trời cũng đặt ra nhiều yêu cầu về quản lý, giám sát, bảo trì và tối ưu hóa hiệu suất hoạt động. Các phương pháp quản lý truyền thống còn tồn tại nhiều hạn chế trong việc theo dõi tình trạng thiết bị theo thời gian thực và phát hiện sớm các sự cố. Với khả năng kết nối các thiết bị thông qua mạng không dây, công nghệ IoT tạo điều kiện xây dựng một hệ thống giám sát và điều khiển thông minh, giúp nâng cao độ tin cậy, giảm chi phí vận hành và tối ưu hiệu quả khai thác năng lượng.



Hình 1.1 Mô hình điều hướng pin năng lượng mặt trời

Xuất phát từ những yêu cầu trên, đề tài tập trung nghiên cứu, thiết kế và xây dựng một hệ thống quản lý, giám sát và điều khiển pin năng lượng mặt trời ứng dụng công nghệ IoT. Hệ thống được phát triển với các mục tiêu chính như sau:

- Thiết kế cơ chế tự động điều chỉnh hướng của tấm pin theo vị trí có cường độ ánh sáng cao nhất thông qua các cảm biến ánh sáng kết hợp thuật toán điều khiển PID và động cơ dẫn động.
- Thu thập, xử lý, hiển thị và lưu trữ theo thời gian thực các thông số môi trường như nhiệt độ, độ ẩm, cường độ ánh sáng cùng các thông số điện gồm điện áp và dòng điện trên trang web
- Kết hợp hai vi điều khiển Arduino Mega và ESP32 nhằm phân chia nhiệm vụ điều khiển, xử lý dữ liệu và truyền thông, qua đó nâng cao hiệu suất và độ ổn định của hệ thống.
- Tích hợp các cảm biến và thiết bị ngoại vi như cảm biến ánh sáng LDR, động cơ STEP, mạch đo công suất INA219 và các công tắc hành trình để xây dựng nên hệ thống hoàn chỉnh
- Ứng dụng thuật toán điều khiển PID nhằm tăng độ chính xác trong quá trình định hướng tấm pin, giảm sai số và đảm bảo hệ thống hoạt động ổn định dưới nhiều điều kiện chiếu sáng khác nhau.
- Tiến hành đánh giá và so sánh các thông số điện cũng như sản lượng điện thu được giữa mô hình pin năng lượng mặt trời có cơ cấu bám mặt trời (tracking) và mô hình pin cố định (static), từ đó chứng minh hiệu quả cải thiện khả năng thu nhận năng lượng của hệ thống.
- Xây dựng giao diện giám sát trực quan, thân thiện với người dùng, cho phép truy cập thông qua trình duyệt web hoặc thiết bị di động để theo dõi và quản lý hệ thống từ xa một cách thuận tiện.

Việc nghiên cứu và triển khai hệ thống giám sát, quản lý pin năng lượng mặt trời dựa trên nền tảng IoT không chỉ góp phần nâng cao hiệu quả khai thác nguồn năng lượng tái tạo mà

còn thúc đẩy quá trình chuyển đổi số trong lĩnh vực năng lượng. Đồng thời, hệ thống được xây dựng có khả năng mở rộng và ứng dụng thực tế tại các hộ gia đình, doanh nghiệp cũng như những khu vực có hạ tầng điện lưới chưa ổn định, góp phần phát triển các mô hình điện mặt trời thông minh theo định hướng phát triển bền vững trong tương lai.

1.2 Lí do chọn đề tài

Trong bối cảnh nguồn năng lượng hóa thạch ngày càng cạn kiệt và các vấn đề về ô nhiễm môi trường, biến đổi khí hậu ngày càng trở nên nghiêm trọng, việc nghiên cứu và phát triển các nguồn năng lượng tái tạo đã trở thành xu hướng tất yếu trên thế giới. Trong số đó, năng lượng mặt trời được đánh giá là nguồn năng lượng sạch, dồi dào, thân thiện với môi trường và có tiềm năng ứng dụng rộng rãi. Tại Việt Nam, với lợi thế nằm trong khu vực có cường độ bức xạ mặt trời tương đối cao, việc khai thác năng lượng mặt trời đang được Nhà nước và các doanh nghiệp đặc biệt quan tâm nhằm đáp ứng nhu cầu sử dụng điện ngày càng tăng và góp phần thực hiện mục tiêu phát triển bền vững.

Tuy nhiên, hiệu suất của các hệ thống pin năng lượng mặt trời phụ thuộc rất lớn vào góc chiếu của ánh sáng lên bề mặt tấm pin. Phần lớn các hệ thống hiện nay sử dụng phương pháp lắp đặt cố định (Static), khiến góc tiếp nhận bức xạ mặt trời thay đổi theo thời gian trong ngày. Điều này làm giảm lượng năng lượng thu được, đặc biệt vào buổi sáng và buổi chiều khi góc chiếu của mặt trời thay đổi đáng kể.

Để khắc phục hạn chế trên, hệ thống điều hướng tấm pin năng lượng mặt trời (Solar Tracking System) được xem là một giải pháp hiệu quả. Bằng cách tự động điều chỉnh hướng của tấm pin theo vị trí của mặt trời, hệ thống giúp duy trì góc thu nhận bức xạ tối ưu, từ đó nâng cao sản lượng điện tạo ra so với mô hình lắp đặt cố định. Bên cạnh đó, sự phát triển mạnh mẽ của các vi điều khiển như ESP32, Arduino Mega cùng các công nghệ điều khiển tự động, cảm biến và Internet of Things (IoT) đã tạo điều kiện thuận lợi để xây dựng các hệ thống điều hướng thông minh có khả năng giám sát và điều khiển từ xa với chi phí hợp lý.

Xuất phát từ những vấn đề trên, nhóm lựa chọn thực hiện đề tài "Thiết kế và thi công mô hình điều hướng pin năng lượng mặt trời" nhằm nghiên cứu, thiết kế và chế tạo một mô hình có khả năng tự động định hướng tấm pin theo cường độ ánh sáng mặt trời thông qua các cảm biến ánh sáng và cơ cấu truyền động. Đồng thời, hệ thống tích hợp chức năng giám sát các thông số hoạt động của tấm pin trên nền tảng IoT, cho phép người dùng theo dõi và quản lý từ xa thông qua giao diện web. Ngoài ra, đề tài còn tiến hành đánh giá và so sánh hiệu suất giữa mô hình pin năng lượng mặt trời có cơ cấu điều hướng (Tracking) và mô hình lắp đặt cố định (Static), từ đó làm rõ hiệu quả của giải pháp trong việc nâng cao khả năng khai thác năng lượng mặt trời.

Thông qua quá trình thực hiện đề tài, nhóm không chỉ vận dụng các kiến thức về điện – điện tử, điều khiển tự động, lập trình nhúng và IoT vào thực tế mà còn góp phần xây dựng một mô hình có tính ứng dụng cao, phù hợp với xu hướng phát triển các hệ thống năng lượng tái tạo thông minh, đáp ứng yêu cầu chuyển đổi số và phát triển bền vững trong tương lai.

1.3 Mục tiêu của phần thiết kế và thi công

Đề tài được thực hiện nhằm nghiên cứu, thiết kế và thi công một mô hình điều hướng pin năng lượng mặt trời có khả năng tự động định hướng tấm pin theo vị trí có cường độ bức xạ mặt trời lớn nhất, từ đó nâng cao hiệu suất thu nhận năng lượng so với phương pháp lắp đặt cố định. Đồng thời, hệ thống được tích hợp công nghệ Internet of Things (IoT) để giám sát và quản lý các thông số hoạt động theo thời gian thực, giúp người dùng dễ dàng theo dõi và đánh giá hiệu quả của hệ thống.

Để đạt được mục tiêu trên, đề tài tập trung thực hiện các nội dung sau:

- Nghiên cứu nguyên lý hoạt động của hệ thống pin năng lượng mặt trời và cơ chế điều hướng tấm pin theo hướng ánh sáng.

- Thiết kế và chế tạo mô hình cơ khí cho phép tấm pin quay theo một trục bằng động cơ bước (Step Motor), kết hợp công tắc hành trình nhằm giới hạn hành trình và đảm bảo vận hành an toàn.
- Thiết kế phần cứng điều khiển sử dụng ESP32 để thực hiện việc thu thập dữ liệu, điều khiển cơ cấu truyền động và truyền dữ liệu lên nền tảng IoT.
- Tích hợp các cảm biến như cảm biến ánh sáng LDR và mạch đo công suất INA219 để thu thập các thông số về cường độ ánh sáng, điện áp, dòng điện và công suất của tấm pin.
- Xây dựng thuật toán điều khiển PID nhằm nâng cao độ chính xác của quá trình điều hướng, giảm sai số và tăng tính ổn định khi hệ thống hoạt động trong các điều kiện ánh sáng khác nhau.
- Xây dựng giao diện giám sát trên nền tảng web bằng HTML, CSS và C++, cho phép hiển thị dữ liệu theo thời gian thực và hỗ trợ người dùng theo dõi hoạt động của hệ thống từ xa thông qua Internet.
- Đánh giá khả năng hoạt động của mô hình thông qua quá trình thực nghiệm, phân tích và so sánh các thông số điện áp, dòng điện và công suất giữa hệ thống điều hướng (Tracking) và hệ thống pin lắp đặt cố định (Static), từ đó xác định mức độ cải thiện hiệu suất thu nhận năng lượng.
- Hoàn thiện mô hình có tính ổn định, dễ vận hành, có khả năng mở rộng và làm cơ sở cho việc nghiên cứu, phát triển các hệ thống điện mặt trời thông minh trong thực tế.

Thông qua việc hoàn thành các mục tiêu trên, đề tài không chỉ giúp nâng cao hiệu quả khai thác năng lượng mặt trời mà còn tạo điều kiện ứng dụng các công nghệ điều khiển tự động và IoT vào lĩnh vực năng lượng tái tạo, góp phần đáp ứng xu hướng chuyển đổi số và phát triển bền vững.

1.4 Phạm vi nghiên cứu của đề tài

Đề tài "Thiết kế và thi công mô hình điều hướng pin năng lượng mặt trời" tập trung nghiên cứu, thiết kế và xây dựng một mô hình thực nghiệm có khả năng tự động điều chỉnh vị trí của tấm pin năng lượng mặt trời theo hướng bức xạ mặt trời nhằm nâng cao hiệu suất thu nhận năng lượng. Mô hình được phát triển theo cơ chế điều hướng hai trục (Dual-Axis Solar Tracking), cho phép tấm pin chuyển động đồng thời theo phương ngang (Azimuth) và phương đứng (Elevation), từ đó giúp bề mặt tấm pin luôn duy trì góc tiếp nhận ánh sáng tối ưu trong suốt quá trình mặt trời di chuyển.

Trong phạm vi của đề tài, nhóm tập trung nghiên cứu các nội dung chính sau:

- Nghiên cứu cơ sở lý thuyết về năng lượng mặt trời, nguyên lý hoạt động của pin quang điện (Photovoltaic - PV), đặc tính phát điện của tấm pin và các yếu tố ảnh hưởng đến hiệu suất chuyển đổi năng lượng như cường độ bức xạ, góc nghiêng, nhiệt độ môi trường và vị trí của mặt trời theo thời gian.
- Nghiên cứu nguyên lý của hệ thống điều hướng pin năng lượng mặt trời hai trục, phân tích ưu điểm và hạn chế của mô hình điều hướng so với hệ thống pin năng lượng mặt trời lắp đặt cố định (Static), đồng thời tìm hiểu các phương pháp xác định hướng mặt trời dựa trên cảm biến ánh sáng.
- Thiết kế và chế tạo mô hình cơ khí cho hệ thống điều hướng hai trục, đảm bảo khả năng quay linh hoạt theo cả phương ngang và phương đứng. Hệ thống sử dụng động cơ bước (Stepper Motor) làm cơ cấu truyền động và công tắc hành trình (Limit Switch) để giới hạn góc quay, tránh va chạm cơ khí và bảo vệ các thiết bị trong quá trình vận hành.
- Thiết kế và xây dựng hệ thống điều khiển sử dụng ESP32 làm bộ điều khiển trung tâm. ESP32 đảm nhiệm việc thu thập dữ liệu từ các cảm biến, xử lý thuật toán điều khiển, điều khiển động cơ bước, đồng thời thực hiện kết nối Wi-Fi và truyền dữ liệu lên nền tảng IoT để phục vụ giám sát từ xa.

- Tích hợp bốn cảm biến ánh sáng LDR nhằm xác định hướng có cường độ ánh sáng lớn nhất để điều khiển quá trình bám mặt trời. Đồng thời, hệ thống sử dụng mạch đo công suất INA219 để đo điện áp, dòng điện và công suất đầu ra của tấm pin năng lượng mặt trời, phục vụ cho việc đánh giá hiệu quả hoạt động của mô hình.
- Xây dựng thuật toán điều khiển PID nhằm cải thiện khả năng đáp ứng của hệ thống, giảm sai số trong quá trình định vị, hạn chế hiện tượng dao động khi điều hướng và nâng cao độ ổn định của mô hình trong điều kiện cường độ ánh sáng thay đổi liên tục.
- Xây dựng hệ thống giám sát và quản lý dữ liệu dựa trên nền tảng Internet of Things (IoT). Các thông số như cường độ ánh sáng, điện áp, dòng điện, công suất và trạng thái hoạt động của hệ thống được truyền lên máy chủ thông qua ESP32 và hiển thị trên giao diện web được xây dựng bằng HTML, CSS và C++. Người dùng có thể theo dõi các thông số này theo thời gian thực thông qua máy tính hoặc thiết bị di động có kết nối Internet.
- Tiến hành chạy thử nghiệm mô hình trong các điều kiện chiếu sáng khác nhau, thu thập dữ liệu thực nghiệm và đánh giá khả năng hoạt động của hệ thống. Đồng thời, thực hiện so sánh các thông số điện áp, dòng điện, công suất và sản lượng điện thu được giữa mô hình điều hướng hai trục (Dual-Axis Tracking) và mô hình pin năng lượng mặt trời cố định (Static) nhằm đánh giá mức độ cải thiện hiệu suất của hệ thống điều hướng.

Mặc dù đề tài hướng đến việc xây dựng một hệ thống điều hướng và giám sát hoàn chỉnh, nhưng do giới hạn về thời gian nghiên cứu, kinh phí thực hiện và điều kiện thí nghiệm, phạm vi nghiên cứu chỉ dừng lại ở mức thiết kế và thi công mô hình thực nghiệm có quy mô nhỏ. Đề tài chưa triển khai trên các hệ thống điện mặt trời công suất lớn hoặc các trang trại điện mặt trời ngoài thực tế. Ngoài ra, các yếu tố ảnh hưởng đến độ bền của hệ thống như tác động của gió mạnh, mưa bão, bụi bẩn, ăn mòn môi trường, khả năng chống nước của thiết bị điện và tuổi thọ cơ cấu truyền động

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Pin năng lượng mặt trời

2.1.1 Khái niệm

Pin năng lượng mặt trời (Solar Cell hay Photovoltaic Cell - PV Cell) là thiết bị bán dẫn có khả năng chuyển đổi trực tiếp năng lượng từ bức xạ mặt trời thành điện năng thông qua hiệu ứng quang điện (Photovoltaic Effect). Đây là một trong những công nghệ khai thác năng lượng tái tạo được ứng dụng rộng rãi nhờ khả năng tạo ra điện sạch, không phát thải khí gây ô nhiễm trong quá trình vận hành và góp phần giảm sự phụ thuộc vào các nguồn nhiên liệu hóa thạch. Cấu tạo của pin năng lượng mặt trời chủ yếu gồm hai lớp vật liệu bán dẫn loại P và loại N, trong đó vật liệu được sử dụng phổ biến nhất là silic (Silicon). Khi ánh sáng mặt trời chiếu lên bề mặt pin, các photon mang năng lượng sẽ tác động vào lớp bán dẫn, giải phóng các electron và tạo thành các cặp electron – lỗ trống. Dưới tác dụng của điện trường tại tiếp giáp P–N, các electron và lỗ trống di chuyển theo hai hướng ngược nhau, tạo ra hiệu điện thế giữa hai cực của pin.



Hình 2.1 Tấm pin năng lượng mặt trời

Khi nối với tải ngoài, dòng điện một chiều (DC) được sinh ra và cung cấp năng lượng cho các thiết bị điện. Trong thực tế, một tế bào quang điện đơn lẻ chỉ tạo ra điện áp khoảng 0,5 đến 0,6 V, vì vậy nhiều tế bào được ghép nối tiếp và song song để tạo thành mô-đun pin năng lượng mặt trời (Solar Module), sau đó tiếp tục ghép thành các mảng pin (Solar Array) nhằm đáp ứng yêu cầu về điện áp và công suất của từng hệ thống. Hiệu suất chuyển đổi năng lượng của pin phụ thuộc vào nhiều yếu tố như cường độ bức xạ mặt trời, nhiệt độ môi trường, chất lượng vật liệu chế tạo, mức độ che bóng và đặc biệt là góc chiếu của ánh sáng lên bề mặt tấm pin. Khi ánh sáng chiếu vuông góc với bề mặt pin, khả năng hấp thụ bức xạ và hiệu suất phát điện đạt giá trị cao nhất; ngược lại, khi góc chiếu thay đổi hoặc ánh sáng bị che khuất, lượng điện năng tạo ra sẽ giảm đáng kể. Hiện nay, các loại pin năng lượng mặt trời phổ biến bao gồm pin silic đơn tinh thể (Monocrystalline), pin silic đa tinh thể (Polycrystalline) và pin màng mỏng (Thin-film), trong đó pin đơn tinh thể có hiệu suất cao nhất nhưng giá thành cũng cao hơn so với các loại còn lại. Với những ưu điểm như tuổi thọ cao, chi phí bảo trì thấp, khả năng vận hành ổn định và thân thiện với môi trường, pin năng lượng mặt trời đang được ứng dụng rộng rãi trong các hệ thống điện mặt trời dân dụng, công nghiệp, nông nghiệp, viễn thông và nhiều lĩnh vực khác.

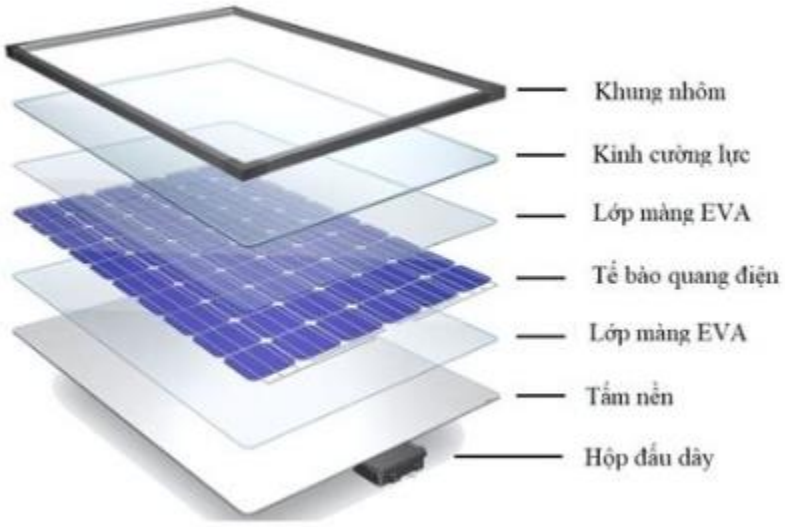
Trong mô hình nghiên cứu, tấm pin năng lượng mặt trời có điện áp định mức 18 V và công suất 30 W được lựa chọn làm nguồn cung cấp điện chính cho toàn bộ hệ thống. Với mức công suất lớn hơn, tấm pin đáp ứng tốt nhu cầu cấp nguồn cho các thiết bị trong mô hình nghiên cứu, đồng thời tạo điều kiện thuận lợi cho việc khảo sát, đánh giá và so sánh hiệu quả của cơ cấu điều hướng trong các điều kiện chiếu sáng khác nhau. Việc sử dụng tấm pin có thông số 18 V – 30 W giúp mô hình tạo ra lượng điện năng lớn hơn, nâng cao độ tin cậy của các kết quả đo đạc về điện áp, dòng điện và công suất trong quá trình thử nghiệm. Nhờ đó, hệ thống có thể đánh giá chính xác hơn khả năng cải thiện hiệu suất thu nhận năng lượng của cơ cấu điều hướng hai trục, đồng thời tạo nền tảng cho việc nghiên cứu, phát

triển và mở rộng ứng dụng cho các hệ thống điện mặt trời có quy mô lớn hơn trong tương lai.



Hình 2.2 Thông số của tấm pin năng lượng mặt trời

2.1.2 Cấu tạo của tấm pin năng lượng mặt trời



Hình 2.3 Cấu tạo của tấm pin năng lượng mặt trời

Tấm pin năng lượng mặt trời được cấu tạo từ nhiều lớp vật liệu khác nhau, mỗi lớp đảm nhận một chức năng riêng nhằm bảo vệ các tế bào quang điện và nâng cao hiệu suất chuyển đổi năng lượng. Theo hình minh họa, cấu tạo của tấm pin bao gồm:

- Khung nhôm (Aluminum Frame):

Khung nhôm là lớp bao ngoài của tấm pin, có chức năng gia cố kết cấu, bảo vệ các lớp bên trong khỏi tác động cơ học và tạo độ cứng vững cho toàn bộ tấm pin. Ngoài ra, khung nhôm còn giúp việc lắp đặt lên giá đỡ hoặc mái nhà trở nên dễ dàng và tăng khả năng chống ăn mòn trong điều kiện môi trường ngoài trời.

- Kính cường lực (Tempered Glass):

Đây là lớp bảo vệ nằm phía trên các tế bào quang điện. Kính cường lực có độ trong suốt cao, cho phép ánh sáng mặt trời truyền qua với tỷ lệ lớn, đồng thời có khả năng chịu lực, chống va đập, chống trầy xước và bảo vệ tấm pin khỏi mưa, gió, bụi bẩn cũng như các tác động từ môi trường.

- Lớp màng EVA (Ethylene Vinyl Acetate):

Lớp EVA phía trên có nhiệm vụ liên kết kính cường lực với các tế bào quang điện, cố định vị trí của các tế bào và bảo vệ chúng khỏi hơi ẩm, bụi bẩn cũng như rung động trong quá trình sử dụng. Lớp vật liệu này có độ trong suốt cao nên không làm giảm lượng ánh sáng truyền đến các tế bào quang điện.

- Tế bào quang điện (Solar Cells):

Đây là bộ phận quan trọng nhất của tấm pin, có chức năng chuyển đổi năng lượng ánh sáng mặt trời thành điện năng thông qua hiệu ứng quang điện. Các tế bào quang điện thường được chế tạo từ silic đơn tinh thể hoặc đa tinh thể và được kết nối nối tiếp hoặc song song để tạo ra điện áp và công suất mong muốn.

- Lớp màng EVA dưới:

Lớp EVA thứ hai nằm bên dưới các tế bào quang điện, có chức năng bao bọc và cố định các tế bào từ phía dưới, tạo thành một khối thống nhất với lớp EVA phía trên. Đồng thời,

lớp này giúp chống ẩm, giảm rung động và bảo vệ tế bào quang điện khỏi các tác động cơ học.

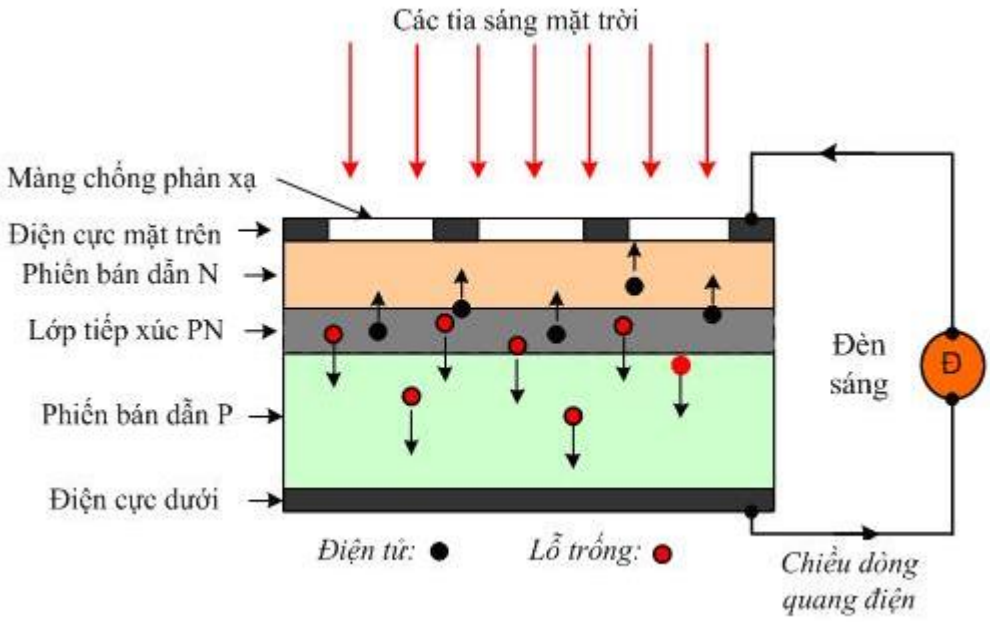
- Tấm nền (Backsheet):

Tấm nền là lớp ngoài cùng ở mặt sau của tấm pin, thường được chế tạo từ vật liệu polymer có khả năng cách điện và chống chịu thời tiết tốt. Lớp này có nhiệm vụ bảo vệ các linh kiện bên trong khỏi độ ẩm, tia cực tím (UV), bụi bẩn và các yếu tố môi trường, đồng thời tăng độ bền và tuổi thọ của tấm pin.

- Hộp đấu dây (Junction Box):

Hộp đấu dây được gắn ở mặt sau của tấm pin, là nơi tập trung các đầu nối điện của các chuỗi tế bào quang điện. Bên trong hộp đấu dây thường tích hợp các diode bypass nhằm hạn chế hiện tượng điểm nóng (Hot Spot) và giảm tổn thất công suất khi một phần tấm pin bị che bóng. Hộp đấu dây cũng là vị trí kết nối tấm pin với hệ thống điện bên ngoài.

2.1.3 Nguyên lí hoạt động của tấm pin năng lượng mặt trời



Hình 2.4 Nguyên lí hoạt động của tấm pin năng lượng mặt trời

Pin năng lượng mặt trời hoạt động dựa trên hiện tượng quang điện xảy ra tại lớp tiếp giáp P–N của vật liệu bán dẫn. Khi ánh sáng mặt trời chiếu lên bề mặt tấm pin, các tia sáng sẽ

đi qua lớp màng chống phản xạ để giảm lượng ánh sáng bị phản xạ và tăng khả năng hấp thụ ánh sáng của tế bào quang điện. Năng lượng của các photon trong ánh sáng sẽ truyền cho các electron trong vật liệu bán dẫn. Nếu năng lượng photon đủ lớn, các electron sẽ tách khỏi liên kết nguyên tử, tạo thành các cặp electron – lỗ trống.

Dưới tác dụng của điện trường bên trong lớp tiếp giáp P–N, các electron sẽ di chuyển về phía lớp bán dẫn N, trong khi các lỗ trống sẽ di chuyển về phía lớp bán dẫn P. Sự dịch chuyển có hướng của các hạt mang điện này tạo ra một hiệu điện thế giữa hai điện cực của pin năng lượng mặt trời.

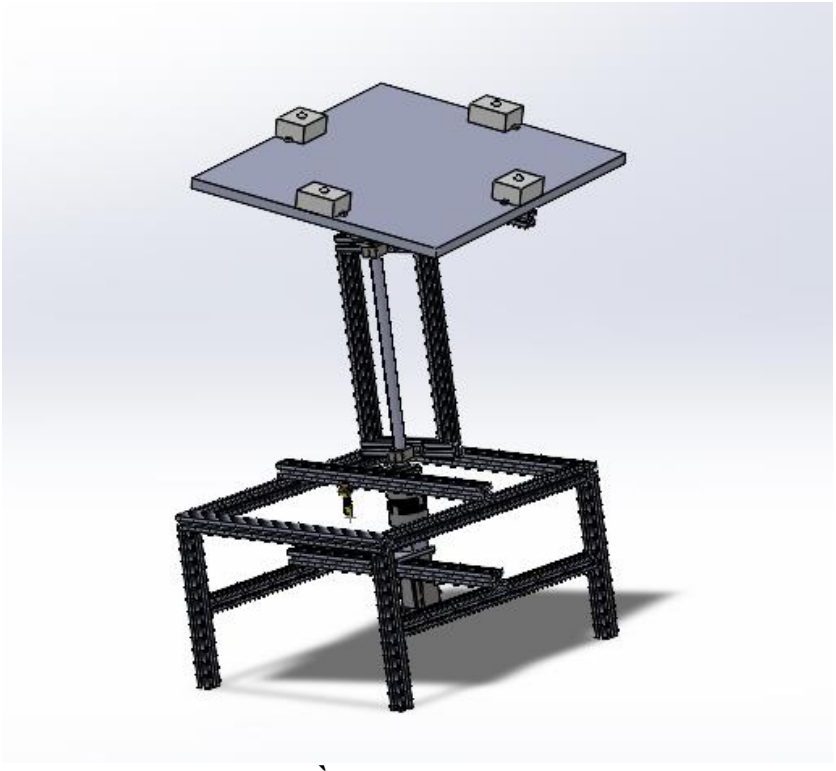
Khi hai điện cực được nối với một tải ngoài, chẳng hạn như bóng đèn trong hình minh họa, các electron sẽ di chuyển từ cực N qua mạch ngoài đến tải, thực hiện công và sau đó quay trở lại cực P. Quá trình chuyển động liên tục của các electron trong mạch ngoài tạo thành dòng điện một chiều (DC), cung cấp năng lượng cho các thiết bị điện.

Trong quá trình hoạt động, lớp điện cực mặt trên có nhiệm vụ thu nhận các electron sinh ra trên bề mặt tế bào quang điện, còn điện cực mặt dưới đóng vai trò hoàn thành mạch điện và dẫn dòng điện ra ngoài. Lớp màng chống phản xạ giúp tăng lượng ánh sáng đi vào tế bào quang điện, từ đó nâng cao hiệu suất chuyển đổi năng lượng của tấm pin.

Hiệu suất hoạt động của pin năng lượng mặt trời phụ thuộc vào nhiều yếu tố như cường độ bức xạ mặt trời, góc chiếu của ánh sáng, nhiệt độ môi trường và chất lượng vật liệu bán dẫn. Khi ánh sáng chiếu vuông góc với bề mặt tấm pin và cường độ bức xạ lớn, số lượng electron được giải phóng sẽ nhiều hơn, làm tăng điện áp, dòng điện và công suất đầu ra của tấm pin. Ngược lại, khi ánh sáng yếu hoặc góc chiếu không thuận lợi, lượng điện năng tạo ra sẽ giảm đáng kể. Chính vì vậy, trong các hệ thống điều hướng pin năng lượng mặt trời, việc tự động điều chỉnh hướng của tấm pin để luôn vuông góc với tia sáng mặt trời sẽ giúp tăng lượng bức xạ hấp thụ, từ đó nâng cao hiệu suất phát điện so với các hệ thống lắp đặt cố định.

2.2 Hệ thống hướng nắng

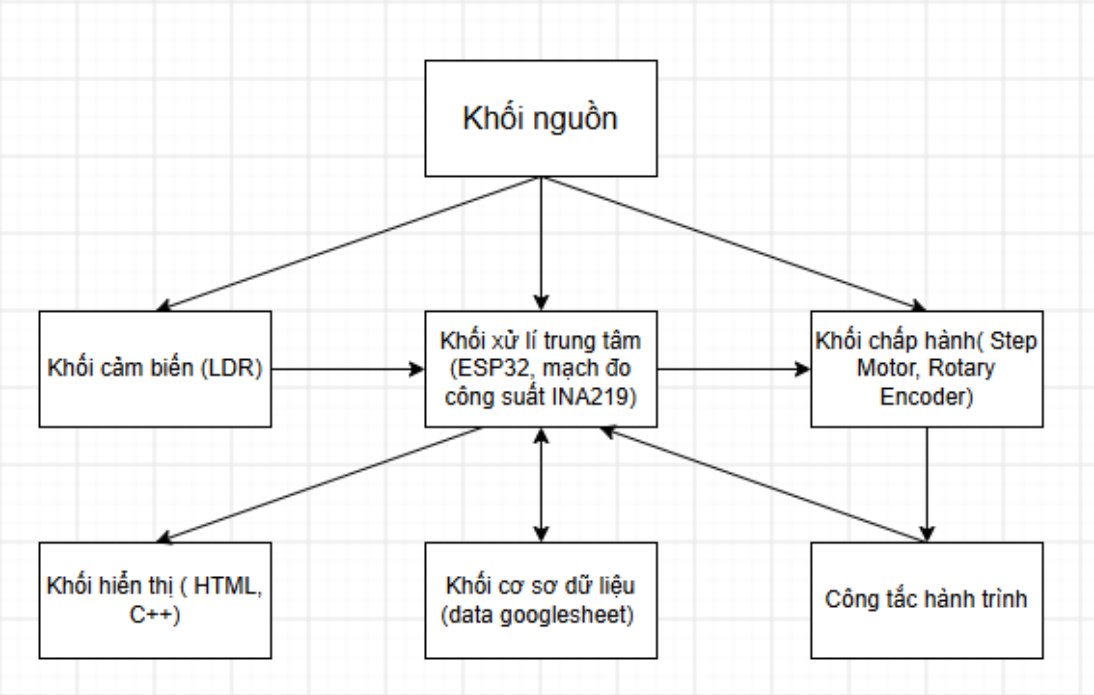
Trong những năm gần đây, công nghệ điều hướng bám mặt trời (Solar Tracker) đã được nghiên cứu và ứng dụng rộng rãi tại nhiều quốc gia nhằm nâng cao hiệu quả khai thác năng lượng mặt trời. Tùy theo cơ cấu chuyển động, các hệ thống này được chia thành hai loại chính: hệ thống điều hướng một trục (Single-Axis Solar Tracker) và hệ thống điều hướng hai trục (Dual-Axis Solar Tracker). Hệ thống một trục cho phép tấm pin quay theo một phương, trong khi hệ thống hai trục có khả năng điều chỉnh đồng thời theo cả phương ngang và phương đứng, giúp tấm pin luôn hướng gần vuông góc với tia bức xạ mặt trời. So với các hệ thống pin năng lượng mặt trời lắp đặt cố định, hệ thống Solar Tracker có khả năng tăng lượng bức xạ hấp thụ và nâng cao hiệu suất phát điện, với mức cải thiện thường đạt trên 20% trong cùng điều kiện chiếu sáng.



Hình 2.5 Mô hình điều hướng pin năng lượng mặt trời

Mỗi loại hệ thống điều hướng pin năng lượng mặt trời đều được thiết kế với các cơ cấu truyền động riêng để thực hiện việc thay đổi vị trí của tấm pin theo hướng bức xạ mặt trời.

Tùy theo phương án thiết kế, các cơ cấu này sẽ có sự khác biệt về độ chính xác, tính ổn định, kết cấu cơ khí, chi phí chế tạo cũng như mức độ phức tạp trong quá trình điều khiển và bảo trì. Do đó, việc lựa chọn cơ cấu truyền động cần được cân nhắc dựa trên yêu cầu kỹ thuật, điều kiện vận hành và hiệu quả kinh tế nhằm đáp ứng tốt nhất mục tiêu của hệ thống. Trong phạm vi đề tài, hệ thống điều hướng hai trục được lựa chọn để triển khai vì đáp ứng tốt yêu cầu về khả năng bám theo quỹ đạo chuyển động của Mặt Trời. Nhờ có thể điều chỉnh đồng thời theo cả phương ngang và phương đứng, tấm pin luôn duy trì góc tiếp nhận bức xạ gần vuông góc với tia sáng trong suốt thời gian hoạt động. Điều này giúp tăng lượng bức xạ hấp thụ, nâng cao hiệu suất phát điện và cải thiện độ chính xác của quá trình điều hướng. Bên cạnh đó, việc sử dụng cơ cấu điều hướng hai trục còn góp phần nâng cao mức độ tự động hóa của hệ thống, phù hợp với mục tiêu nghiên cứu và xây dựng mô hình điều hướng pin năng lượng mặt trời có hiệu quả khai thác năng lượng cao.



Hình 2.6 Sơ đồ khối

2.3 Khối xử lí trung tâm

2.3.1 ESP 32

ESP32 là một vi điều khiển 32 bit do Espressif Systems phát triển, được tích hợp sẵn kết nối Wi-Fi 802.11 b/g/n và Bluetooth (Bluetooth Classic và Bluetooth Low Energy - BLE) trên cùng một chip. Nhờ khả năng kết nối không dây, hiệu năng xử lý cao, mức tiêu thụ năng lượng thấp và giá thành hợp lý, ESP32 được ứng dụng rộng rãi trong các hệ thống Internet of Things (IoT), tự động hóa, nhà thông minh, giám sát môi trường và điều khiển công nghiệp.

ESP32 được trang bị bộ vi xử lý Xtensa® LX6 lõi kép với xung nhịp lên đến 240 MHz, cùng bộ nhớ SRAM tích hợp và khả năng mở rộng bộ nhớ Flash. Ngoài ra, vi điều khiển này còn hỗ trợ nhiều chuẩn giao tiếp ngoại vi cho phép kết nối linh hoạt với nhiều loại cảm biến, module truyền thông và thiết bị chấp hành. Với số lượng chân I/O lớn và khả năng xử lý đa nhiệm, ESP32 đáp ứng tốt các ứng dụng yêu cầu thu thập dữ liệu, điều khiển thời gian thực và truyền dữ liệu qua mạng.



Hình 2.7 ESP32 DevKit V1

Một trong những ưu điểm nổi bật của ESP32 là khả năng kết nối Internet trực tiếp thông qua mạng Wi-Fi mà không cần sử dụng thêm module truyền thông bên ngoài. Điều này giúp đơn giản hóa thiết kế phần cứng, giảm chi phí và tăng tính linh hoạt khi xây dựng các

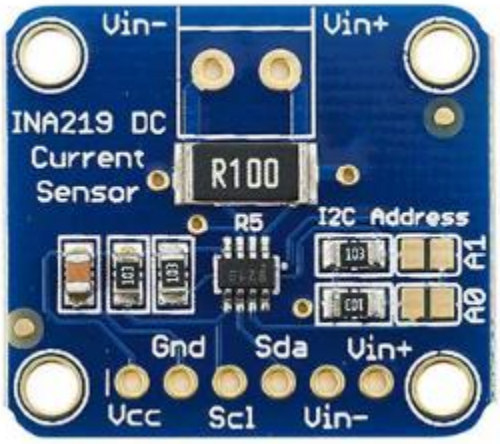
hệ thống IoT. Bên cạnh đó, ESP32 còn hỗ trợ nhiều chế độ tiết kiệm năng lượng như Deep Sleep và Light Sleep, giúp giảm mức tiêu thụ điện trong các ứng dụng sử dụng nguồn pin hoặc năng lượng tái tạo.

Thông số kỹ thuật:

Bộ vi điều khiển	ESP32-D0WDQ6, vi xử lý Xtensa LX6 32-bit lõi kép
Xung nhịp CPU	Lên đến 240 MHz
Bộ nhớ ROM	448 KB
Bộ nhớ SRAM	520 KB
Bộ nhớ Flash	4 MB SPI Flash (tùy phiên bản có thể 8 MB hoặc 16 MB)
Chuẩn Wi-Fi	IEEE 802.11 b/g/n, băng tần 2.4 GHz
Bluetooth	Bluetooth v4.2 BR/EDR và Bluetooth Low Energy (BLE)
Điện áp hoạt động	3.0 – 3.6 VDC (điện áp logic 3.3 V)
Điện áp cấp qua USB	5V
Số chân GPIO	Tối đa 34 GPIO (tùy phiên bản DevKit đưa ra khoảng 30–38 chân)
ADC	ADC 12-bit, tối đa 18 kênh
DAC	2 kênh DAC 8-bit
PWM	Hỗ trợ trên hầu hết các chân GPIO
UART	3 cổng UART
Nhiệt độ hoạt động	-40 °C đến +85 °C
Kích thước module ESP-WROOM-32	Khoảng 25.5 × 18.0 × 3.1 mm
Timer	4 bộ định thời 64-bit
I2C	2 giao tiếp I2C

Bảng 2.1 Thông số kỹ thuật ESP32 DevKit V1

2.3.2 Mạch đo công suất INA219



Hình 2.8 Mạch đo công suất INA219

INA219 là một mạch cảm biến dòng điện và công suất do Texas Instruments phát triển, được sử dụng để đo đồng thời điện áp, dòng điện và công suất tiêu thụ của tải trong các hệ thống điện một chiều (DC). Module này được tích hợp mạch khuếch đại đo dòng có độ chính xác cao cùng bộ chuyển đổi tương tự sang số (ADC) 12 bit, cho phép thu thập dữ liệu với độ ổn định và sai số thấp. Nhờ khả năng giao tiếp thông qua chuẩn I2C (Inter-Integrated Circuit), INA219 có thể kết nối trực tiếp với các vi điều khiển như ESP32, Arduino, STM32 hoặc Raspberry Pi mà không cần sử dụng nhiều chân tín hiệu.

Nguyên lý hoạt động của INA219 dựa trên việc đo điện áp rơi trên điện trở Shunt có giá trị rất nhỏ được mắc nối tiếp với tải. Theo định luật Ohm, khi có dòng điện chạy qua điện trở Shunt sẽ xuất hiện một điện áp rơi tỉ lệ với cường độ dòng điện. INA219 sẽ đo điện áp rơi này, sau đó tính toán giá trị dòng điện dựa trên giá trị điện trở Shunt đã được xác định trước. Đồng thời, module cũng đo điện áp của nguồn cấp (Bus Voltage) và sử dụng hai giá trị điện áp và dòng điện để tính toán công suất tiêu thụ theo công thức:

$$P=U\times I$$

Trong đó:

P: Công suất (W)

U: Điện áp (V)

I: Dòng điện (A)

Nhờ được tích hợp sẵn bộ xử lý bên trong, INA219 có thể trực tiếp xuất ra các giá trị điện áp, dòng điện và công suất dưới dạng số mà không cần thực hiện các phép tính phức tạp trên vi điều khiển. Điều này giúp giảm tải cho bộ xử lý trung tâm và nâng cao độ chính xác của hệ thống đo.

Thông số kỹ thuật:

IC chính	INA219 – Texas Instruments
Điện áp hoạt động	3 V – 5,5 V
Điện áp Bus đo được	0 – 26 VDC
Điện áp Shunt tối đa	±320 mV
Dòng điện đo tối đa (module tiêu chuẩn)	±3,2 A
Độ phân giải ADC	12 bit
Chuẩn giao tiếp	I2C
Địa chỉ I2C mặc định	0x40
Tần số giao tiếp I2C	Lên đến 400 kHz
Độ chính xác	Khoảng ±1%
Công suất tính toán	Điện áp × Dòng điện

Bảng 2.2 Thông số kỹ thuật của mạch đo công suất INA219

2.4 Khối chấp hành

2.4.1 Rotary Encoder HN3806-AB-600G

Rotary Encoder HN3806-AB-600G là cảm biến đo vị trí và tốc độ quay dạng encoder gia tăng (Incremental Rotary Encoder), được sử dụng để chuyển đổi chuyển động quay của trục cơ khí thành các tín hiệu xung điện. Các tín hiệu này được bộ điều khiển xử lý để xác định góc quay, tốc độ quay và chiều quay của động cơ hoặc cơ cấu truyền động. Nhờ độ chính xác cao, khả năng phản hồi nhanh và kết cấu nhỏ gọn, encoder HN3806-AB-600G

được ứng dụng rộng rãi trong các hệ thống điều khiển tự động, robot, máy CNC, băng tải, máy in, máy đóng gói và các thiết bị yêu cầu điều khiển vị trí chính xác.



Hình 2.9 Rotary Encoder

Encoder HN3806-AB-600G hoạt động theo nguyên lý phát sinh các xung điện khi trục quay. Bên trong encoder có một đĩa mã hóa (code disk) gắn đồng trục với trục quay. Khi đĩa quay, cảm biến quang sẽ tạo ra hai tín hiệu xung vuông kênh A và kênh B lệch pha nhau 90° điện (Quadrature Output). Dựa vào số lượng xung thu được, bộ điều khiển có thể tính toán góc quay và tốc độ của trục. Đồng thời, dựa trên thứ tự xuất hiện của hai tín hiệu A và B, hệ thống có thể xác định chiều quay theo chiều kim đồng hồ hoặc ngược chiều kim đồng hồ. Ngoài hai kênh A và B, encoder còn có kênh Z (Index), phát ra một xung duy nhất sau mỗi vòng quay, được sử dụng làm tín hiệu gốc để xác định vị trí chuẩn.

encoder có độ phân giải 600 xung trên một vòng quay (600 PPR – Pulses Per Revolution). Điều này có nghĩa là khi trục encoder quay hết một vòng 360°, encoder sẽ tạo ra 600 xung

trên mỗi kênh A hoặc B. Nếu bộ điều khiển sử dụng phương pháp đọc cả sườn lên và sườn xuống của hai kênh (x4 decoding), độ phân giải có thể đạt 2.400 xung/vòng, giúp tăng đáng kể độ chính xác trong việc xác định vị trí.

Thông số kỹ thuật:

Model	HN3806-AB-600G
Loại encoder	Incremental Rotary Encoder
Điện áp hoạt động	5 VDC
Độ phân giải	600 xung/vòng (600 PPR)
Ngõ ra	A, B, Z
Góc lệch pha A và B	90° ± 30° điện
Tín hiệu đầu ra	NPN Open Collector hoặc Push-Pull
Tốc độ quay tối đa	Khoảng 5.000–6.000 vòng/phút
Đường kính thân	38 mm
Đường kính trục	6 mm
Nhiệt độ làm việc	-20 °C đến +70 °C

Bảng 2.3 Thông số kỹ thuật của Rotary Encoder HN3806-AB-600g

Ứng dụng của Rotary Encoder HN3806-AB-600g

Rotary Encoder HN3806-AB-600G được sử dụng để đo và phản hồi chính xác vị trí quay của cơ cấu điều hướng theo hai trục. Encoder được lắp đồng trục với động cơ bước hoặc cơ cấu truyền động để ghi nhận số xung phát sinh trong quá trình quay, từ đó xác định góc quay thực tế của tấm pin năng lượng mặt trời.

Các tín hiệu xung từ encoder được truyền về ESP32 để xử lý và tính toán vị trí hiện tại của hệ thống. Dữ liệu này được sử dụng làm tín hiệu phản hồi (Feedback) trong thuật toán điều khiển PID, giúp so sánh giữa góc quay thực tế và góc quay mong muốn. Khi xuất hiện sai số, ESP32 sẽ điều chỉnh tín hiệu điều khiển gửi đến driver TB6600 để động cơ bước NEMA57 tiếp tục quay hoặc hiệu chỉnh vị trí cho đến khi tấm pin đạt đúng góc yêu cầu.

Ngoài chức năng xác định vị trí, encoder còn giúp phát hiện hiện tượng mất bước của động cơ bước do tải lớn, ma sát cơ khí hoặc tác động từ môi trường như gió. Thông qua việc so sánh số xung điều khiển với số xung phản hồi từ encoder, hệ thống có thể nhận biết sai lệch vị trí và thực hiện bù sai số, góp phần nâng cao độ chính xác và độ ổn định trong quá trình điều hướng.

Việc sử dụng Rotary Encoder HN3806-AB-600G còn giúp hệ thống chuyển từ phương pháp điều khiển vòng hở (Open-loop) sang điều khiển vòng kín (Closed-loop). Điều này không chỉ nâng cao độ tin cậy của hệ thống mà còn đảm bảo tấm pin luôn được định hướng chính xác theo hướng có cường độ bức xạ mặt trời lớn nhất. Nhờ đó, hiệu suất thu nhận năng lượng được cải thiện, đồng thời tăng khả năng hoạt động ổn định của mô hình trong các điều kiện vận hành khác nhau.

2.4.2 Động cơ Step 57

Động cơ bước 57 (NEMA 23 Stepper Motor) là một loại động cơ điện hoạt động theo nguyên lý biến đổi các xung điện điều khiển thành những bước quay có góc xác định. Tên gọi NEMA 23 được đặt theo tiêu chuẩn của Hiệp hội các nhà sản xuất thiết bị điện quốc gia Hoa Kỳ (National Electrical Manufacturers Association - NEMA), trong đó số 57 biểu thị kích thước mặt bích của động cơ là 57 × 57 mm. Khác với động cơ DC thông thường, động cơ bước không quay liên tục khi được cấp nguồn mà quay theo từng bước với góc quay cố định, giúp điều khiển chính xác vị trí, tốc độ và chiều quay của trục động cơ. ộng cơ NEMA 23 thường có góc bước 1,8°/bước, tương ứng 200 bước cho một vòng quay 360°, và có thể tăng độ phân giải khi kết hợp với bộ điều khiển vi bước (Microstepping Driver). Nhờ khả năng định vị chính xác, mô-men xoắn lớn ở tốc độ thấp, độ tin cậy cao và dễ dàng điều khiển bằng các vi điều khiển như ESP32 hoặc Arduino, động cơ bước NEMA23 được ứng dụng rộng rãi trong các hệ thống tự động hóa, máy CNC, máy in 3D, robot công nghiệp và các hệ thống điều hướng pin năng lượng mặt trời. Trong đề tài "Thiết kế và thi công mô hình điều hướng pin năng lượng mặt trời", động cơ bước NEMA23 được

sử dụng làm cơ cấu truyền động cho hệ thống điều hướng hai trục, giúp điều chỉnh chính xác góc quay của tấm pin theo hướng bức xạ mặt trời nhằm nâng cao hiệu suất thu nhận năng lượng và đảm bảo hệ thống hoạt động ổn định.



Hình 2.10 Động cơ Step Nema23

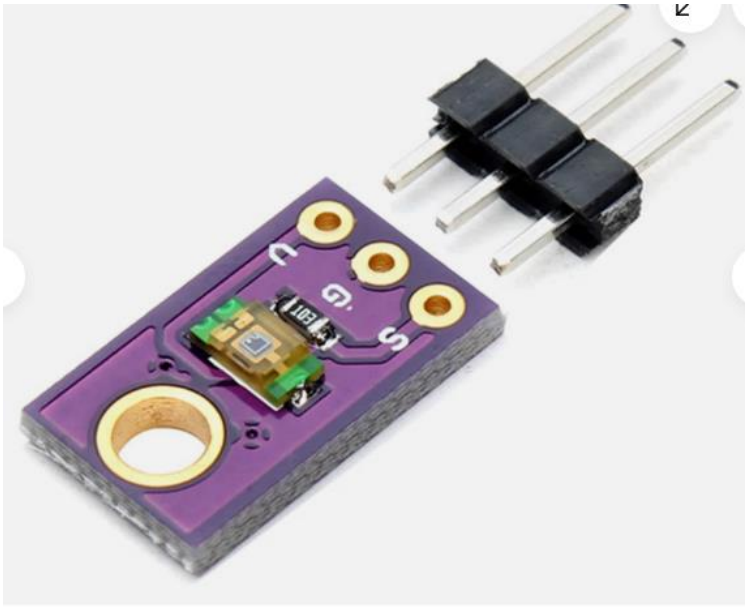
Thông số kỹ thuật:

Kích thước mặt bích	57 × 57 mm
Góc bước	1,8°/bước
Số bước/vòng	200 bước
Số pha	2 pha
Điện áp định mức	2,8 – 4,2 V
Dòng điện mỗi pha	2 – 4,2 A
Mô-men giữ	2.3 N·m
Trọng lượng	Khoảng 0,8 – 1,5 kg

Bảng 2.4 Thông số kỹ thuật của động cơ Step Nema23

2.5 Cảm biến ánh sáng TEMT6000 (LDR)

Cảm biến ánh sáng TEMT6000 là cảm biến quang sử dụng phototransistor NPN có độ nhạy cao với ánh sáng nhìn thấy, được sản xuất bởi Vishay Semiconductors. Cảm biến được thiết kế để mô phỏng gần với độ nhạy của mắt người, do đó có khả năng đo cường độ ánh sáng môi trường một cách chính xác và ổn định. Khác với quang trở (LDR), TEMT6000 có thời gian đáp ứng nhanh hơn, độ tuyến tính tốt hơn và ít chịu ảnh hưởng của nhiệt độ, giúp nâng cao độ chính xác trong các ứng dụng đo ánh sáng.



Hình 2.11 Cảm biến ánh sáng TEMT6000 (LDR)

LDR có cấu tạo đơn giản, giá thành thấp, độ nhạy cao với ánh sáng nhìn thấy và dễ dàng kết nối với các vi điều khiển ESP32 thông qua mạch phân áp. Khi hoạt động, sự thay đổi điện trở của LDR sẽ làm thay đổi điện áp tại đầu ra của mạch phân áp. Vi điều khiển sẽ đọc giá trị điện áp này thông qua bộ chuyển đổi tương tự sang số (ADC), từ đó xác định mức độ chiếu sáng của môi trường và đưa ra các lệnh điều khiển phù hợp.

Trong đề tài,. bốn cảm biến TEMT6000 được bố trí tại bốn vị trí trên cụm cảm biến của tấm pin để đo cường độ ánh sáng từ các hướng khác nhau. ESP32 sẽ thu thập và so sánh giá trị từ các cảm biến nhằm xác định hướng có cường độ bức xạ lớn nhất, sau đó điều

khiển động cơ bước quay theo hai trục để tấm pin luôn hướng về phía Mặt Trời, góp phần nâng cao hiệu suất thu nhận năng lượng.

Thông số kỹ thuật:

Tên cảm biến	TEMT6000 Ambient Light Sensor
Loại cảm biến	Phototransistor NPN
Vật liệu chế tạo	Cadmium Sulfide (CdS)
Điện áp hoạt động (Module)	3.3 V – 5 VDC
Điện áp đầu ra	Analog (0 – VCC)
Điện trở khi có ánh sáng mạnh	5 – 10 kΩ
Điện trở trong bóng tối	≥ 1 MΩ
Bước sóng nhạy sáng	360 – 970 nm
Thời gian đáp ứng	Khoảng 20 ms (tăng), 30 ms (giảm)
Nhiệt độ hoạt động	-40 °C đến +85 °C
Công suất tiêu tán tối đa	100 mW
Kích thước đầu cảm biến	5 mm

Bảng 2.5 Thông số kỹ thuật của cảm biến ánh sáng (LDR)

2.6 Khối nguồn:

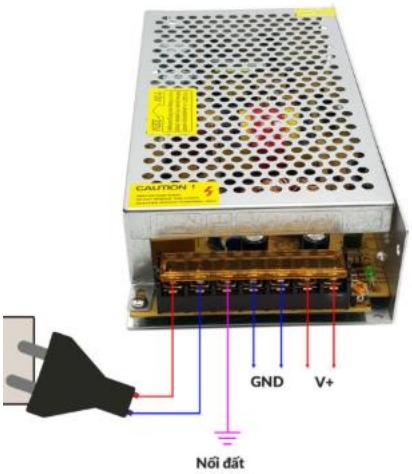
2.6.1 Nguồn tổ ong 24V

Nguồn tổ ong 24VDC (Switching Power Supply 24VDC) là bộ nguồn chuyển mạch có chức năng chuyển đổi điện áp xoay chiều (AC) từ lưới điện thành điện áp một chiều (DC) ổn định để cung cấp cho các thiết bị điện và điện tử. Tên gọi "nguồn tổ ong" xuất phát từ thiết kế vỏ kim loại có nhiều lỗ thông gió dạng tổ ong, giúp tản nhiệt hiệu quả và tăng độ bền của thiết bị trong quá trình hoạt động.

Khác với các bộ nguồn tuyến tính truyền thống, nguồn tổ ong sử dụng công nghệ nguồn xung (Switching Mode Power Supply - SMPS), cho hiệu suất chuyển đổi cao, kích thước nhỏ gọn, trọng lượng nhẹ và ít sinh nhiệt. Nhờ khả năng cung cấp điện áp đầu ra ổn định,

nguồn tổ ong được sử dụng rộng rãi trong các hệ thống điều khiển tự động, PLC, động cơ bước, camera giám sát, đèn LED, máy CNC và các ứng dụng IoT.

Nguồn tổ ong **24VDC** được sử dụng để cung cấp nguồn điện cho driver điều khiển động cơ bước và các thiết bị ngoại vi của hệ thống. Nguồn 24VDC đảm bảo điện áp ổn định, đáp ứng yêu cầu công suất của động cơ, giúp hệ thống vận hành liên tục và nâng cao độ tin cậy trong quá trình thử nghiệm.



Hình 2.11 Nguồn tổ ong 24V-5A

Thông số kỹ thuật:

Loại nguồn	Switching Power Supply (SMPS)
Điện áp đầu vào	100 – 240 VAC, 50/60 Hz
Điện áp đầu ra	24 VDC
Sai số điện áp đầu ra	±1%
Dòng điện đầu ra	5A
Công suất đầu ra	120W
Hiệu suất	80 – 90%
Tần số chuyển mạch	Khoảng 50 – 100 kHz
Nhiệt độ hoạt động	-10 °C đến +60 °C

Bảng 2.6 Thông số kỹ thuật nguồn tổ ong 24V-5A

2.6.2 Driver TB6600

Driver TB6600 là mạch điều khiển chuyên dụng dành cho động cơ bước 2 pha (Bipolar Stepper Motor), có nhiệm vụ nhận tín hiệu điều khiển từ vi điều khiển như ESP32, Arduino hoặc PLC để điều khiển chiều quay, tốc độ và vị trí của động cơ bước. Driver sử dụng phương pháp điều khiển dòng điện không đổi (Constant Current Control), giúp động cơ hoạt động ổn định, giảm rung, tăng mô-men xoắn và bảo vệ cuộn dây động cơ khỏi hiện tượng quá dòng. TB6600 hỗ trợ nhiều chế độ vi bước (Microstepping), cho phép tăng độ phân giải của động cơ và cải thiện độ chính xác trong các ứng dụng điều khiển vị trí. Ngoài ra, mạch còn tích hợp các chức năng bảo vệ như quá dòng, quá áp và quá nhiệt, giúp nâng cao độ an toàn và tuổi thọ của hệ thống. Nhờ khả năng điều khiển động cơ có dòng lớn, TB6600 được sử dụng rộng rãi trong các máy CNC, máy khắc laser, robot, băng tải tự động và các hệ thống điều hướng pin năng lượng mặt trời.

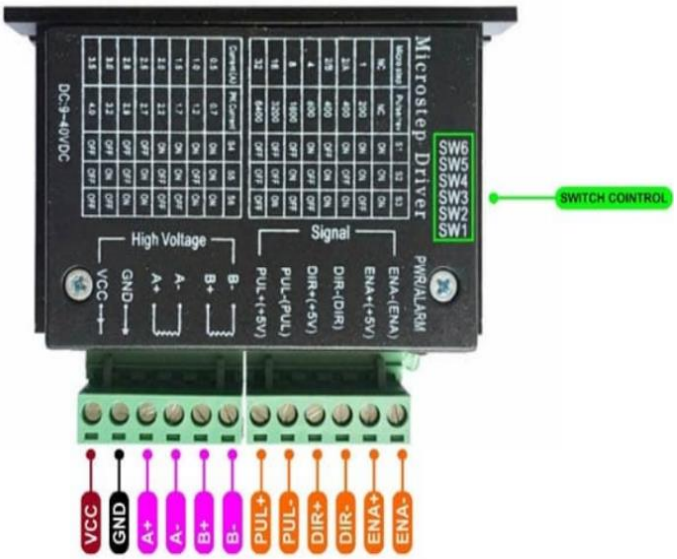


Hình 2.12 Driver TB6600

Thông số kỹ thuật:

Thông số	Giá trị
Model	TB6600
Loại driver	Driver điều khiển động cơ bước 2 pha
Điện áp cấp	9 – 40 VDC (một số phiên bản hỗ trợ đến 42 VDC)
Dòng điện đầu ra	0,5 – 4,0 A (điều chỉnh bằng công tắc DIP)
Công suất tối đa	Khoảng 160 W
Động cơ hỗ trợ	Động cơ bước 2 pha (NEMA 17, NEMA 23, NEMA 24...)
Góc vi bước	Full, 1/2, 1/4, 1/8, 1/16, 1/32 (tùy phiên bản)
Tín hiệu điều khiển	PUL (Step), DIR (Direction), ENA (Enable)
Mức tín hiệu điều khiển	5V TTL (nhiều phiên bản tương thích 5–24 V nhờ cách ly quang)
Tần số xung tối đa	Khoảng 20 – 50 kHz (tùy phiên bản)
Cách ly tín hiệu	Opto-coupler (cách ly quang)
Bảo vệ	Quá dòng, quá áp, quá nhiệt, ngắn mạch
Nhiệt độ hoạt động	-10 °C đến +45 °C
Kích thước	Khoảng 96 × 56 × 33 mm

Bảng 2.7 Thông số kỹ thuật của Driver TB6600



Hình 2.13 Sơ đồ chân của Driver TB6600

Ứng dụng trong hệ thống điều hướng pin năng lượng mặt trời:

Trong hệ thống điều hướng pin năng lượng mặt trời, driver TB6600 được sử dụng để điều khiển các động cơ Step quay theo ý muốn. Nhờ khả năng điều khiển chính xác, cấu hình linh hoạt và độ ổn định cao, driver TB6600 đáp ứng tốt yêu cầu điều khiển của hệ thống trong phạm vi đồ án.

2.7 Khối hiển thị

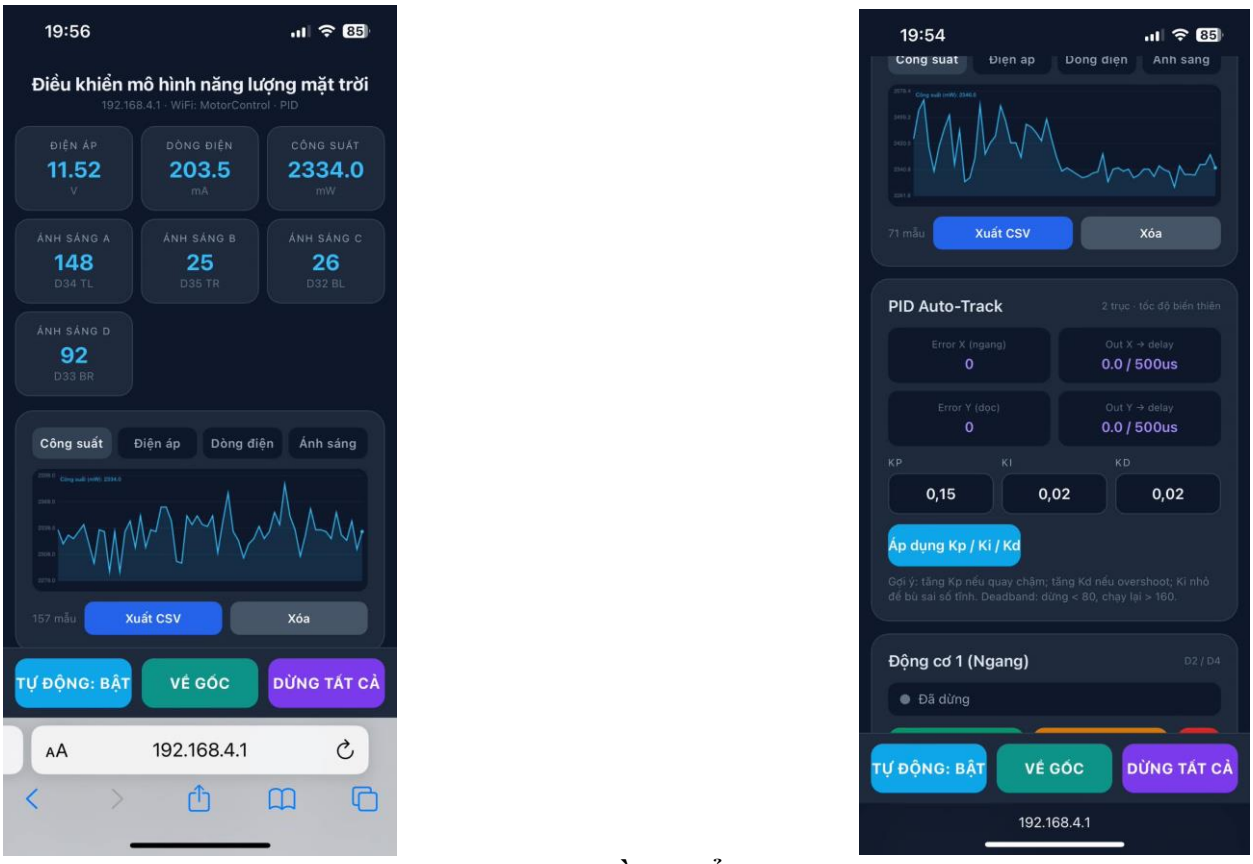
2.7.1 Ngôn ngữ lập trình

1. HTML

HTML (HyperText Markup Language) là ngôn ngữ đánh dấu siêu văn bản được sử dụng để xây dựng cấu trúc và nội dung của các trang web. HTML do Tim Berners-Lee phát triển vào năm 1991 và hiện được quản lý, phát triển bởi World Wide Web Consortium (W3C) cùng WHATWG (Web Hypertext Application Technology Working Group). Mặc dù thường được gọi là ngôn ngữ lập trình, HTML thực chất là một ngôn ngữ đánh dấu (Markup Language), có chức năng mô tả cấu trúc và cách tổ chức nội dung trên trang web thông qua các thẻ (Tags).

HTML sử dụng các phần tử như tiêu đề, đoạn văn, bảng biểu, hình ảnh, liên kết, biểu mẫu và các đối tượng đa phương tiện để xây dựng giao diện hiển thị. Các trình duyệt web như Google Chrome, Microsoft Edge, Firefox hoặc Safari sẽ đọc các thẻ HTML và hiển thị nội dung theo đúng cấu trúc mà người lập trình thiết kế. Để tạo nên một trang web hoàn chỉnh, HTML thường được kết hợp với CSS (Cascading Style Sheets) nhằm định dạng giao diện và JavaScript để bổ sung các chức năng tương tác động.

Nhờ cú pháp đơn giản, dễ học và khả năng tương thích với hầu hết các trình duyệt hiện nay, HTML được sử dụng rộng rãi trong phát triển website, ứng dụng web, hệ thống quản lý dữ liệu và các nền tảng giám sát trực tuyến. Phiên bản phổ biến hiện nay là **HTML5**, hỗ trợ nhiều tính năng mới như hiển thị đồ họa, âm thanh, video và khả năng tương thích tốt với các thiết bị di động cũng như các ứng dụng Internet of Things (IoT).



Hình 2.14 Giao diện điều khiển trên điện thoại

HTML được sử dụng để xây dựng giao diện web phục vụ việc giám sát và quản lý hệ thống. Giao diện được thiết kế để hiển thị các thông số hoạt động của mô hình như cường độ ánh sáng, điện áp, dòng điện, công suất của tấm pin và trạng thái hoạt động của hệ thống theo thời gian thực. Thông qua kết nối Wi-Fi của ESP32, dữ liệu được truyền đến trang web để người dùng có thể theo dõi và giám sát mô hình từ xa bằng máy tính hoặc các thiết bị di động có kết nối Internet. Việc sử dụng HTML giúp giao diện có cấu trúc rõ ràng, dễ sử dụng, đồng thời tạo nền tảng thuận lợi cho việc tích hợp với CSS và JavaScript nhằm nâng cao tính trực quan và khả năng tương tác của hệ thống.

2. Ngôn ngữ lập trình C++

C++ là ngôn ngữ lập trình bậc cao được phát triển bởi Bjarne Stroustrup tại Bell Labs vào năm 1979, dựa trên sự mở rộng của ngôn ngữ lập trình C. C++ được thiết kế nhằm kết hợp giữa khả năng lập trình hướng thủ tục của C và các đặc điểm của lập trình hướng đối tượng (Object-Oriented Programming – OOP), giúp người lập trình xây dựng các chương trình có cấu trúc rõ ràng, dễ bảo trì và có hiệu suất xử lý cao. Nhờ tính linh hoạt, tốc độ thực thi nhanh và khả năng tương tác trực tiếp với phần cứng, C++ hiện là một trong những ngôn ngữ lập trình phổ biến trong các lĩnh vực như hệ thống nhúng, tự động hóa, robot, phát triển phần mềm, trò chơi điện tử và các ứng dụng điều khiển công nghiệp.

C++ hỗ trợ nhiều đặc điểm nổi bật như lập trình hướng đối tượng, lập trình tổng quát (Generic Programming), quản lý bộ nhớ linh hoạt và thư viện chuẩn phong phú (Standard Template Library – STL). Ngôn ngữ này cũng cho phép truy cập trực tiếp đến tài nguyên phần cứng, giúp tối ưu tốc độ xử lý và đáp ứng tốt các ứng dụng yêu cầu điều khiển theo thời gian thực. Ngoài ra, C++ có khả năng tương thích với nhiều hệ điều hành và nền tảng phần cứng khác nhau, từ máy tính cá nhân đến các vi điều khiển và bộ xử lý nhúng.



Hình 2.15 Ngôn ngữ lập trình C++

2.7.2 Bộ điều khiển sạc năng lượng mặt trời 12V

Bộ điều khiển sạc năng lượng mặt trời 12V (Solar Charge Controller 12V) là thiết bị điện tử có chức năng quản lý quá trình nạp năng lượng từ tấm pin mặt trời vào ắc quy hoặc pin lưu trữ, đồng thời cung cấp nguồn điện ổn định cho tải. Thiết bị có nhiệm vụ điều chỉnh điện áp và dòng điện từ tấm pin nhằm đảm bảo quá trình sạc diễn ra an toàn, hiệu quả và kéo dài tuổi thọ của ắc quy. Ngoài ra, bộ điều khiển còn tích hợp các chức năng bảo vệ như chống sạc quá mức, chống xả sâu, chống quá dòng, chống ngắn mạch và chống đấu ngược cực, giúp nâng cao độ tin cậy của toàn bộ hệ thống điện mặt trời.



Hình 2.16 Bộ điều khiển sạc pin năng lượng mặt trời 12V

Trong đề tài, bộ điều khiển sạc 12V được sử dụng để quản lý nguồn điện từ tấm pin mặt trời, thực hiện sạc cho ắc quy 12V và cung cấp nguồn ổn định cho hệ thống điều khiển. Việc sử dụng bộ điều khiển sạc giúp bảo vệ ắc quy, nâng cao hiệu suất sử dụng năng lượng và đảm bảo hệ thống hoạt động liên tục trong quá trình vận hành.

Thông số kỹ thuật:

Thông số	Giá trị
Loại điều khiển	PWM (Pulse Width Modulation)
Điện áp hệ thống	12 VDC
Dòng sạc tối đa	10 A
Điện áp đầu vào từ tấm pin	17 – 25 VDC
Công suất tấm pin hỗ trợ	Tối đa khoảng 120 W (hệ 12V)
Điện áp ngắt sạc	Khoảng 14,4 V
Điện áp duy trì sạc	Khoảng 13,7 V
Điện áp ngắt tải (chống xả sâu)	Khoảng 10,8 – 11,1 V
Hiệu suất	≥ 95%
Nhiệt độ hoạt động	-20 °C đến +60 °C
Chức năng bảo vệ	Quá sạc, quá xả, quá dòng, ngắn mạch, đấu ngược cực

Bảng 2.8 Thông số kĩ thuật của bộ điều khiển sạc năng lượng mặt trời

Nguyên lí hoạt động:

Bộ điều khiển sạc năng lượng mặt trời 12V hoạt động như một thiết bị trung gian giữa tấm pin năng lượng mặt trời, ắc quy lưu trữ và tải tiêu thụ. Chức năng chính của bộ điều khiển là quản lý quá trình sạc và xả của ắc quy, đảm bảo năng lượng từ tấm pin được sử dụng hiệu quả, đồng thời bảo vệ ắc quy và các thiết bị trong hệ thống.

Khi tấm pin năng lượng mặt trời nhận được bức xạ mặt trời, nó sẽ tạo ra dòng điện một chiều (DC). Dòng điện này được đưa vào bộ điều khiển sạc, tại đây điện áp và dòng điện sẽ được giám sát liên tục. Nếu điện áp của ắc quy thấp hơn mức sạc cho phép, bộ điều khiển sẽ cấp dòng điện từ tấm pin để nạp vào ắc quy. Trong quá trình sạc, bộ điều khiển sẽ điều chỉnh dòng điện và điện áp phù hợp để đảm bảo ắc quy được sạc đúng chế độ, tránh hiện tượng quá sạc gây hư hỏng hoặc giảm tuổi thọ.

Khi ắc quy được sạc đầy và điện áp đạt đến ngưỡng cài đặt (thường khoảng 14,4 V đối với hệ thống 12 V), bộ điều khiển sẽ tự động giảm dòng sạc hoặc ngắt quá trình sạc để bảo vệ ắc quy. Khi điện áp giảm xuống dưới mức duy trì, bộ điều khiển sẽ tiếp tục cấp dòng sạc để giữ cho ắc quy luôn ở trạng thái hoạt động ổn định.

Trong trường hợp tải sử dụng nguồn từ ắc quy, bộ điều khiển sẽ theo dõi điện áp của ắc quy. Nếu điện áp giảm xuống dưới ngưỡng cho phép (khoảng 10,8 – 11 V), bộ điều khiển sẽ tự động ngắt tải nhằm tránh hiện tượng xả sâu, giúp kéo dài tuổi thọ của ắc quy. Khi điện áp được phục hồi nhờ quá trình sạc từ tấm pin, tải sẽ được kết nối trở lại.

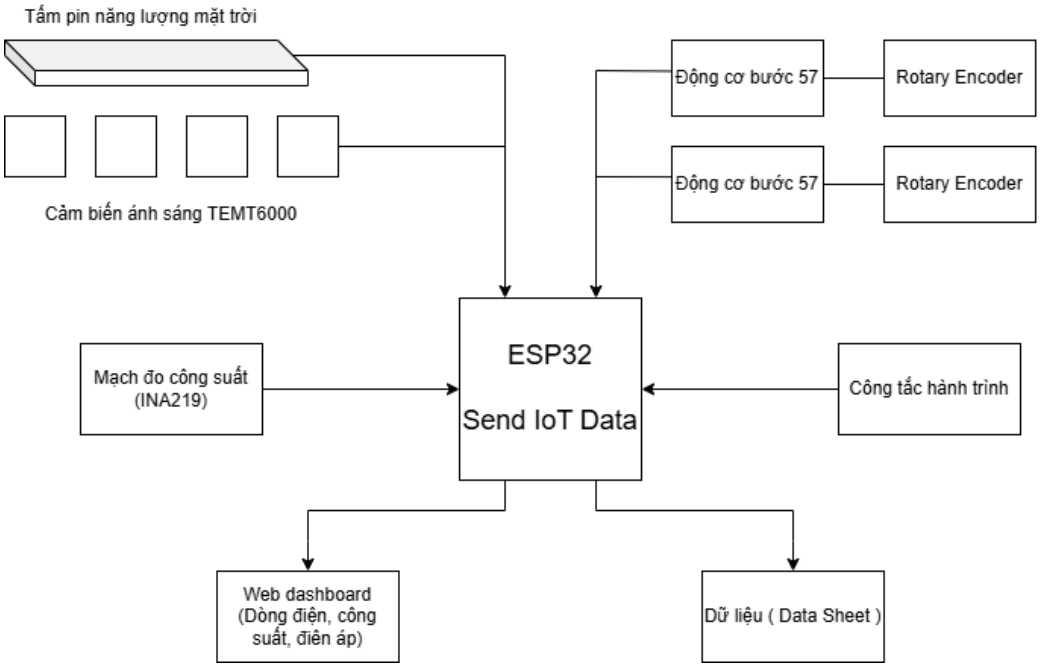
Ngoài chức năng quản lý sạc và xả, bộ điều khiển còn tích hợp các cơ chế bảo vệ như chống đấu ngược cực, chống quá dòng, chống ngắn mạch và chống quá tải, giúp hệ thống vận hành an toàn và ổn định trong nhiều điều kiện làm việc khác nhau.

CHƯƠNG 3. THIẾT KẾ VÀ TÍNH TOÁN

3.1 Thiết kế hệ thống

3.1.1 Sơ đồ khối hệ thống

Để mô tả quá trình vận hành của hệ thống điều hướng pin năng lượng mặt trời hai trục, đề tài xây dựng lưu đồ giải thuật thể hiện toàn bộ quá trình xử lý của vi điều khiển ESP32. Thuật toán bắt đầu từ việc khởi tạo các thiết bị, thu thập dữ liệu từ các cảm biến LDR, INA219, encoder và công tắc hành trình, sau đó xử lý dữ liệu, thực hiện thuật toán điều khiển PID để điều khiển động cơ bước NEMA57. Đồng thời, các thông số điện áp, dòng điện và công suất của tấm pin được cập nhật và truyền lên giao diện Web để người dùng theo dõi theo thời gian thực.



Hình 3.1 Sơ đồ khối tổng quát của hệ thống

Giải thích:

Trong hệ thống, ESP32 đóng vai trò là bộ điều khiển trung tâm, có nhiệm vụ thu thập dữ liệu từ các cảm biến, xử lý thuật toán điều khiển, điều khiển cơ cấu quay của tấm pin và truyền dữ liệu lên giao diện giám sát thông qua kết nối Wi-Fi.

Để xác định hướng có cường độ bức xạ mặt trời lớn nhất, hệ thống sử dụng bốn cảm biến ánh sáng TEMT6000 được bố trí tại bốn vị trí trên tấm pin năng lượng mặt trời. Các cảm biến liên tục đo cường độ ánh sáng ở từng hướng và gửi tín hiệu analog về ESP32. Dựa trên sự chênh lệch giá trị giữa các cảm biến, ESP32 tính toán hướng quay phù hợp nhằm đưa bề mặt tấm pin luôn vuông góc với tia sáng Mặt Trời.

Trong quá trình điều hướng, hệ thống sử dụng hai động cơ bước 57 để điều khiển chuyển động theo hai trục, bao gồm trục phương vị (Azimuth) và trục độ cao (Elevation). Mỗi động cơ được kết hợp với một Rotary Encoder để đo góc quay thực tế và phản hồi vị trí về ESP32. Tín hiệu phản hồi này được sử dụng trong thuật toán điều khiển PID, giúp hiệu chỉnh sai số, phát hiện hiện tượng mất bước và đảm bảo động cơ quay đúng vị trí yêu cầu, từ đó nâng cao độ chính xác và tính ổn định của hệ thống.

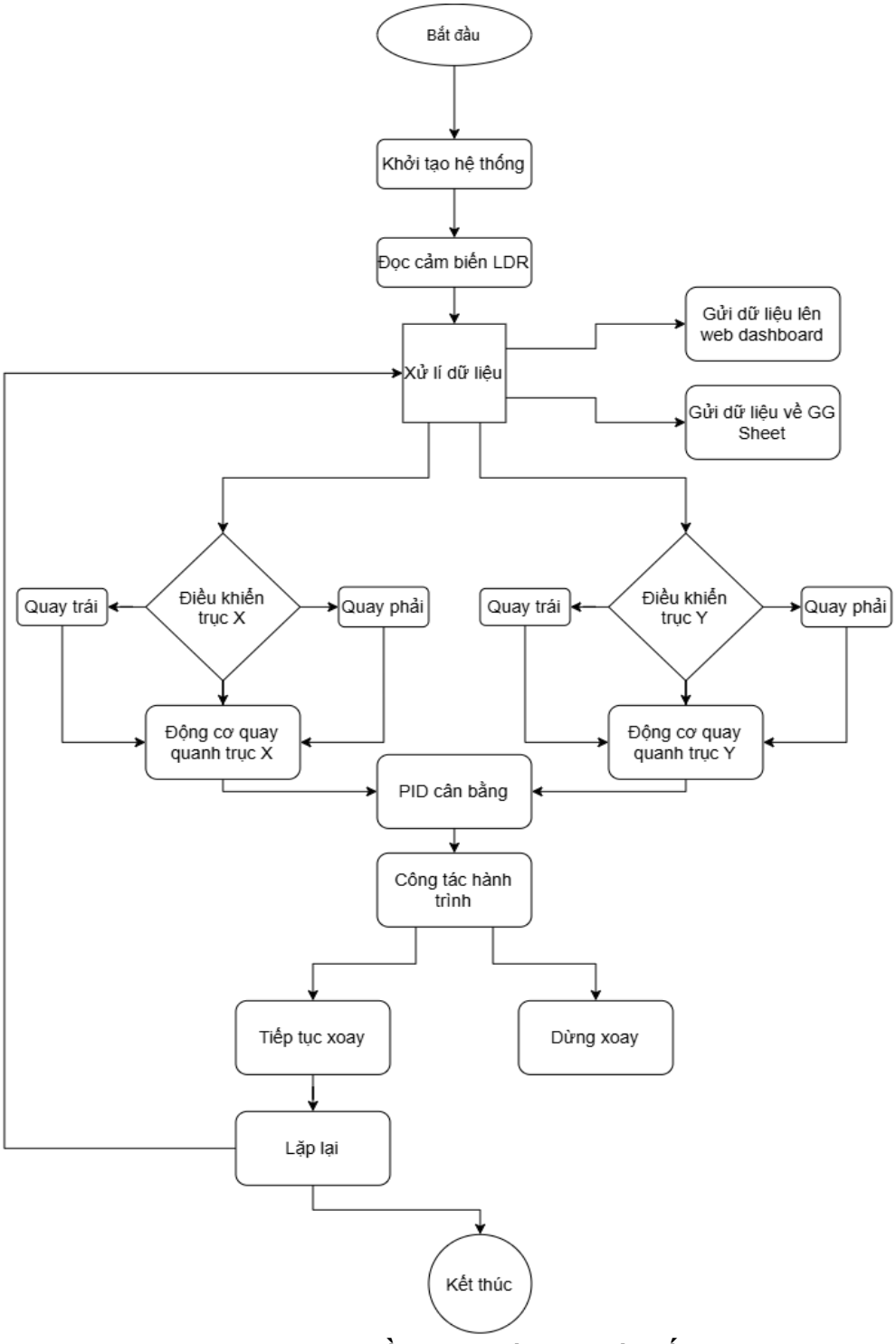
Để bảo vệ cơ cấu truyền động, mỗi trục được lắp đặt công tắc hành trình nhằm giới hạn phạm vi chuyển động của động cơ. Khi cơ cấu quay đến vị trí giới hạn, công tắc hành trình sẽ gửi tín hiệu về ESP32 để dừng động cơ, tránh va đập cơ khí và bảo vệ các thiết bị khỏi hư hỏng.

Bên cạnh chức năng điều hướng, hệ thống còn sử dụng mạch đo công suất INA219 để giám sát các thông số điện của tấm pin năng lượng mặt trời. INA219 thực hiện đo điện áp, dòng điện và công suất đầu ra của tấm pin, sau đó truyền dữ liệu về ESP32 thông qua giao tiếp I^2C. Các thông số này được sử dụng để đánh giá hiệu suất hoạt động của hệ thống cũng như phục vụ việc so sánh giữa mô hình điều hướng và mô hình cố định.

Sau khi thu thập và xử lý toàn bộ dữ liệu, ESP32 sử dụng kết nối Wi-Fi để truyền dữ liệu lên nền tảng IoT. Người dùng có thể truy cập Web Dashboard để theo dõi các thông số như điện áp (V), dòng điện (A), công suất (W) cùng trạng thái hoạt động của hệ thống theo thời

gian thực. Đồng thời, toàn bộ dữ liệu đo được cũng được lưu trữ dưới dạng Data Sheet, phục vụ cho việc thống kê, phân tích, đánh giá hiệu suất và theo dõi lịch sử hoạt động của hệ thống.

3.1.2 Lưu đồ giải thuật



Hình 3.2 Lưu đồ giải thuật của hệ thống

Giải thích sơ đồ

Lưu đồ giải thuật mô tả toàn bộ quá trình hoạt động của hệ thống điều hướng pin năng lượng mặt trời hai trục sử dụng vi điều khiển ESP32. Thuật toán được thực hiện theo trình tự từ khởi tạo hệ thống, thu thập dữ liệu cảm biến, xử lý thông tin, điều khiển động cơ đến giám sát và truyền dữ liệu lên nền tảng IoT.

Đầu tiên, khi hệ thống được cấp nguồn, chương trình bắt đầu thực hiện quá trình khởi tạo hệ thống. Tại bước này, ESP32 tiến hành cấu hình các chân vào/ra, khởi tạo giao tiếp với các thiết bị ngoại vi như cảm biến ánh sáng TEMT6000, mạch đo công suất INA219, encoder, công tắc hành trình và kết nối mạng Wi-Fi để phục vụ việc truyền dữ liệu lên giao diện giám sát.

Sau khi quá trình khởi tạo hoàn tất, ESP32 liên tục đọc giá trị từ các cảm biến ánh sáng. Các cảm biến được bố trí tại bốn vị trí trên tấm pin nhằm xác định cường độ ánh sáng ở các hướng khác nhau. Đồng thời, ESP32 cũng thu thập các thông số điện của tấm pin như điện áp, dòng điện và công suất thông qua mạch INA219.

Tiếp theo, hệ thống tiến hành xử lý dữ liệu. ESP32 so sánh giá trị cường độ ánh sáng giữa các cảm biến để xác định hướng lệch của tấm pin so với hướng chiếu của Mặt Trời. Đồng thời, các giá trị điện áp, dòng điện và công suất cũng được xử lý để phục vụ việc giám sát và đánh giá hiệu suất hoạt động của hệ thống.

Sau khi xử lý dữ liệu, ESP32 thực hiện gửi các thông số hoạt động lên Web Dashboard và Google Sheet thông qua kết nối Wi-Fi. Web Dashboard cho phép người dùng theo dõi các thông số của hệ thống theo thời gian thực, trong khi Google Sheet được sử dụng để lưu trữ dữ liệu phục vụ quá trình thống kê, phân tích và đánh giá sau này.

Tiếp theo là quá trình điều khiển trục X. Dựa trên kết quả xử lý từ các cảm biến ánh sáng, ESP32 xác định chiều quay của động cơ. Nếu cường độ ánh sáng ở phía bên trái lớn hơn, động cơ sẽ quay sang trái; ngược lại, nếu cường độ ánh sáng phía bên phải lớn hơn, động

cơ sẽ quay sang phải. Động cơ bước NEMA57 sẽ điều khiển cơ cấu quay quanh trục X cho đến khi đạt vị trí tối ưu.

Tương tự, hệ thống tiếp tục điều khiển trục Y. ESP32 so sánh giá trị ánh sáng theo phương thẳng đứng để quyết định quay lên hoặc quay xuống, giúp tấm pin luôn hướng vuông góc với tia sáng Mặt Trời theo cả hai phương.

Trong quá trình hai động cơ hoạt động, thuật toán PID được sử dụng để giảm sai số giữa vị trí hiện tại và vị trí mong muốn. Dữ liệu phản hồi từ encoder được ESP32 sử dụng để tính toán sai số và điều chỉnh tốc độ hoặc số bước quay của động cơ, giúp hệ thống đạt vị trí cân bằng nhanh, giảm dao động và nâng cao độ chính xác khi điều hướng.

Bên cạnh đó, công tắc hành trình luôn được giám sát để đảm bảo cơ cấu quay không vượt quá giới hạn cơ khí đã thiết kế. Khi công tắc hành trình được kích hoạt, ESP32 sẽ dừng động cơ ngay lập tức nhằm bảo vệ hệ thống khỏi va chạm hoặc hư hỏng.

Sau mỗi lần điều chỉnh, hệ thống kiểm tra lại giá trị ánh sáng. Nếu giữa các cảm biến vẫn còn xuất hiện sai lệch lớn hơn ngưỡng cho phép, động cơ sẽ tiếp tục xoay để hiệu chỉnh vị trí. Ngược lại, nếu cường độ ánh sáng giữa các cảm biến đã cân bằng và sai số nằm trong giới hạn cho phép, động cơ sẽ dừng quay, giữ nguyên vị trí của tấm pin.

Cuối cùng, chương trình lặp lại toàn bộ quá trình từ bước đọc cảm biến, xử lý dữ liệu, điều khiển động cơ và cập nhật dữ liệu lên nền tảng IoT. Chu trình này được thực hiện liên tục trong suốt thời gian hệ thống hoạt động, giúp tấm pin luôn bám theo hướng chiếu của Mặt Trời và duy trì hiệu suất thu nhận năng lượng ở mức cao nhất. Sau mỗi chu kỳ thực hiện đầy đủ các bước trên, hệ thống sẽ chờ 2 giây rồi tiếp tục đo lặp lại, tạo nên một quy trình giám sát và điều khiển liên tục, tự động, chính xác và ổn định.

3.1.3 Thuật toán điều khiển PID

Giới thiệu:

PID (Proportional – Integral – Derivative) là một thuật toán điều khiển phản hồi được sử dụng rộng rãi trong các hệ thống điều khiển tự động. Đây là một trong những phương pháp

điều khiển vòng kín phổ biến nhất nhờ khả năng nâng cao độ chính xác, cải thiện tính ổn định và đáp ứng nhanh trước các biến đổi của hệ thống. PID được ứng dụng trong nhiều lĩnh vực như tự động hóa công nghiệp, điện – điện tử, robot, điều khiển động cơ và các hệ thống cơ điện tử.

Nguyên lý hoạt động của bộ điều khiển PID dựa trên việc liên tục so sánh giữa giá trị đặt (Setpoint) và giá trị thực tế (Feedback) của hệ thống để xác định sai số. Từ sai số này, bộ điều khiển sẽ tính toán tín hiệu điều khiển thích hợp nhằm đưa đầu ra của hệ thống tiến dần đến giá trị mong muốn và duy trì trạng thái ổn định.

Bộ điều khiển PID bao gồm ba thành phần cơ bản. Thành phần tỷ lệ (Proportional – P) tạo ra tín hiệu điều khiển có giá trị tỷ lệ thuận với sai số tức thời, giúp hệ thống phản ứng nhanh khi xuất hiện sai lệch. Thành phần tích phân (Integral – I) thực hiện việc tích lũy sai số theo thời gian để loại bỏ sai số xác lập và nâng cao độ chính xác của hệ thống. Thành phần vi phân (Derivative – D) dự đoán xu hướng thay đổi của sai số thông qua tốc độ biến thiên của nó, từ đó giảm hiện tượng dao động, hạn chế vượt quá giá trị đặt và cải thiện độ ổn định trong quá trình điều khiển.

Với cấu trúc đơn giản, khả năng điều chỉnh linh hoạt và hiệu quả điều khiển cao, PID đã trở thành một trong những thuật toán điều khiển được ứng dụng rộng rãi nhất hiện nay. Nhờ khả năng đáp ứng nhanh, giảm sai số và duy trì sự ổn định của hệ thống, bộ điều khiển PID được sử dụng trong nhiều ứng dụng thực tế, đặc biệt là các hệ thống yêu cầu điều khiển vị trí, tốc độ và quá trình một cách chính xác

Ứng dụng trong đề tài điều hướng pin năng lượng mặt trời

bộ điều khiển PID được ứng dụng để điều khiển chính xác vị trí quay của hai động cơ bước NEMA57 theo hai trục Azimuth (trục ngang) và Elevation (trục đứng). Mục tiêu của bộ điều khiển là đưa tấm pin năng lượng mặt trời luôn hướng vuông góc với tia bức xạ Mặt Trời nhằm tối đa hóa lượng năng lượng thu được.

Hệ thống sử dụng bốn cảm biến ánh sáng TEMT6000 để đo cường độ ánh sáng tại bốn vị trí khác nhau trên bề mặt tấm pin. ESP32 liên tục thu thập và so sánh các giá trị đo được

để xác định hướng lệch của tấm pin so với hướng có cường độ ánh sáng lớn nhất. Đồng thời, Rotary Encoder HN3806-AB-600G được lắp trên mỗi trục để phản hồi góc quay thực tế của động cơ về ESP32. Từ các tín hiệu phản hồi này, ESP32 xác định sai số giữa vị trí hiện tại và vị trí mong muốn, sau đó áp dụng thuật toán PID để tính toán tín hiệu điều khiển phù hợp gửi đến driver TB6600.

Thành phần P (Proportional) giúp động cơ phản ứng nhanh với sai số vị trí, đưa tấm pin quay về đúng hướng trong thời gian ngắn. Thành phần I (Integral) có nhiệm vụ loại bỏ sai số tích lũy, đảm bảo tấm pin đạt đúng vị trí yêu cầu sau khi quá trình điều chỉnh kết thúc. Thành phần D (Derivative) dự đoán xu hướng thay đổi của sai số, giúp giảm hiện tượng dao động và hạn chế việc động cơ quay vượt quá vị trí cần thiết. Nhờ sự kết hợp của ba thành phần này, hệ thống đạt được khả năng điều khiển ổn định, chính xác và có thời gian đáp ứng nhanh.

Bên cạnh đó, dữ liệu phản hồi từ encoder kết hợp với tín hiệu từ công tắc hành trình giúp ESP32 kiểm soát giới hạn chuyển động của hai trục, đảm bảo cơ cấu quay hoạt động an toàn và tránh va chạm cơ khí. Khi tấm pin đã đạt đến vị trí tối ưu hoặc chạm giới hạn hành trình, bộ điều khiển sẽ dừng cấp xung điều khiển đến động cơ, góp phần giảm tiêu thụ năng lượng và tăng tuổi thọ của hệ thống.

Việc ứng dụng bộ điều khiển PID trong đề tài giúp nâng cao độ chính xác của cơ cấu điều hướng, giảm sai số vị trí, hạn chế rung lắc và tăng độ ổn định khi vận hành. Nhờ đó, tấm pin luôn duy trì hướng thu nhận bức xạ mặt trời tối ưu trong suốt quá trình hoạt động, góp phần cải thiện hiệu suất phát điện và nâng cao hiệu quả khai thác năng lượng mặt trời của mô hình.

Công thức điều khiển PID được sử dụng như sau:

$$Output(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt}$$

Trong đó:

- $e(t)$là sai số tại thời điểm t, được xác định bằng hiệu giữa giá trị đặt và giá trị phản hồi.
- $Output(t)$ là tín hiệu tần số xung điều khiển được gửi đến cơ cấu chấp hành. Quyết định tốc độ phát xung STEP đến driver TB6600
- Thành phần P (Proportional) là thành phần tỷ lệ, quyết định mức độ phản ứng của hệ thống đối với sai số hiện tại.

$$P = K_p * e(t)$$

- Thành phần I (Integral) là thành phần tích phân, có nhiệm vụ cộng dồn sai số theo thời gian nhằm loại bỏ sai số xác lập.

$$I = K_i \int e(t)dt$$

- Thành phần D (Derivative) là thành phần vi phân, giúp dự đoán xu hướng biến thiên của sai số để giảm dao động và tăng độ ổn định.

$$D = K_d \frac{de(t)}{dt}$$

3.1.4 PID vòng ngoài (PID quang học/vị trí)

Hệ thống liên tục đọc dữ liệu từ bốn cảm biến ánh sáng TEMT6000 để xác định mức độ chênh lệch cường độ ánh sáng giữa các hướng, từ đó tính toán góc lệch của tấm pin so với vị trí có bức xạ Mặt Trời lớn nhất. Sai số này được đưa vào bộ điều khiển PID, trong đó tín hiệu điều khiển được xác định theo công thức:

$$Output(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt}$$

Trong đó:

- $e_x = L - R$. Biểu thị độ lệch theo phương ngang
- $e_y = top - bottom$. Biểu thị độ lệch theo phương dọc

Thành phần K_p=0,15 tạo ra tín hiệu điều khiển tỉ lệ với sai số hiện tại, giúp hệ thống phản ứng nhanh khi tấm pin lệch khỏi hướng tối ưu. Do tín hiệu đầu vào của cảm biến có dải giá

trị ADC lớn (0–4095), việc lựa chọn hệ số K_p nhỏ giúp động cơ vận hành êm hơn, hạn chế hiện tượng tăng tốc đột ngột hoặc quay quá nhanh.

Thành phần K_i=0,02 có nhiệm vụ tích lũy sai số theo thời gian, từ đó loại bỏ sai số còn tồn tại khi tấm pin đã gần đạt vị trí mong muốn. Nhờ vậy, hệ thống có thể điều chỉnh chính xác hơn và đưa tấm pin về vị trí gần vuông góc với tia sáng Mặt Trời.

Thành phần K_d=0,02 phản ánh tốc độ thay đổi của sai số, có tác dụng giảm dao động và hãm chuyển động của động cơ khi tấm pin tiến gần đến vị trí cân bằng. Đồng thời, thành phần này cũng giúp hạn chế ảnh hưởng của những biến đổi ánh sáng đột ngột do mây che hoặc điều kiện thời tiết thay đổi.

Giá trị Output(t) được ESP32 sử dụng để điều khiển chiều quay và tốc độ của động cơ, đưa tấm pin về vị trí thu nhận bức xạ Mặt Trời tối ưu.

3.1.5 PID vòng trong (PID tốc độ cơ khí của Encoder)

Vòng điều khiển này sử dụng tín hiệu phản hồi từ Encoder HN3806 để giám sát tốc độ quay thực tế của động cơ bước và so sánh với tốc độ yêu cầu từ vòng điều khiển ngoài. Sai số tốc độ được đưa vào bộ điều khiển PID theo công thức:

$$Output(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt}$$

Trong đó:

e(t) là sai số giữa tốc độ đặt và tốc độ thực tế của động cơ. Nếu động cơ quay chậm hơn yêu cầu do tải lớn, ma sát hoặc tác động từ môi trường, sai số sẽ tăng và bộ điều khiển sẽ tự động điều chỉnh để đưa tốc độ về giá trị mong muốn.

- K_p là thành phần tỷ lệ, tạo tín hiệu điều khiển tỉ lệ với sai số hiện tại, giúp động cơ phản ứng nhanh khi tốc độ thay đổi và nhanh chóng bù lại lượng tốc độ bị thiếu.
- $K_i = 5$ là thành phần tích phân, có nhiệm vụ cộng dồn sai số theo thời gian để loại bỏ sai số kéo dài do ma sát cơ khí, tải trọng hoặc sự thay đổi mômen cản khi tấm pin thay đổi góc nghiêng.

- $K_d = 0$ nên thành phần vi phân không được sử dụng. Việc bỏ khâu D giúp tránh khuếch đại nhiễu từ tín hiệu encoder và làm cho hệ thống hoạt động ổn định hơn.

Giá trị Output(t) sau khi tính toán được ESP32 sử dụng để điều chỉnh độ trễ giữa các xung STEP gửi đến driver TB6600

Hệ thống Solar Tracker hai trục được xây dựng dựa trên nguyên lý điều khiển vòng kín kép (Cascade PID), kết hợp giữa phản hồi từ bốn cảm biến ánh sáng TEMT6000 và hai encoder quay tương đối HN3806-AB-600G, dưới sự điều khiển của vi điều khiển ESP32. Hệ thống gồm hai vòng điều khiển hoạt động liên kết nhằm đảm bảo khả năng bám theo Mặt Trời chính xác và ổn định.

Ở vòng điều khiển ngoài (Outer Loop), ESP32 liên tục thu thập giá trị ADC từ bốn cảm biến ánh sáng. Dữ liệu sau đó được xử lý qua bộ lọc thông thấp (Low-Pass Filter) để giảm nhiễu trước khi tính toán độ chênh lệch cường độ ánh sáng theo hai trục ngang và dọc (rawError). Sai số này được đưa vào bộ điều khiển PID thứ nhất với các hệ số $K_p = 0{,}15$, $K_i = 0{,}02$, $K_d = 0{,}02$, đồng thời tích hợp các cơ chế Anti-Windup và Bumpless Transfer nhằm hạn chế hiện tượng bão hòa tích phân và đảm bảo quá trình chuyển đổi chế độ điều khiển diễn ra mượt mà. Kết quả đầu ra của PID được sử dụng để xác định hướng quay, tốc độ mong muốn của động cơ (targetEncoderSpeed) và thời gian trễ phát xung ban đầu (baseStepDelayUs).

Tiếp theo, vòng điều khiển trong (Inner Loop) có nhiệm vụ kiểm soát tốc độ quay thực tế của động cơ. ESP32 nhận tín hiệu phản hồi từ encoder HN3806 sau mỗi 100 ms, tính toán tốc độ thực tế và so sánh với tốc độ đặt từ vòng điều khiển ngoài. Sai số tốc độ được xử lý bởi bộ điều khiển PID thứ hai với các hệ số $K_p = 20{,}0$và $K_i = 5{,}0$để tạo ra giá trị hiệu chỉnh (correction). Giá trị này được dùng để điều chỉnh thời gian trễ giữa các xung STEP (stepDelayUs) gửi đến driver TB6600, giúp động cơ bước luôn duy trì đúng tốc độ yêu cầu, kể cả khi chịu tác động của tải, ma sát hoặc gió.

Để đảm bảo chuyển động ổn định, tốc độ điều khiển tiếp tục được xử lý qua hàm smoothStepDelay, giới hạn mức thay đổi thời gian trễ không vượt quá 120 µs trong mỗi

chu kỳ. Nhờ đó, động cơ tăng tốc và giảm tốc một cách êm ái, hạn chế rung lắc và giảm nguy cơ mất bước khi tấm pin tiến đến vị trí tối ưu.

Ngoài ra, hệ thống còn tích hợp vùng chết có trễ (Hysteresis Deadband) với hai ngưỡng LIGHT_DEADBAND_IN = 80 và LIGHT_DEADBAND_OUT = 160, giúp động cơ dừng ổn định khi tấm pin đã hướng chính xác về phía Mặt Trời, tránh hiện tượng dao động liên tục quanh điểm cân bằng. Bên cạnh đó, ngưỡng LIGHT_MIN = 300 được sử dụng để nhận biết điều kiện ánh sáng yếu vào ban đêm, từ đó tự động dừng quá trình dò tìm và ngắt điều khiển động cơ nhằm tiết kiệm năng lượng.

Các thông số của hai bộ điều khiển PID được xác định thông qua nhiều lần thử nghiệm và hiệu chỉnh thực tế. Sau quá trình tinh chỉnh, bộ thông số K_p, K_i, K_dđược lựa chọn đã đáp ứng tốt yêu cầu của hệ thống, đảm bảo động cơ phản ứng nhanh, vận hành ổn định, giảm dao động và duy trì sai số điều hướng ở mức thấp, góp phần nâng cao hiệu quả bám theo Mặt Trời của mô hình.

Ưu điểm của bộ điều khiển PID trong hệ thống

Việc ứng dụng bộ điều khiển PID trong hệ thống điều hướng pin năng lượng mặt trời mang lại nhiều ưu điểm nổi bật, góp phần nâng cao hiệu quả hoạt động và độ ổn định của toàn bộ mô hình.

Trước hết, bộ điều khiển PID giúp nâng cao độ chính xác trong quá trình điều khiển góc quay của tấm pin. Thông qua việc liên tục tính toán sai số giữa vị trí hiện tại và vị trí mong muốn, PID điều chỉnh tín hiệu điều khiển phù hợp để đưa tấm pin đến đúng vị trí cần thiết. Nhờ đó, hiện tượng quay quá vị trí đặt (overshoot) hoặc dao động liên tục quanh vị trí cân bằng được hạn chế đáng kể.

Bên cạnh đó, PID còn giúp động cơ hoạt động êm và ổn định hơn. Quá trình tăng tốc, giảm tốc và dừng động cơ được thực hiện một cách mượt mà, giảm rung lắc và hạn chế các tác động cơ học lên hệ thống truyền động. Điều này không chỉ kéo dài tuổi thọ của động cơ và các chi tiết cơ khí mà còn góp phần giảm mức tiêu thụ năng lượng trong quá trình vận hành.

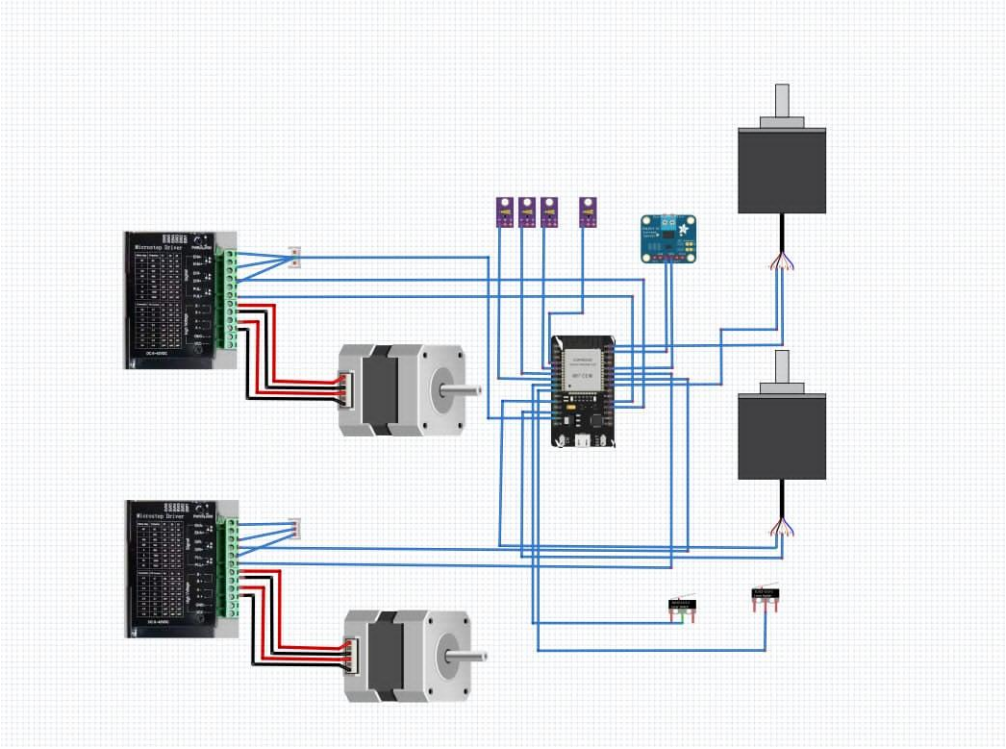
Ngoài ra, việc duy trì tấm pin luôn hướng về phía có cường độ bức xạ mặt trời lớn nhất giúp tối ưu khả năng thu nhận năng lượng. Hệ thống có thể nhanh chóng hiệu chỉnh vị trí của tấm pin khi hướng chiếu của Mặt Trời thay đổi, từ đó nâng cao sản lượng điện tạo ra so với phương pháp lắp đặt cố định.

Qua quá trình thử nghiệm thực tế, hệ thống cho thấy thời gian điều chỉnh tấm pin về vị trí tối ưu chỉ khoảng 10 đến 15 giây sau khi phát hiện sự thay đổi về hướng ánh sáng. Đồng thời, sai số định hướng luôn được duy trì dưới 3°, đáp ứng yêu cầu về độ chính xác đối với mô hình điều hướng hai trục.

Nhờ những ưu điểm trên, bộ điều khiển PID giúp hệ thống đạt được khả năng đáp ứng nhanh, độ ổn định cao và hiệu quả điều hướng tốt, góp phần nâng cao hiệu suất khai thác năng lượng mặt trời cũng như tăng độ tin cậy của mô hình trong quá trình vận hành.

3.1.6 Thiết kế mạch điện

Sơ đồ mạch điện



Hình 3.3 Sơ đồ mạch điện

Giải thích:

Sơ đồ nguyên lý mạch điện của mô hình điều hướng pin năng lượng mặt trời được thiết kế theo cấu trúc điều khiển tập trung, trong đó vi điều khiển ESP32 đóng vai trò là bộ điều khiển trung tâm, thực hiện chức năng thu thập dữ liệu từ các cảm biến, xử lý tín hiệu và điều khiển các cơ cấu chấp hành nhằm đảm bảo tấm pin năng lượng mặt trời luôn hướng về phía có cường độ bức xạ lớn nhất. Hệ thống bao gồm các thành phần chính như vi điều khiển ESP32, hai driver điều khiển động cơ bước TB6600, hai động cơ bước 57 (NEMA 23), bốn cảm biến ánh sáng TEMT6000, mạch đo công suất INA219 và hai công tắc hành trình. Các khối chức năng được kết nối với nhau tạo thành một hệ thống điều khiển khép kín, vừa thực hiện chức năng điều hướng tự động, vừa giám sát hiệu suất làm việc của tấm pin năng lượng mặt trời.

ESP32 là trung tâm điều khiển của toàn bộ hệ thống, có nhiệm vụ thu nhận dữ liệu từ các cảm biến ánh sáng TEMT6000 thông qua các kênh chuyển đổi tương tự sang số (ADC) và giao tiếp với mạch đo công suất INA219 thông qua chuẩn truyền thông I^2C để thu thập các thông số điện áp, dòng điện và công suất của tấm pin năng lượng mặt trời. Sau khi xử lý các tín hiệu đầu vào theo thuật toán điều hướng đã được lập trình, ESP32 phát các tín hiệu điều khiển STEP và DIR đến hai driver TB6600 để điều khiển hai động cơ bước thực hiện chuyển động theo yêu cầu. Nhờ bộ vi xử lý có tốc độ xử lý cao, tích hợp nhiều chuẩn giao tiếp và khả năng hoạt động ổn định, ESP32 đáp ứng tốt yêu cầu điều khiển đồng thời nhiều thiết bị trong mô hình.

Để xác định hướng của nguồn sáng, hệ thống sử dụng bốn cảm biến ánh sáng TEMT6000 được bố trí tại bốn góc xung quanh tâm của tấm pin năng lượng mặt trời. Các cảm biến này hoạt động dựa trên nguyên lý quang điện, trong đó cường độ ánh sáng chiếu vào sẽ làm thay đổi điện áp đầu ra của cảm biến. Các tín hiệu điện áp này được ESP32 đọc thông qua các chân ADC và tiến hành so sánh theo từng cặp cảm biến ở hai phương ngang và phương dọc. Khi xuất hiện sự chênh lệch cường độ ánh sáng giữa các cảm biến vượt quá giá trị ngưỡng cài đặt, ESP32 sẽ xác định hướng cần điều chỉnh và phát tín hiệu điều khiển để

động cơ bước quay về phía có cường độ ánh sáng lớn hơn. Quá trình điều chỉnh được thực hiện liên tục cho đến khi giá trị thu được từ bốn cảm biến gần như cân bằng, khi đó bề mặt tấm pin được xem là đã hướng gần vuông góc với tia bức xạ mặt trời, giúp tối ưu hóa khả năng hấp thụ năng lượng.

Để điều khiển chuyển động của hệ thống, mô hình sử dụng hai động cơ bước 57 (NEMA 23) kết hợp với hai driver TB6600. Động cơ bước 57 là loại động cơ có kích thước mặt bích 57 × 57 mm, được sử dụng phổ biến trong các hệ thống tự động hóa nhờ khả năng điều khiển chính xác vị trí, mô-men xoắn lớn và hoạt động ổn định. Trong mô hình, một động cơ được lắp trên trục phương vị (Azimuth) để điều khiển chuyển động quay của tấm pin theo hướng Đông – Tây, trong khi động cơ còn lại được lắp trên trục cao độ (Elevation) để điều chỉnh góc nghiêng của tấm pin theo sự thay đổi độ cao của mặt trời. Hai driver TB6600 đóng vai trò là mạch công suất, nhận tín hiệu STEP và DIR từ ESP32 để điều khiển chiều quay và số bước của động cơ, đồng thời cung cấp dòng điện phù hợp giúp động cơ hoạt động ổn định, hạn chế hiện tượng mất bước và đảm bảo mô-men xoắn cần thiết khi mang tải là tấm pin cùng cơ cấu truyền động.

Nhằm đánh giá hiệu quả hoạt động của hệ thống, mô hình được tích hợp mạch đo công suất INA219. Đây là mạch đo sử dụng giao tiếp I²C, có khả năng đo điện áp trên tải, dòng điện và tính toán công suất tiêu thụ hoặc công suất phát ra của tấm pin năng lượng mặt trời với độ chính xác cao. INA219 được kết nối giữa tấm pin và tải để giám sát liên tục các thông số điện trong quá trình vận hành. Dữ liệu đo được truyền về ESP32 để hiển thị, lưu trữ hoặc sử dụng trong việc đánh giá hiệu quả của hệ thống điều hướng. Thông qua các giá trị điện áp, dòng điện và công suất thu được, có thể so sánh hiệu suất giữa chế độ tấm pin cố định và chế độ điều hướng tự động, từ đó đánh giá mức độ cải thiện hiệu quả thu nhận năng lượng của mô hình.

Để đảm bảo an toàn và độ tin cậy trong quá trình hoạt động, hệ thống sử dụng hai công tắc hành trình được lắp đặt tại các vị trí giới hạn của hai trục chuyển động. Các công tắc này có nhiệm vụ giới hạn góc quay của cơ cấu, ngăn ngừa động cơ quay vượt quá giới hạn cơ

khí gây hư hỏng cho hệ thống. Đồng thời, khi khởi động, ESP32 sẽ điều khiển các động cơ di chuyển đến vị trí công tắc hành trình được kích hoạt để xác định vị trí gốc (Home Position). Sau khi hoàn tất quá trình này, hệ thống bắt đầu thực hiện thuật toán điều hướng dựa trên tín hiệu từ các cảm biến ánh sáng.

Trong sơ đồ nguyên lý, các đường dây màu đỏ biểu diễn đường cấp nguồn cho các driver và động cơ, các đường dây màu đen biểu diễn đường nối đất (GND) dùng chung cho toàn bộ hệ thống nhằm đảm bảo ổn định điện áp tham chiếu, còn các đường dây màu xanh biểu diễn các đường truyền tín hiệu điều khiển và tín hiệu đo lường giữa ESP32 với các thiết bị ngoại vi. Trong quá trình vận hành, ESP32 liên tục thu thập dữ liệu từ bốn cảm biến TEMT6000 để xác định hướng có cường độ ánh sáng lớn nhất, đồng thời đọc các thông số điện áp, dòng điện và công suất từ mạch INA219 để theo dõi hiệu suất làm việc của tấm pin. Dựa trên kết quả xử lý, vi điều khiển phát tín hiệu điều khiển đến hai driver TB6600, từ đó điều khiển hai động cơ bước 57 quay theo hai trục cho đến khi cường độ ánh sáng trên các cảm biến đạt trạng thái cân bằng. Nhờ sự phối hợp giữa khối cảm biến ánh sáng, khối điều khiển, khối truyền động và khối đo lường công suất, mô hình có khả năng tự động điều hướng tấm pin năng lượng mặt trời theo sự thay đổi vị trí của mặt trời, đồng thời giám sát các thông số điện theo thời gian thực, góp phần nâng cao hiệu suất thu nhận năng lượng, tăng độ ổn định và tạo cơ sở thực nghiệm để đánh giá hiệu quả của hệ thống.

3.2 Tính toán cơ cấu

1. **Diện tích của tấm pin được xác định theo công thức:**

$$A = L \times W$$

Trong đó:

- $L = 0{,}6\ m$
- $W = 0{,}4\ m$

Thay số:

$$A = 0{,}6 \times 0{,}4$$

$$A = 0{,}24m^2$$

Vậy diện tích bề mặt của tấm pin là:

$$A = 0.24\ m^2$$

2. Cường độ dòng điện định mức

Theo công thức:

$$P = UI$$

Suy ra:

$$I = \frac{P}{U}$$

$$I = \frac{18}{10}$$

$$I = 1{,}8A$$

3. Mật độ công suất

Ta có công thức:

Thay số:

$$P_d = \frac{P}{A}$$

$$P_d = \frac{18}{0{,}24}$$

$$P_d = 75W/m^2$$

4. Lực gió tác dụng lên tấm pin

Theo tiêu chuẩn khí động học ASCE 7-16, ta có:

$$F = \frac{1}{2} * \rho C_d A V^2$$

Trong đó:

- $\rho = 1{,}225 kg/m^3$
- $C_d = 1{,}28$
- $A = 0{,}24 m^2$

Giả sử gió 10 m/s

$$F = 0.5 * 1.225 * 1.28 * 0.24 * 10^2$$

$$F = 18.8\ N$$

5. Trọng lượng của tấm pin:

Ta có khối lượng của tấm pin: m=2kg

Ta có công thức tính trọng lượng:

$$P = mg$$

$$W = 2 \times 9{,}81$$

$$W = 19{,}62N$$

6. Mômen nâng:

Trọng tâm ở giữa tấm pin:

$$L = \frac{0{,}6}{2} = 0{,}3m$$

Theo công thức:

$$M = W \times L$$
$$M = 19{,}6 \times 0{,}3$$
$$M = 5{,}89Nm$$

7. Công suất động cơ:

Do momen động cơ NEMA57 = 2.3Nm (<11.8 Nm)

Nên ta dùng bộ truyền đai có:

Động cơ: 20 răng

Pulley tải: 60 răng

Ta có tỉ số truyền :

$$i = \frac{60}{20} = 3$$

Momen xoắn:

$$M = 2.3 * 3 = 6.9\ Nm$$

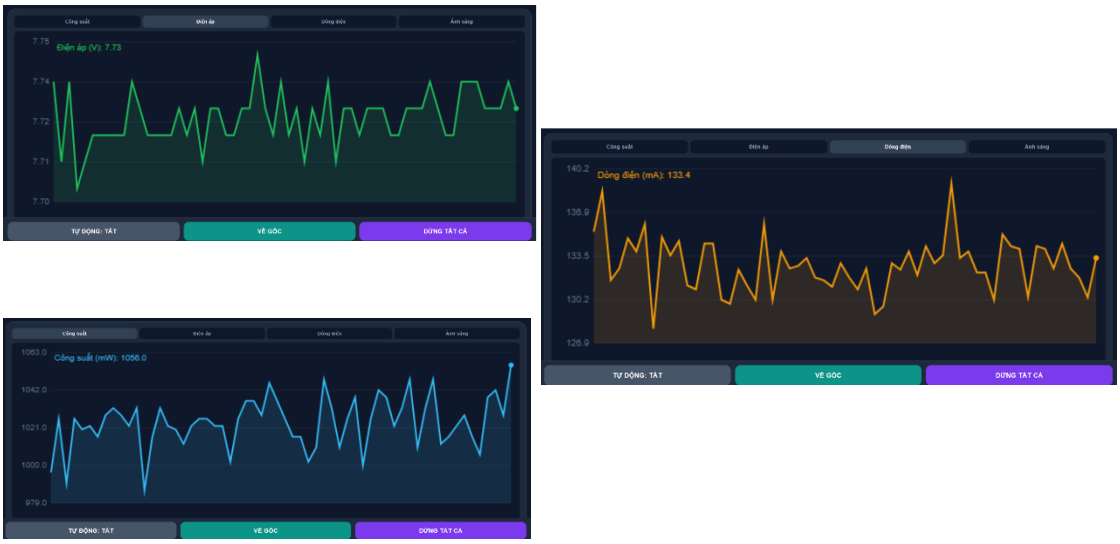
So với yêu cầu: 6.9 > 5.89 (Nm)

⇨ Động cơ đạt yêu cầu

CHƯƠNG 4. THỰC NGHIỆM

4.1 Giao diện từ xa IoT:

- **Nền tảng sử dụng:** DashBoard
- **Kết nối:** ESP32 truyền dữ liệu qua WiFi
- **Chức năng:**
  - Gửi dữ liệu đo được theo chu kỳ của cả 2 tấm pin tracking và static
  - Lưu trữ và hiển thị dữ liệu dạng bảng và biểu đồ (chart) theo thời gian.
  - So sánh dữ liệu giữa hệ thống điều hướng tracking và hệ cố định static.



Hình 4.1 Giao diện biểu đồ đo điện áo, công suất và dòng điện

Phân tích:

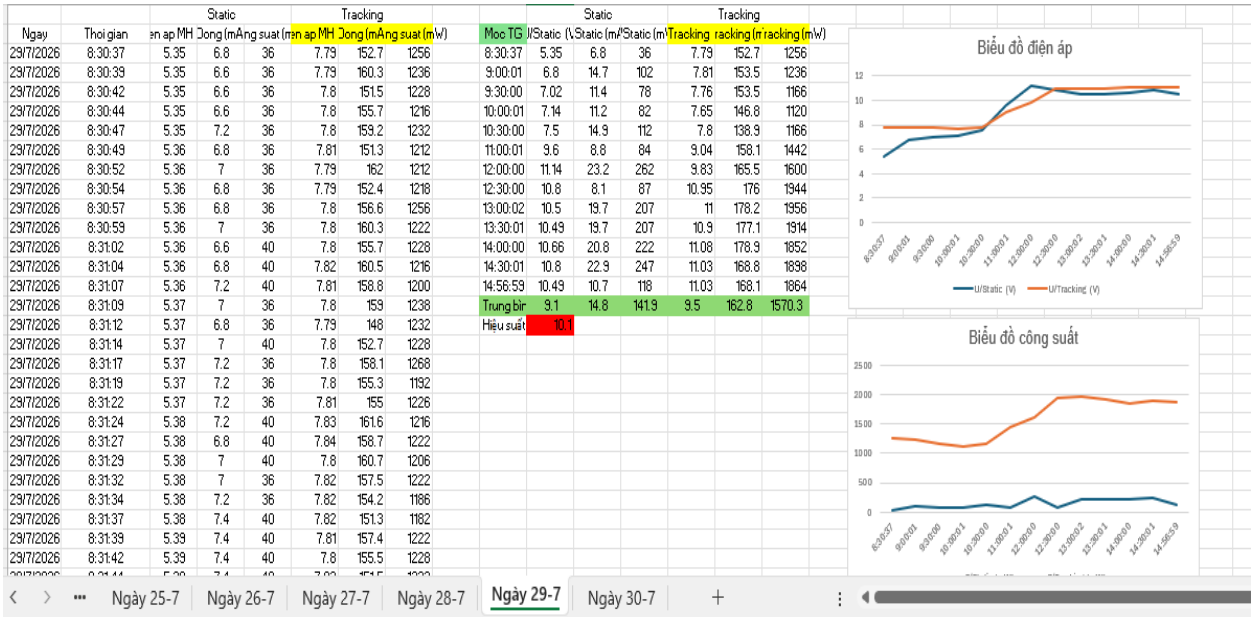
Ba biểu đồ chính, lần lượt thể hiện các thông số điện áp (Voltage), công suất (Power) và dòng điện (Current) của tấm pin năng lượng mặt trời theo thời gian thực. Mỗi biểu đồ được hiển thị bằng đường đồ thị liên tục, phản ánh sự thay đổi của thông số trong quá trình hệ thống vận hành.

Biểu đồ điện áp (Voltage) được hiển thị bằng đường màu xanh lá. Giá trị điện áp thay đổi theo cường độ bức xạ mặt trời và góc chiếu của ánh sáng lên bề mặt tấm pin. Trong quá

trình hoạt động, điện áp có sự dao động nhưng nhìn chung duy trì ở mức tương đối ổn định, cho thấy hệ thống điều hướng luôn điều chỉnh tấm pin về hướng có cường độ ánh sáng phù hợp.

Biểu đồ công suất (Power) được biểu diễn bằng đường màu vàng. Đây là thông số quan trọng nhất để đánh giá hiệu quả của hệ thống. Công suất được tính toán từ giá trị điện áp và dòng điện đo được thông qua mạch đo công suất INA219. Đồ thị cho thấy công suất thay đổi theo điều kiện chiếu sáng và hướng của tấm pin. Khi tấm pin được điều chỉnh đúng hướng, công suất có xu hướng đạt giá trị cao hơn; ngược lại, khi cường độ ánh sáng giảm hoặc góc chiếu thay đổi, công suất cũng giảm theo.

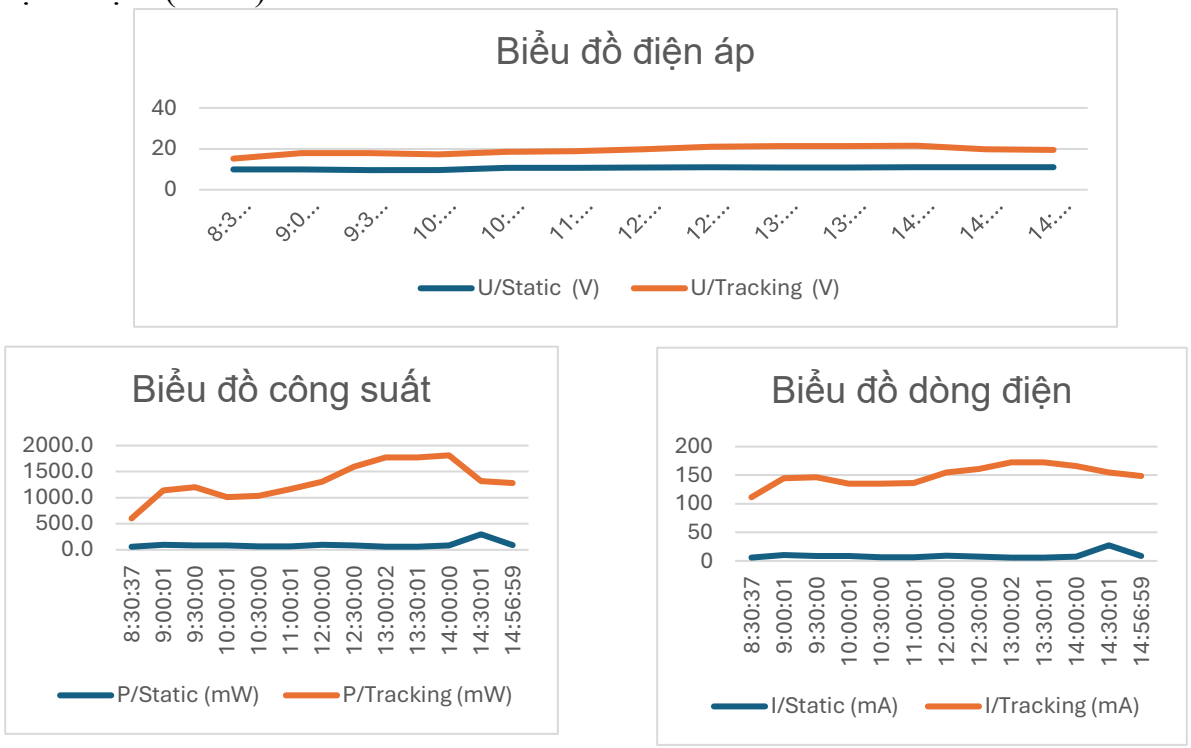
Biểu đồ dòng điện (Current) được hiển thị bằng đường màu xanh dương. Giá trị dòng điện phản ánh lượng điện năng mà tấm pin cung cấp tại từng thời điểm. Dòng điện có xu hướng biến thiên cùng với công suất và chịu ảnh hưởng trực tiếp bởi cường độ ánh sáng mặt trời. Các dao động trên biểu đồ là bình thường do điều kiện ánh sáng ngoài thực tế luôn thay đổi.



Hình 4.2 Dữ liệu trên DataSheet

4.2 Kết quả

Để đánh giá hiệu quả hoạt động của mô hình, hệ thống đã được triển khai vận hành và theo dõi liên tục trong **10 ngày** tại khu vực **Quận 7, Thành phố Hồ Chí Minh**. Quá trình thu thập dữ liệu được thực hiện hằng ngày trong khoảng thời gian từ **08:30 đến 13:00**, tương ứng với khoảng thời gian có cường độ bức xạ mặt trời lớn nhất. Trong suốt quá trình thử nghiệm, hệ thống tiến hành ghi nhận các thông số như **điện áp đầu ra của tấm pin, dòng điện phát ra**. Dựa trên các dữ liệu điện áp và dòng điện thu thập được, hệ thống tính toán công suất và năng lượng điện tích lũy theo từng ngày. Các kết quả này là cơ sở để đánh giá hiệu suất hoạt động của hệ thống điều hướng pin năng lượng mặt trời, đồng thời so sánh khả năng khai thác năng lượng giữa mô hình bám theo Mặt Trời (Tracking) và mô hình lắp đặt cố định (Static).



Hình 4.3 Biểu đồ ngày 1 xuất các giá trị trên DataSheet

a) Biểu đồ điện áp

Quan sát biểu đồ điện áp cho thấy điện áp của cả hai hệ thống đều có xu hướng ổn định trong suốt quá trình đo. Tuy nhiên, điện áp của hệ thống Tracking luôn cao hơn so với hệ

thống Static ở hầu hết các thời điểm. Giá trị điện áp của hệ thống Tracking dao động khoảng 28–30 V, trong khi hệ thống Static chỉ đạt khoảng 24–26 V.

Sự chênh lệch này cho thấy cơ cấu điều hướng hai trục đã giúp tấm pin luôn duy trì góc thu nhận bức xạ mặt trời tối ưu, nhờ đó điện áp đầu ra được cải thiện so với tấm pin lắp đặt cố định. Đến cuối thời gian đo, điện áp của cả hai hệ thống đều giảm nhẹ do cường độ bức xạ mặt trời giảm vào đầu giờ chiều.

b) Biểu đồ công suất

Biểu đồ công suất thể hiện sự khác biệt rõ rệt giữa hai hệ thống. Công suất của hệ thống Tracking tăng dần từ buổi sáng và đạt giá trị cao nhất vào khoảng 13:30–14:00, sau đó giảm nhẹ vào cuối buổi đo. Trong khi đó, công suất của hệ thống Static luôn duy trì ở mức thấp hơn đáng kể và chỉ tăng nhẹ vào một số thời điểm.

Khoảng cách giữa hai đường biểu diễn cho thấy hệ thống điều hướng đã khai thác bức xạ mặt trời hiệu quả hơn. Việc tấm pin liên tục được điều chỉnh hướng theo vị trí Mặt Trời giúp tăng lượng ánh sáng nhận được, từ đó nâng cao công suất phát ra. Đây là minh chứng cho hiệu quả của cơ cấu bám Mặt Trời kết hợp với thuật toán điều khiển PID trong việc tối ưu hóa quá trình thu nhận năng lượng.

c) Biểu đồ dòng điện

Biểu đồ dòng điện cũng cho thấy xu hướng tương tự biểu đồ công suất. Dòng điện của hệ thống Tracking luôn lớn hơn hệ thống Static trong toàn bộ thời gian đo. Giá trị dòng điện của hệ thống Tracking tăng dần từ buổi sáng, đạt giá trị cực đại vào khoảng giữa trưa, sau đó giảm nhẹ khi cường độ bức xạ mặt trời suy giảm.

Đối với hệ thống Static, dòng điện thu được thấp hơn và mức biến thiên không đáng kể. Điều này phản ánh việc tấm pin cố định không thể duy trì góc chiếu tối ưu khi vị trí Mặt Trời thay đổi trong ngày, dẫn đến khả năng hấp thụ bức xạ và sản lượng điện tạo ra thấp hơn.

CHƯƠNG 5. KẾT LUẬN

5.1 Kết luận

Trong quá trình thực hiện, nhóm đã nghiên cứu cơ sở lý thuyết về năng lượng mặt trời, nguyên lý hoạt động của hệ thống điều hướng hai trục, bộ điều khiển PID, các cảm biến đo cường độ ánh sáng TEMT6000, mạch đo công suất INA219, động cơ bước NEMA57, driver TB6600, Rotary Encoder HN3806-AB-600G và vi điều khiển ESP32. Trên cơ sở đó, nhóm đã tiến hành thiết kế mô hình cơ khí bằng phần mềm SolidWorks, xây dựng sơ đồ nguyên lý phần cứng, thiết kế hệ thống điều khiển và lập trình thuật toán cho ESP32 nhằm điều khiển toàn bộ quá trình vận hành của hệ thống.

Mô hình được thiết kế với cơ cấu điều hướng hai trục, cho phép tấm pin thay đổi góc quay theo cả phương ngang (Azimuth) và phương đứng (Elevation), nhờ đó luôn duy trì hướng gần vuông góc với tia bức xạ mặt trời trong suốt quá trình hoạt động. Hệ thống sử dụng bốn cảm biến ánh sáng TEMT6000 để xác định hướng có cường độ ánh sáng lớn nhất, từ đó ESP32 xử lý dữ liệu và áp dụng thuật toán điều khiển PID để điều khiển hai động cơ bước NEMA57 quay đến vị trí tối ưu. Việc sử dụng bộ điều khiển PID giúp giảm sai số vị trí, hạn chế dao động và tăng độ ổn định trong quá trình điều hướng.

Bên cạnh chức năng điều hướng, hệ thống còn tích hợp mạch đo công suất INA219 để thu thập các thông số điện như điện áp, dòng điện và công suất của tấm pin năng lượng mặt trời. Các dữ liệu này được ESP32 xử lý và truyền lên giao diện Web thông qua kết nối Wi-Fi, giúp người dùng có thể giám sát trạng thái hoạt động của hệ thống theo thời gian thực từ bất kỳ đâu có kết nối Internet. Đồng thời, dữ liệu cũng được lưu trữ phục vụ cho việc thống kê, phân tích và đánh giá hiệu quả hoạt động của mô hình.



Hình 5.1 Mô hình sau khi hoàn thiện

5.2 Ưu điểm của hệ thống

Kết quả thử nghiệm cho thấy hệ thống hoạt động ổn định, có khả năng tự động điều chỉnh hướng quay của tấm pin theo sự thay đổi của vị trí Mặt Trời.

Thuật toán PID giúp động cơ vận hành êm, giảm hiện tượng dao động và nhanh chóng đưa tấm pin về vị trí tối ưu với sai số nhỏ.

Hệ thống giám sát hoạt động ổn định, các thông số điện áp, dòng điện và công suất được cập nhật liên tục và hiển thị chính xác trên giao diện Web.

Qua quá trình so sánh giữa mô hình điều hướng hai trục (Tracking) và mô hình cố định (Static), kết quả cho thấy mô hình điều hướng có khả năng thu nhận năng lượng mặt trời tốt hơn, góp phần nâng cao hiệu suất khai thác năng lượng và tận dụng tối đa nguồn bức xạ mặt trời trong suốt thời gian hoạt động.

5.3 Nhược điểm của hệ thống

Bên cạnh những kết quả đạt được, mô hình vẫn còn một số hạn chế.

Mô hình mới được triển khai ở quy mô nghiên cứu và thử nghiệm nên chưa đánh giá đầy đủ trong các điều kiện thời tiết khắc nghiệt như gió lớn, mưa kéo dài hoặc môi trường ngoài trời trong thời gian dài.

Hệ thống điều hướng hiện chủ yếu dựa trên cảm biến ánh sáng nên hiệu quả có thể bị ảnh hưởng khi thời tiết nhiều mây hoặc ánh sáng thay đổi đột ngột.

Ngoài ra, giao diện giám sát mới tập trung vào việc hiển thị các thông số cơ bản và chưa tích hợp nhiều chức năng phân tích dữ liệu hoặc cảnh báo thông minh.

Còn phụ thuộc vào Wifi của ESP32

5.4 Định hướng phát triển trong tương lai

Trong tương lai, hệ thống có thể được tiếp tục nghiên cứu và phát triển theo nhiều hướng nhằm nâng cao hiệu quả và tính ứng dụng.

Có thể tích hợp thêm các thuật toán dự đoán vị trí Mặt Trời dựa trên thời gian thực và tọa độ địa lý để kết hợp với phương pháp điều hướng bằng cảm biến, giúp hệ thống hoạt động chính xác hơn trong điều kiện thời tiết nhiều mây.

Đồng thời, việc ứng dụng các thuật toán điều khiển thông minh như điều khiển mờ (Fuzzy Logic), điều khiển thích nghi hoặc trí tuệ nhân tạo (AI) cũng là hướng nghiên cứu tiềm năng nhằm tối ưu hiệu suất điều hướng.

Bên cạnh đó, hệ thống có thể được mở rộng bằng cách bổ sung chức năng lưu trữ dữ liệu trên nền tảng đám mây, gửi cảnh báo khi phát hiện sự cố, giám sát qua ứng dụng di động và quản lý nhiều cụm pin năng lượng mặt trời cùng lúc.

TÀI LIỆU THAM KHẢO

1. CHUYÊN ĐỀ ARDUINO VÀ TRUYỀN THÔNG

https://kythuat.upt.edu.vn/uploads/news/2021_06/giao-trinh-arduino-1.pdf

2. GIỚI THIỆU VỀ ARDUINO VÀ ỨNG DỤNG ARDUINO

https://www.vietnic.vn/gioi-thieu-ve-arduino-va-ung-dung-arduino

3. Ahmed, M., Khan, A., & Rehman, A. (2021). IoT-based Solar Energy Monitoring System. ResearchGate.

https://www.researchgate.net/publication/350877330

4. Shaikh, F. K., Zeadally, S., & Exposito, E. (2020). Smart Solar Tracking System Using IoT. IEEE Access, 8, 148779–148789.

https://ieeexplore.ieee.org/document/9142383

5. Wang, J., Li, Y., & Zhang, H. (2019). A Cloud-Based Solar Energy Monitoring System Using IoT. Procedia Computer Science, 154, 564–569. Elsevier.

https://doi.org/10.1016/j.procs.2019.06.083

6. Patel, S., Kumar, A., & Pandey, P. (2022). Performance Analysis of Solar PV System using IoT for Remote Areas. Lecture Notes in Electrical Engineering, Springer.

https://doi.org/10.1007/978-981-16-7885-4_31

7. K.J. Åström and T. Hägglund, PID Controllers: Theory, Design, and Tuning, Instrument Society of America, 1995.
8. S. K. Singh, Industrial Instrumentation and Control, Tata McGraw-Hill Education, 2003.
9. Google Developers, Apps Script with Google Sheets – Hướng dẫn tự động hóa lưu dữ liệu IoT vào Google Sheet
10. GIÁO TRÌNH MÔ ĐUN: CƠ BẢN VỀ HTML & CSS

MD-15.-Kien-thuc-co-ban-ve-HTMLCSS_compressed.pdf

PHỤ LỤC A

Code Adruino Mô hình động

```
#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>
#include <Adafruit_INA219.h>

const char* AP_SSID     = "SolarMonitor";
const char* AP_PASSWORD = "12345678";

WebServer server(80);
Adafruit_INA219 ina219;

float busVoltage_V = 0;
float current_mA   = 0;
float power_mW     = 0;
bool  ina219Found  = false;

const char* WEB_PAGE = R"rawliteral(<!DOCTYPE html>
<html lang="vi">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no">
<title>Giám sát năng lượng mặt trời</title>
<style>
```

```
* { margin:0; padding:0; box-sizing:border-box; }
body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
  background: #0f172a;
  color: #e2e8f0;
  min-height: 100vh;
  padding: 12px;
  -webkit-tap-highlight-color: transparent;
}
.header { text-align: center; padding: 8px 0 14px 0; }
.header h1 { font-size: 18px; font-weight: 700; color: #f1f5f9; }
.header p { font-size: 11px; color: #64748b; margin-top: 2px; }

.sensors {
  display: grid; grid-template-columns: repeat(3, 1fr); gap: 8px;
  margin-bottom: 14px;
}
.sensor-card {
  background: #1e293b; border-radius: 14px; padding: 12px 8px; text-align: center;
  border: 1px solid #334155;
}
.sensor-label { font-size: 10px; color: #94a3b8; text-transform: uppercase; letter-spacing:
1px; margin-bottom: 4px; }
.sensor-value { font-size: 22px; font-weight: 700; color: #38bdf8; }
.sensor-unit { font-size: 11px; color: #64748b; }

.chart-section {
```

```
  background: #1e293b; border-radius: 18px; padding: 16px;
  margin-bottom: 12px; border: 1px solid #334155;
}
.chart-tabs { display: flex; gap: 6px; margin-bottom: 10px; }
.chart-tab {
  flex: 1; border: none; border-radius: 9px; padding: 8px 0;
  font-size: 12px; font-weight: 600; color: #94a3b8; cursor: pointer;
  background: #0f172a; letter-spacing: 0.3px;
}
.chart-tab.active { background: #334155; color: #f1f5f9; }
.chart-section canvas { width: 100%; height: auto; display: block; border-radius: 10px; }
.chart-actions { display: flex; align-items: center; gap: 8px; margin-top: 10px; }
.btn-export, .btn-clear {
  border: none; border-radius: 9px; padding: 9px 14px;
  font-size: 12px; font-weight: 600; color: #fff; cursor: pointer;
}
.btn-export { background: #2563eb; }
.btn-clear { background: #475569; }
.log-info { font-size: 11px; color: #64748b; margin-left: auto; }

.footer-bar {
  display: flex; justify-content: space-between; align-items: center;
  padding: 4px;
}
.ip-tag {
  background: #334155; color: #94a3b8; font-size: 10px;
  border-radius: 8px; padding: 4px 10px;
```

```
  }
  .refresh-indicator { font-size: 10px; color: #64748b; }
</style>
</head>
<body>

<div class="header">
  <h1>Giám sát năng lượng mặt trời</h1>
  <p>192.168.4.1 &middot; WiFi: SolarMonitor</p>
</div>

<div class="sensors">
  <div class="sensor-card">
    <div class="sensor-label">Điện áp</div>
    <div class="sensor-value" id="voltage">--</div>
    <div class="sensor-unit">V</div>
  </div>
  <div class="sensor-card">
    <div class="sensor-label">Dòng điện</div>
    <div class="sensor-value" id="current">--</div>
    <div class="sensor-unit">mA</div>
  </div>
  <div class="sensor-card">
    <div class="sensor-label">Công suất</div>
    <div class="sensor-value" id="power">--</div>
    <div class="sensor-unit">mW</div>
  </div>
```

```
</div>

<div class="chart-section">
  <div class="chart-tabs">
                <button class="chart-tab active" data-metric="power" onclick="selectChart(this)">Công suất</button>
     <button class="chart-tab" data-metric="voltage" onclick="selectChart(this)">Điện áp</button>
     <button class="chart-tab" data-metric="current" onclick="selectChart(this)">Dòng điện</button>
  </div>
  <canvas id="chart" width="600" height="220"></canvas>
  <div class="chart-actions">
    <span class="log-info" id="logInfo">0 mẫu</span>
    <button class="btn-export" onclick="exportCSV()">Xuất CSV</button>
    <button class="btn-clear" onclick="clearLog()">Xóa</button>
  </div>
</div>

<div class="footer-bar">
  <span class="ip-tag">INA219</span>
  <span class="refresh-indicator" id="refreshInfo">Cập nhật 1s</span>
</div>

<script>
function fetchSensor() {
  var t0 = Date.now();
```

```
  fetch('/sensor')
    .then(function(r) { return r.json(); })
    .then(function(d) {
      document.getElementById('voltage').textContent = d.v.toFixed(2);
      document.getElementById('current').textContent = d.c.toFixed(1);
      document.getElementById('power').textContent = d.p.toFixed(1);
      pushHistory(d);
      drawChart();
      document.getElementById('refreshInfo').textContent = 'Cập nhật ' + (Date.now() - t0)
+ 'ms';
    });
}

var MAXPTS = 60;
var hist = { power: [], voltage: [], current: [] };
var curMetric = 'power';
var MAXLOG = 3600;
var logData = [];

function pushHistory(d) {
  hist.power.push(d.p);
  hist.voltage.push(d.v);
  hist.current.push(d.c);
  for (var k in hist) { if (hist[k].length > MAXPTS) hist[k].shift(); }

  var now = new Date();
  logData.push({
```

```
    t: now.toLocaleString('vi-VN'),
    v: d.v, c: d.c, p: d.p
  });
  if (logData.length > MAXLOG) logData.shift();
  updateLogInfo();
}

function exportCSV() {
  if (logData.length === 0) { alert('Chưa có dữ liệu để xuất'); return; }
  var lines = ['Thoi gian,Dien ap (V),Dong (mA),Cong suat (mW)'];
  for (var i = 0; i < logData.length; i++) {
    var r = logData[i];
    lines.push('"' + r.t + '",' + r.v + ',' + r.c + ',' + r.p);
  }
  var csv = '\uFEFF' + lines.join('\r\n');
  var blob = new Blob([csv], { type: 'text/csv;charset=utf-8;' });
  var url = URL.createObjectURL(blob);
  var a = document.createElement('a');
  var ts = new Date();
  var name = 'solar_' + ts.getFullYear()
          + ('0' + (ts.getMonth() + 1)).slice(-2)
          + ('0' + ts.getDate()).slice(-2) + '_'
          + ('0' + ts.getHours()).slice(-2)
          + ('0' + ts.getMinutes()).slice(-2)
          + ('0' + ts.getSeconds()).slice(-2) + '.csv';
  a.href = url; a.download = name;
  document.body.appendChild(a); a.click();
```

```
  document.body.removeChild(a);
  URL.revokeObjectURL(url);
}

function clearLog() {
  logData = [];
  for (var k in hist) hist[k] = [];
  updateLogInfo();
  drawChart();
}

function updateLogInfo() {
  document.getElementById('logInfo').textContent = logData.length + ' mẫu';
}

var CHART_CFG = {
  power:   { label: 'Công suất (mW)', color: '#38bdf8', dec: 1 },
  voltage: { label: 'Điện áp (V)',    color: '#22c55e', dec: 2 },
  current: { label: 'Dòng điện (mA)', color: '#f59e0b', dec: 1 }
};

function selectChart(btn) {
  curMetric = btn.getAttribute('data-metric');
  var tabs = document.querySelectorAll('.chart-tab');
  for (var i = 0; i < tabs.length; i++) tabs[i].classList.remove('active');
  btn.classList.add('active');
  drawChart();
```

```
}

function drawChart() {
  var cv = document.getElementById('chart');
  var ctx = cv.getContext('2d');
  var W = cv.width, H = cv.height;
  var padL = 46, padR = 10, padT = 12, padB = 20;
  var cfg = CHART_CFG[curMetric];
  var data = hist[curMetric];

  ctx.clearRect(0, 0, W, H);
  ctx.fillStyle = '#0f172a';
  ctx.fillRect(0, 0, W, H);

  var mn = Infinity, mx = -Infinity;
  for (var i = 0; i < data.length; i++) {
    if (data[i] < mn) mn = data[i];
    if (data[i] > mx) mx = data[i];
  }
  if (!isFinite(mn)) { mn = 0; mx = 1; }
  if (mn === mx) { mn -= 1; mx += 1; }
  var range = mx - mn;
  mn -= range * 0.1; mx += range * 0.1;

  var plotW = W - padL - padR, plotH = H - padT - padB;

  ctx.strokeStyle = '#1e293b';
```

```
ctx.fillStyle = '#64748b';
ctx.font = '10px sans-serif';
ctx.lineWidth = 1;
ctx.textAlign = 'right';
for (var g = 0; g <= 4; g++) {
  var y = padT + plotH * g / 4;
  var val = mx - (mx - mn) * g / 4;
  ctx.beginPath(); ctx.moveTo(padL, y); ctx.lineTo(W - padR, y); ctx.stroke();
  ctx.fillText(val.toFixed(cfg.dec), padL - 5, y + 3);
}

if (data.length < 2) {
  ctx.fillStyle = '#475569'; ctx.textAlign = 'center';
  ctx.fillText('Đang thu thập dữ liệu...', W / 2, H / 2);
  return;
}

function px(i) { return padL + plotW * i / (MAXPTS - 1); }
function py(v) { return padT + plotH * (1 - (v - mn) / (mx - mn)); }

ctx.beginPath();
ctx.moveTo(px(0), py(data[0]));
for (var i = 1; i < data.length; i++) ctx.lineTo(px(i), py(data[i]));
ctx.lineTo(px(data.length - 1), padT + plotH);
ctx.lineTo(px(0), padT + plotH);
ctx.closePath();
ctx.fillStyle = cfg.color + '22';
```

```
  ctx.fill();

  ctx.beginPath();
  ctx.moveTo(px(0), py(data[0]));
  for (var i = 1; i < data.length; i++) ctx.lineTo(px(i), py(data[i]));
  ctx.strokeStyle = cfg.color;
  ctx.lineWidth = 2;
  ctx.stroke();

  var last = data[data.length - 1];
  ctx.fillStyle = cfg.color;
  ctx.beginPath(); ctx.arc(px(data.length - 1), py(last), 3, 0, 6.29); ctx.fill();
  ctx.textAlign = 'left';
  ctx.fillText(cfg.label + ': ' + last.toFixed(cfg.dec), padL + 4, padT + 10);
}

window.addEventListener('load', function() {
  fetchSensor();
  setInterval(fetchSensor, 1000);
});
</script>
</body>
</html>)rawliteral";

void handleRoot() {
  server.send(200, "text/html; charset=utf-8", WEB_PAGE);
}
```

```
void handleSensor() {
  String json = "{\"v\":" + String(busVoltage_V, 2)
              + ",\"c\":" + String(current_mA, 1)
              + ",\"p\":" + String(power_mW, 1) + "}";
  server.send(200, "application/json", json);
}

void readINA219() {
  if (!ina219Found) return;
  busVoltage_V = ina219.getBusVoltage_V();
  current_mA   = ina219.getCurrent_mA();
  power_mW     = ina219.getPower_mW();
}

void setup() {
  Serial.begin(115200);
  while (!Serial) delay(10);

  WiFi.softAP(AP_SSID, AP_PASSWORD);
  Serial.println("\n=== ESP32 + INA219 Monitor ===");
  Serial.print("WiFi AP: "); Serial.println(AP_SSID);
  Serial.print("IP:      "); Serial.println(WiFi.softAPIP());
  Serial.println("Mat khau:  12345678");
  Serial.println("Web:       http://192.168.4.1");

  Wire.begin(21, 22);
```

```
Serial.println("\n--- Quet I2C bus ---");
int foundCount = 0;
for (int addr = 1; addr < 127; addr++) {
  Wire.beginTransmission(addr);
  if (Wire.endTransmission() == 0) {
    Serial.print("  Tim thay: 0x"); Serial.print(addr, HEX);
    if (addr >= 0x40 && addr <= 0x43) {
      ina219Found = true;
      Serial.print(" [INA219]");
    }
    Serial.println();
    foundCount++;
  }
}
if (foundCount == 0) {
  Serial.println("  KHONG co thiet bi nao tren I2C!");
  Serial.println("  Kiem tra: SDA->D21, SCL->D22, VCC->3.3V, GND->GND");
} else {
  Serial.print("  Tong cong: "); Serial.print(foundCount); Serial.println(" thiet bi");
}

if (ina219Found && ina219.begin()) {
  Serial.println("INA219: OK");
  readINA219();
} else {
  Serial.println("INA219: KHONG doc duoc!");
}
```

```
  server.on("/",       handleRoot);
  server.on("/sensor", handleSensor);
  server.begin();
  Serial.println("Web Server da san sang!");
}

void loop() {
  server.handleClient();

  static unsigned long lastSensorRead = 0;
  unsigned long now = millis();

  if (now - lastSensorRead >= 500) {
    lastSensorRead = now;
    readINA219();

    static unsigned long lastSerialPrint = 0;
    if (now - lastSerialPrint >= 1000) {
      lastSerialPrint = now;
      Serial.print("V="); Serial.print(busVoltage_V, 2);
      Serial.print("V  I="); Serial.print(current_mA, 1);
      Serial.print("mA  P="); Serial.print(power_mW, 1);
      Serial.println("mW");
    }
  }
}
```

Code Adruino mô hình tĩnh

```
#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>
#include <Adafruit_INA219.h>

const char* AP_SSID     = "MotorControl";
const char* AP_PASSWORD = "12345678";

WebServer server(80);
Adafruit_INA219 ina219;

// 4 cam bien anh sang TEMT6000 (analog) tren ADC1
const int PIN_LDR1 = 34;   // TL Tren-Trai
const int PIN_LDR2 = 35;   // TR Tren-Phai
const int PIN_LDR3 = 32;   // BL Duoi-Trai
const int PIN_LDR4 = 33;   // BR Duoi-Phai

const int PIN_STEP1 = 2;
const int PIN_DIR1  = 4;
const int PIN_STEP2 = 18;
const int PIN_DIR2  = 19;

// Khai bao kieu truoc cac ham de Arduino tao prototype dung thu tu.
enum MotorState { STOP, FORWARD, BACKWARD };
```

```
// Encoder vi tri truc xoay ben duoi
const int PIN_ENCODER_A = 5;
const int PIN_ENCODER_B = 23;
// Khong dung D12: day la chan strap dien ap flash cua ESP32.
const int PIN_ENCODER2_A = 13;
const int PIN_ENCODER2_B = 14;
const unsigned long ENCODER_DEBOUNCE_US = 20000;

volatile long encoderCount = 0;
volatile unsigned long lastEncoderEdgeUs = 0;
volatile bool encoderCountingEnabled = false;
portMUX_TYPE encoderMux = portMUX_INITIALIZER_UNLOCKED;

volatile long encoder2Count = 0;
volatile unsigned long lastEncoder2EdgeUs = 0;
volatile bool encoder2CountingEnabled = false;
portMUX_TYPE encoder2Mux = portMUX_INITIALIZER_UNLOCKED;

void IRAM_ATTR encoderISR() {
  // Bo qua moi xung encoder khi dong co truc duoi dang dung.
  if (!encoderCountingEnabled) return;

  const unsigned long now = micros();
  if (now - lastEncoderEdgeUs < ENCODER_DEBOUNCE_US) return;

  portENTER_CRITICAL_ISR(&encoderMux);
  if (digitalRead(PIN_ENCODER_B) == HIGH) {
```

```
    encoderCount++;
  } else {
    encoderCount--;
  }
  lastEncoderEdgeUs = now;
  portEXIT_CRITICAL_ISR(&encoderMux);
}

long readEncoderCount() {
  portENTER_CRITICAL(&encoderMux);
  long count = encoderCount;
  portEXIT_CRITICAL(&encoderMux);
  return count;
}

void resetEncoderCount() {
  portENTER_CRITICAL(&encoderMux);
  encoderCount = 0;
  lastEncoderEdgeUs = micros();
  portEXIT_CRITICAL(&encoderMux);
}

void IRAM_ATTR encoder2ISR() {
  if (!encoder2CountingEnabled) return;

  const unsigned long now = micros();
  if (now - lastEncoder2EdgeUs < ENCODER_DEBOUNCE_US) return;
```

```
  portENTER_CRITICAL_ISR(&encoder2Mux);
  if (digitalRead(PIN_ENCODER2_B) == HIGH) {
    encoder2Count++;
  } else {
    encoder2Count--;
  }
  lastEncoder2EdgeUs = now;
  portEXIT_CRITICAL_ISR(&encoder2Mux);
}

long readEncoder2Count() {
  portENTER_CRITICAL(&encoder2Mux);
  long count = encoder2Count;
  portEXIT_CRITICAL(&encoder2Mux);
  return count;
}

void resetEncoder2Count() {
  portENTER_CRITICAL(&encoder2Mux);
  encoder2Count = 0;
  lastEncoder2EdgeUs = micros();
  portEXIT_CRITICAL(&encoder2Mux);
}

// Cong tac hanh trinh HOME (INPUT_PULLUP -> cham = LOW)
const int PIN_HOME1 = 25;
```

```
const int PIN_HOME2 = 26;
const char HOME_DIR1 = 'B';
const char HOME_DIR2 = 'B';

// Toc do step: thu cong / home dung delay co dinh; auto PID dung delay bien thien
const unsigned long STEP_DELAY_MANUAL = 500;   // us - dieu khien tay / home
const unsigned long STEP_DELAY_MIN    = 200;   // us - nhanh nhat (PID output lon)
const unsigned long STEP_DELAY_MAX    = 2500;  // us - cham nhat (PID output nho)

MotorState state1 = STOP, state2 = STOP;
MotorState lastDir1 = STOP, lastDir2 = STOP;

// Timing non-blocking cho tung motor
unsigned long stepDelayUs1 = STEP_DELAY_MANUAL;
unsigned long stepDelayUs2 = STEP_DELAY_MANUAL;
unsigned long lastStepUs1  = 0;
unsigned long lastStepUs2  = 0;

bool homing1 = false, homing2 = false;

float busVoltage_V    = 0;
float shuntVoltage_mV = 0;
float current_mA      = 0;
float power_mW        = 0;
bool  ina219Found     = false;

// Gia tri anh sang ADC 0-4095
```

```
//   light1 (D34) = TL    light2 (D35) = TR
//   light3 (D32) = BL    light4 (D33) = BR
int light1 = 0, light2 = 0, light3 = 0, light4 = 0;


// ------------------------------------------------------------------------
// PID Controller — smooth version (low-pass filter + bumpless transfer)
// error  = chenh lech anh sang (right-left hoac top-bottom), pham vi ~ -8000..8000
// output = -PID_OUT_MAX .. +PID_OUT_MAX  -> map sang huong + toc do step
// ------------------------------------------------------------------------
struct PIDController {
  float Kp, Ki, Kd;
  float integral;
  float lastError;
  float lastOutput;       // bumpless transfer: dua output ve cu gia tri truoc do
  unsigned long lastMs;
  float outMin, outMax;
  bool  firstRun;

  PIDController(float p = 0.15f, float i = 0.02f, float d = 0.02f)
    : Kp(p), Ki(i), Kd(d),
      integral(0), lastError(0), lastOutput(0), lastMs(0),
      outMin(-1000.0f), outMax(1000.0f), firstRun(true) {}

  void reset() {
    integral       = 0;
    lastError      = 0;
    lastOutput     = 0;
```

```
  filteredError_ = 0;
  firstRun       = true;
}

void setGains(float p, float i, float d) {
  Kp = p; Ki = i; Kd = d;
}

// Low-pass filter cho input error de giam nhay voi nhieu cam bien
float filter(float rawError, float alpha = 0.3f) {
  filteredError_ = alpha * rawError + (1.0f - alpha) * filteredError_;
  return filteredError_;
}

float compute(float rawError) {
  unsigned long now = millis();
  float dt = firstRun ? 0.05f : (now - lastMs) / 1000.0f;
  if (dt < 0.001f) dt = 0.001f;
  if (dt > 0.5f)   dt = 0.5f;   // tranh nhay lon khi bi ngat lau
  firstRun = false;
  lastMs   = now;

  // Filter error de D-term khong nhay theo nhieu ADC
  float err = filter(rawError);

  float pTerm = Kp * err;
```

```
  // I + anti-windup: gioi han tich phan theo outMax
  integral += err * dt;
  float iLimit = (Ki > 0.0001f) ? (outMax / Ki) : outMax;
  if (integral >  iLimit) integral =  iLimit;
  if (integral < -iLimit) integral = -iLimit;
  float iTerm = Ki * integral;

  float dTerm = Kd * (err - lastError) / dt;
  lastError = err;

  float output = pTerm + iTerm + dTerm;
  if (output > outMax) output = outMax;
  if (output < outMin) output = outMin;

  // Anti-windup bo sung: neu output bi cat, khong tich luy them I cung chieu
  if ((output >= outMax && err > 0) || (output <= outMin && err < 0)) {
    integral -= err * dt;   // rollback mau vua cong
  }

  // Bumpless transfer: gan output vao lastOutput de lan sau bat dau tu do
  lastOutput = output;

  return output;
 }

private:
 float filteredError_ = 0;   // low-pass filter state
```

```
};

// Auto-tracking dual-axis + PID
//   Motor 1 = truc NGANG (azimuth)   errorX = right - left
//   Motor 2 = truc DOC  (elevation)  errorY = top - bottom
bool autoTrack = false;

// Deadband co hysteresis: vao nhanh, ra cham de tranh rung quanh diem can bang
const int LIGHT_DEADBAND_IN    = 80;   // |error| < nay -> STOP (bat dau)
const int LIGHT_DEADBAND_OUT   = 160;  // phai > nay moi chay lai sau khi STOP
const int LIGHT_MIN            = 300;   // TB 4 cam bien < nay -> troi toi, dung het

PIDController pidX(0.15f, 0.02f, 0.02f);  // truc ngang — giam Kd de giam nhay
PIDController pidY(0.15f, 0.02f, 0.02f);  // truc doc

// PID vong trong: error = toc do encoder muc tieu - toc do encoder thuc te.
// Output la so micro-giay hieu chinh STEP delay; error duong -> giam delay de tang toc.
PIDController pidEncoder1(20.0f, 5.0f, 0.0f);
PIDController pidEncoder2(20.0f, 5.0f, 0.0f);

const float ENCODER_SPEED_MIN_PPS = 5.0f;
const float ENCODER_SPEED_MAX_PPS = 45.0f;
const float ENCODER_SPEED_FILTER  = 0.35f;
const float ENCODER_PID_MAX_US    = 1200.0f;

float targetEncoderSpeed1 = 0, targetEncoderSpeed2 = 0;
float actualEncoderSpeed1 = 0, actualEncoderSpeed2 = 0;
```

```
unsigned long baseStepDelayUs1 = STEP_DELAY_MANUAL;
unsigned long baseStepDelayUs2 = STEP_DELAY_MANUAL;

// Output deadband: |output| < nay -> dung (tranh rung khi gan ve 0)
const float PID_OUT_DEAD = 25.0f;

// Gia tri debug de hien thi tren web
float lastErrorX = 0, lastErrorY = 0;
float lastOutX   = 0, lastOutY   = 0;

// Hysteresis state cho deadband (global de reset khi toggle auto/home)
bool axisXStopped = true;   // bat dau tu trang thai stopped
bool axisYStopped = true;

// Map |PID output| -> step delay (us): output lon = quay nhanh
unsigned long delayFromPidOut(float absOut) {
  float span = 1000.0f - PID_OUT_DEAD;
  if (span < 1.0f) span = 1.0f;
  float t = (absOut - PID_OUT_DEAD) / span;
  if (t < 0.0f) t = 0.0f;
  if (t > 1.0f) t = 1.0f;
    return (unsigned long)(STEP_DELAY_MAX - t * (STEP_DELAY_MAX -
STEP_DELAY_MIN));
}

float speedFromPidOut(float absOut) {
  float t = (absOut - PID_OUT_DEAD) / (1000.0f - PID_OUT_DEAD);
```

```
  if (t < 0.0f) t = 0.0f;
  if (t > 1.0f) t = 1.0f;
    return ENCODER_SPEED_MIN_PPS + t * (ENCODER_SPEED_MAX_PPS -
ENCODER_SPEED_MIN_PPS);
}

void resetEncoderSpeedPid() {
  pidEncoder1.reset();
  pidEncoder2.reset();
  targetEncoderSpeed1 = targetEncoderSpeed2 = 0;
  actualEncoderSpeed1 = actualEncoderSpeed2 = 0;
}

void updateEncoderSpeedPid() {
  static long previousCount1 = 0;
  static long previousCount2 = 0;
  static unsigned long previousMs = 0;

  unsigned long nowMs = millis();
  long count1 = readEncoderCount();
  long count2 = readEncoder2Count();

  if (previousMs == 0 || !autoTrack || homing1 || homing2) {
    previousCount1 = count1;
    previousCount2 = count2;
    previousMs = nowMs;
    resetEncoderSpeedPid();
```

```
  return;
 }

 unsigned long elapsedMs = nowMs - previousMs;
 if (elapsedMs < 100) return;

 float dt = elapsedMs / 1000.0f;
 float measured1 = fabsf((float)(count1 - previousCount1)) / dt;
 float measured2 = fabsf((float)(count2 - previousCount2)) / dt;
    actualEncoderSpeed1 += ENCODER_SPEED_FILTER * (measured1 -
actualEncoderSpeed1);
    actualEncoderSpeed2 += ENCODER_SPEED_FILTER * (measured2 -
actualEncoderSpeed2);

 previousCount1 = count1;
 previousCount2 = count2;
 previousMs = nowMs;

 if (state1 != STOP && targetEncoderSpeed1 > 0) {
  float correction = pidEncoder1.compute(targetEncoderSpeed1 - actualEncoderSpeed1);
  if (correction > ENCODER_PID_MAX_US) correction = ENCODER_PID_MAX_US;
        if (correction < -ENCODER_PID_MAX_US) correction = -
ENCODER_PID_MAX_US;
  long correctedDelay = (long)baseStepDelayUs1 - (long)correction;
       if (correctedDelay < (long)STEP_DELAY_MIN) correctedDelay =
STEP_DELAY_MIN;
```

```
    if (correctedDelay > (long)STEP_DELAY_MAX) correctedDelay = STEP_DELAY_MAX;
   stepDelayUs1 = smoothStepDelay(stepDelayUs1, (unsigned long)correctedDelay);
  } else {
   pidEncoder1.reset();
   actualEncoderSpeed1 = 0;
  }

  if (state2 != STOP && targetEncoderSpeed2 > 0) {
   float correction = pidEncoder2.compute(targetEncoderSpeed2 - actualEncoderSpeed2);
   if (correction > ENCODER_PID_MAX_US) correction = ENCODER_PID_MAX_US;
    if (correction < -ENCODER_PID_MAX_US) correction = -ENCODER_PID_MAX_US;
   long correctedDelay = (long)baseStepDelayUs2 - (long)correction;
    if (correctedDelay < (long)STEP_DELAY_MIN) correctedDelay = STEP_DELAY_MIN;
    if (correctedDelay > (long)STEP_DELAY_MAX) correctedDelay = STEP_DELAY_MAX;
   stepDelayUs2 = smoothStepDelay(stepDelayUs2, (unsigned long)correctedDelay);
  } else {
   pidEncoder2.reset();
   actualEncoderSpeed2 = 0;
  }
}

unsigned long smoothStepDelay(unsigned long oldDelay, unsigned long targetDelay) {
  // Chi cho delay thay doi tu tu de motor khong bi giat toc do
```

```
  const unsigned long MAX_CHANGE_US = 120;
    if (targetDelay > oldDelay + MAX_CHANGE_US) return oldDelay +
MAX_CHANGE_US;
    if (oldDelay > targetDelay + MAX_CHANGE_US) return oldDelay -
MAX_CHANGE_US;
  return targetDelay;
}

// Ap dung output PID len 1 truc — co hysteresis + ramp toc do
void applyPidToMotor(int motor, float output, bool &axisStopped) {
  float absOut = fabsf(output);

  // Output hysteresis: neu dang dung thi can output lon hon moi chay lai
  float stopThreshold  = PID_OUT_DEAD;
  float startThreshold = PID_OUT_DEAD * 1.8f;
  if (axisStopped && absOut < startThreshold) {
    output = 0;
    absOut = 0;
  }

  if (absOut < stopThreshold) {
    axisStopped = true;
    if (motor == 1) {
      if (state1 != STOP) { state1 = STOP; digitalWrite(PIN_STEP1, LOW); }
      targetEncoderSpeed1 = 0;
      stepDelayUs1 = smoothStepDelay(stepDelayUs1, STEP_DELAY_MANUAL);
    } else {
```

```
    if (state2 != STOP) { state2 = STOP; digitalWrite(PIN_STEP2, LOW); }
    targetEncoderSpeed2 = 0;
    stepDelayUs2 = smoothStepDelay(stepDelayUs2, STEP_DELAY_MANUAL);
   }
   return;
  }

  axisStopped = false;
  MotorState ns = (output > 0) ? FORWARD : BACKWARD;
  unsigned long targetDly = delayFromPidOut(absOut);

  if (motor == 1) {
   state1 = ns;
   targetEncoderSpeed1 = speedFromPidOut(absOut);
   baseStepDelayUs1 = targetDly;
  } else {
   state2 = ns;
   targetEncoderSpeed2 = speedFromPidOut(absOut);
   baseStepDelayUs2 = targetDly;
  }
}

const char* WEB_PAGE = R"rawliteral(<!DOCTYPE html>
<html lang="vi">
<head>
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no">
<title>Điều khiển mô hình năng lượng mặt trời</title>
<style>
  * { margin:0; padding:0; box-sizing:border-box; }
  body {
    font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
    background: #0f172a;
    color: #e2e8f0;
    min-height: 100vh;
    padding: 12px 12px 80px 12px;
    -webkit-tap-highlight-color: transparent;
  }
  .header { text-align: center; padding: 8px 0 14px 0; }
  .header h1 { font-size: 18px; font-weight: 700; color: #f1f5f9; }
  .header p { font-size: 11px; color: #64748b; margin-top: 2px; }

  .sensors {
    display: grid; grid-template-columns: repeat(3, 1fr); gap: 8px;
    margin-bottom: 14px;
  }
  .sensor-card {
    background: #1e293b; border-radius: 14px; padding: 12px 8px; text-align: center;
    border: 1px solid #334155;
  }
  .sensor-label { font-size: 10px; color: #94a3b8; text-transform: uppercase; letter-spacing: 1px; margin-bottom: 4px; }
```

```
.sensor-value { font-size: 22px; font-weight: 700; color: #38bdf8; }
.sensor-unit { font-size: 11px; color: #64748b; }

.motor-section, .pid-section, .chart-section {
  background: #1e293b; border-radius: 18px; padding: 16px;
  margin-bottom: 12px; border: 1px solid #334155;
}
.motor-header, .section-header {
  display: flex; align-items: center; justify-content: space-between; margin-bottom: 12px;
}
.motor-name, .section-title { font-size: 16px; font-weight: 600; }
.motor-pins, .section-sub { font-size: 10px; color: #64748b; }
.motor-status {
  display: flex; align-items: center; gap: 8px;
  background: #0f172a; border-radius: 10px; padding: 8px 12px; margin-bottom: 12px;
}
.status-dot {
  width: 10px; height: 10px; border-radius: 50%; flex-shrink: 0;
  background: #64748b; transition: background 0.3s, box-shadow 0.3s;
}
.status-dot.fwd { background: #22c55e; box-shadow: 0 0 8px rgba(34,197,94,0.6); }
.status-dot.rev { background: #f59e0b; box-shadow: 0 0 8px rgba(245,158,11,0.6); }
.status-label { font-size: 13px; color: #94a3b8; }

.btn-row { display: flex; gap: 8px; }
button {
  flex: 1; border: none; border-radius: 12px; padding: 14px 0;
```

```
  font-size: 15px; font-weight: 600; color: #fff; cursor: pointer;
  -webkit-tap-highlight-color: transparent; user-select: none; outline: none;
  transition: transform 0.1s, opacity 0.15s;
}
button:active { transform: scale(0.95); opacity: 0.85; }

.btn-go { background: #059669; }
.btn-back { background: #d97706; }
.btn-stop-one { background: #dc2626; flex: 0 0 44px; padding: 14px 0; font-size: 18px;
flex-shrink: 0; }

.bottom-bar {
  position: fixed; bottom: 0; left: 0; right: 0;
  padding: 10px 12px; background: #1e293b;
  border-top: 1px solid #334155; display: flex; gap: 10px; z-index: 20;
}
.btn-all-stop {
  flex: 1; border: none; border-radius: 12px; padding: 15px 0;
  font-size: 15px; font-weight: 700; color: #fff; cursor: pointer;
  background: #7c3aed; letter-spacing: 0.5px;
}
.btn-all-stop:active { transform: scale(0.95); opacity: 0.85; }
.btn-auto {
  flex: 1; border: none; border-radius: 12px; padding: 15px 0;
  font-size: 15px; font-weight: 700; color: #fff; cursor: pointer;
  background: #475569; letter-spacing: 0.5px;
  transition: transform 0.1s, opacity 0.15s, background 0.2s;
```

```
}
.btn-auto.on { background: #0ea5e9; }
.btn-auto:active { transform: scale(0.95); opacity: 0.85; }
.btn-home {
  flex: 1; border: none; border-radius: 12px; padding: 15px 0;
  font-size: 15px; font-weight: 700; color: #fff; cursor: pointer;
  background: #0d9488; letter-spacing: 0.5px;
}
.btn-home:active { transform: scale(0.95); opacity: 0.85; }

.footer-bar {
  display: flex; justify-content: space-between; align-items: center;
  padding: 0 4px 4px 4px;
}
.ip-tag {
  background: #334155; color: #94a3b8; font-size: 10px;
  border-radius: 8px; padding: 4px 10px;
}
.refresh-indicator { font-size: 10px; color: #64748b; }

.chart-tabs { display: flex; gap: 6px; margin-bottom: 10px; }
.chart-tab {
  flex: 1; border: none; border-radius: 9px; padding: 8px 0;
  font-size: 12px; font-weight: 600; color: #94a3b8; cursor: pointer;
  background: #0f172a; letter-spacing: 0.3px;
}
.chart-tab.active { background: #334155; color: #f1f5f9; }
```

```
.chart-section canvas { width: 100%; height: auto; display: block; border-radius: 10px; }
.chart-actions { display: flex; align-items: center; gap: 8px; margin-top: 10px; }
.btn-export, .btn-clear, .btn-pid {
  border: none; border-radius: 9px; padding: 9px 14px;
  font-size: 12px; font-weight: 600; color: #fff; cursor: pointer;
}
.btn-export { background: #2563eb; }
.btn-clear { background: #475569; }
.btn-pid { background: #0ea5e9; flex: 1; padding: 12px 0; font-size: 14px; border-radius:
12px; }
.log-info { font-size: 11px; color: #64748b; margin-left: auto; }

.pid-grid {
  display: grid; grid-template-columns: repeat(3, 1fr); gap: 8px; margin-bottom: 10px;
}
.pid-field label {
  display: block; font-size: 10px; color: #94a3b8; text-transform: uppercase;
  letter-spacing: 1px; margin-bottom: 4px;
}
.pid-field input {
  width: 100%; background: #0f172a; border: 1px solid #334155; border-radius: 10px;
  color: #f1f5f9; font-size: 15px; font-weight: 600; padding: 10px 8px; text-align: center;
  outline: none;
}
.pid-field input:focus { border-color: #0ea5e9; }
.pid-live {
  display: grid; grid-template-columns: 1fr 1fr; gap: 8px; margin-bottom: 10px;
```

```
  }
  .pid-live-card {
    background: #0f172a; border-radius: 10px; padding: 10px; text-align: center;
  }
  .pid-live-label { font-size: 10px; color: #64748b; margin-bottom: 4px; }
  .pid-live-val { font-size: 14px; font-weight: 600; color: #a78bfa; }
  .pid-hint { font-size: 11px; color: #64748b; margin-top: 8px; line-height: 1.4; }
</style>
</head>
<body>

<div class="header">
  <h1>Điều khiển mô hình năng lượng mặt trời</h1>
  <p>192.168.4.1 &middot; WiFi: MotorControl &middot; PID</p>
</div>

<div class="sensors">
  <div class="sensor-card">
    <div class="sensor-label">Điện áp</div>
    <div class="sensor-value" id="voltage">--</div>
    <div class="sensor-unit">V</div>
  </div>
  <div class="sensor-card">
    <div class="sensor-label">Dòng điện</div>
    <div class="sensor-value" id="current">--</div>
    <div class="sensor-unit">mA</div>
  </div>
```

```
<div class="sensor-card">
  <div class="sensor-label">Công suất</div>
  <div class="sensor-value" id="power">--</div>
  <div class="sensor-unit">mW</div>
</div>
<div class="sensor-card">
  <div class="sensor-label">Ánh sáng A</div>
  <div class="sensor-value" id="lux1">--</div>
  <div class="sensor-unit">D34 TL</div>
</div>
<div class="sensor-card">
  <div class="sensor-label">Ánh sáng B</div>
  <div class="sensor-value" id="lux2">--</div>
  <div class="sensor-unit">D35 TR</div>
</div>
<div class="sensor-card">
  <div class="sensor-label">Ánh sáng C</div>
  <div class="sensor-value" id="lux3">--</div>
  <div class="sensor-unit">D32 BL</div>
</div>
<div class="sensor-card">
  <div class="sensor-label">Ánh sáng D</div>
  <div class="sensor-value" id="lux4">--</div>
  <div class="sensor-unit">D33 BR</div>
</div>
<div class="sensor-card">
  <div class="sensor-label">Vị trí trục dưới</div>
```

```
  <div class="sensor-value" id="encoder">--</div>
  <div class="sensor-unit">xung encoder</div>
 </div>
 <div class="sensor-card">
  <div class="sensor-label">Vị trí trục dọc</div>
  <div class="sensor-value" id="encoder2">--</div>
  <div class="sensor-unit">xung encoder</div>
 </div>
</div>

<div class="chart-section">
 <div class="chart-tabs">
                <button class="chart-tab active" data-metric="power"
onclick="selectChart(this)">Công suất</button>
    <button class="chart-tab" data-metric="voltage" onclick="selectChart(this)">Điện
áp</button>
    <button class="chart-tab" data-metric="current" onclick="selectChart(this)">Dòng
điện</button>
     <button class="chart-tab" data-metric="light" onclick="selectChart(this)">Ánh
sáng</button>
 </div>
 <canvas id="chart" width="600" height="220"></canvas>
 <div class="chart-actions">
  <span class="log-info" id="logInfo">0 mẫu</span>
  <button class="btn-export" onclick="exportCSV()">Xuất CSV</button>
  <button class="btn-clear" onclick="clearLog()">Xóa</button>
 </div>
```

```
</div>

<div class="pid-section">
  <div class="section-header">
    <span class="section-title">PID Auto-Track</span>
    <span class="section-sub">2 trục · tốc độ biến thiên</span>
  </div>
  <div class="pid-live">
    <div class="pid-live-card">
      <div class="pid-live-label">Error X (ngang)</div>
      <div class="pid-live-val" id="errX">--</div>
    </div>
    <div class="pid-live-card">
      <div class="pid-live-label">Out X → delay</div>
      <div class="pid-live-val" id="outX">--</div>
    </div>
    <div class="pid-live-card">
      <div class="pid-live-label">Error Y (dọc)</div>
      <div class="pid-live-val" id="errY">--</div>
    </div>
    <div class="pid-live-card">
      <div class="pid-live-label">Out Y → delay</div>
      <div class="pid-live-val" id="outY">--</div>
    </div>
    <div class="pid-live-card">
      <div class="pid-live-label">Encoder 1 mục tiêu / thực</div>
      <div class="pid-live-val" id="speed1">--</div>
```

```
  </div>
  <div class="pid-live-card">
    <div class="pid-live-label">Encoder 2 mục tiêu / thực</div>
    <div class="pid-live-val" id="speed2">--</div>
  </div>
</div>
<div class="pid-grid">
  <div class="pid-field">
    <label>Kp</label>
    <input type="number" id="kp" step="0.01" value="0.15">
  </div>
  <div class="pid-field">
    <label>Ki</label>
    <input type="number" id="ki" step="0.01" value="0.02">
  </div>
  <div class="pid-field">
    <label>Kd</label>
    <input type="number" id="kd" step="0.01" value="0.02">
  </div>
</div>
<button class="btn-pid" onclick="applyPid()">Áp dụng Kp / Ki / Kd</button>
<p class="pid-hint">Gợi ý: tăng Kp nếu quay chậm; tăng Kd nếu overshoot; Ki nhỏ để bù sai số tĩnh. Deadband: dừng &lt; 80, chạy lại &gt; 160.</p>
<div class="section-header" style="margin-top:16px">
  <span class="section-title">PID tốc độ encoder</span>
  <span class="section-sub">vòng trong · cả 2 trục</span>
</div>
```

```
  <div class="pid-grid">
    <div class="pid-field"><label>Kp encoder</label><input type="number" id="ekp" step="1" value="20"></div>
    <div class="pid-field"><label>Ki encoder</label><input type="number" id="eki" step="1" value="5"></div>
    <div class="pid-field"><label>Kd encoder</label><input type="number" id="ekd" step="0.1" value="0"></div>
  </div>
  <button class="btn-pid" onclick="applyEncoderPid()">Áp dụng PID encoder</button>
  <p class="pid-hint">Mục tiêu/thực tế tính bằng xung/giây. Tăng Kp nếu tốc độ thực bám chậm; tăng Ki từ từ nếu còn sai lệch; giữ Kd thấp vì encoder cơ khí dễ nhiễu.</p>
</div>

<div class="chart-section">
  <div class="section-header">
    <span class="section-title">Biểu đồ PID encoder</span>
    <span class="section-sub">xung/giây · 60 mẫu</span>
  </div>
  <div class="chart-tabs">
        <button class="chart-tab pid-axis-tab active" data-axis="1" onclick="selectPidAxis(this)">Trục 1</button>
        <button class="chart-tab pid-axis-tab" data-axis="2" onclick="selectPidAxis(this)">Trục 2</button>
  </div>
  <canvas id="pidChart" width="600" height="240"></canvas>
  <div class="chart-actions">
    <span style="font-size:11px;color:#38bdf8">● Mục tiêu</span>
```

```
  <span style="font-size:11px;color:#f59e0b">● Thực tế</span>
    <button class="btn-clear" style="margin-left:auto" onclick="clearPidChart()">Xóa PID</button>
 </div>
</div>

<div class="motor-section">
 <div class="motor-header">
  <span class="motor-name">Động cơ 1 (Ngang)</span>
  <span class="motor-pins">D2 / D4</span>
 </div>
 <div class="motor-status">
  <div class="status-dot" id="dot1"></div>
  <span class="status-label" id="txt1">Đã dừng</span>
 </div>
 <div class="btn-row">
  <button class="btn-go" onclick="sendCmd(1,'F')">THUẬN</button>
  <button class="btn-back" onclick="sendCmd(1,'B')">NGƯỢC</button>
  <button class="btn-stop-one" onclick="sendCmd(1,'S')">&#9632;</button>
 </div>
</div>

<div class="motor-section">
 <div class="motor-header">
  <span class="motor-name">Động cơ 2 (Dọc)</span>
  <span class="motor-pins">D18 / D19</span>
 </div>
```

```
  <div class="motor-status">
    <div class="status-dot" id="dot2"></div>
    <span class="status-label" id="txt2">Đã dừng</span>
  </div>
  <div class="btn-row">
    <button class="btn-go" onclick="sendCmd(2,'F')">THUẬN</button>
    <button class="btn-back" onclick="sendCmd(2,'B')">NGƯỢC</button>
    <button class="btn-stop-one" onclick="sendCmd(2,'S')">&#9632;</button>
  </div>
</div>

<div class="footer-bar">
  <span class="ip-tag">INA219 &middot; PID</span>
  <span class="refresh-indicator" id="refreshInfo">Cập nhật 1s</span>
</div>

<div class="bottom-bar">
    <button class="btn-auto" id="btnAuto" onclick="toggleAuto()">TỰ ĐỘNG: TẮT</button>
  <button class="btn-home" onclick="goHome()">VỀ GỐC</button>
  <button class="btn-all-stop" onclick="sendCmd(0,'S')">DỪNG TẤT CẢ</button>
</div>

<script>
function updateUI(motor, state) {
  var dot = document.getElementById('dot' + motor);
  var txt = document.getElementById('txt' + motor);
```

```
  dot.classList.remove('fwd','rev');
  if (state === 'F') { dot.classList.add('fwd'); txt.textContent = 'Đang xoay THUẬN'; }
  else if (state === 'B') { dot.classList.add('rev'); txt.textContent = 'Đang xoay NGƯỢC'; }
  else { txt.textContent = 'Đã dừng'; }
}

function sendCmd(motor, cmd) {
  fetch('/cmd?m=' + motor + '&c=' + cmd)
    .then(function(r) { return r.text(); })
    .then(function(state) {
      if (motor === 0) { updateUI(1, 'S'); updateUI(2, 'S'); }
      else { updateUI(motor, state); }
    });
}

var autoOn = false;

function setAutoUI(on) {
  autoOn = on;
  var b = document.getElementById('btnAuto');
  b.textContent = 'TỰ ĐỘNG: ' + (on ? 'BẬT' : 'TẮT');
  b.classList.toggle('on', on);
}

function toggleAuto() {
  fetch('/auto?v=' + (autoOn ? 0 : 1))
    .then(function(r) { return r.text(); })
```

```
    .then(function(s) { setAutoUI(s === '1'); });
}

function goHome() {
  fetch('/home')
    .then(function(r) { return r.text(); })
    .then(function() { setAutoUI(false); });
}

function applyPid() {
  var kp = document.getElementById('kp').value;
  var ki = document.getElementById('ki').value;
  var kd = document.getElementById('kd').value;
  fetch('/pid?kp=' + kp + '&ki=' + ki + '&kd=' + kd)
    .then(function(r) { return r.json(); })
    .then(function(d) {
      document.getElementById('kp').value = d.kp;
      document.getElementById('ki').value = d.ki;
      document.getElementById('kd').value = d.kd;
    });
}

function applyEncoderPid() {
  var kp = document.getElementById('ekp').value;
  var ki = document.getElementById('eki').value;
  var kd = document.getElementById('ekd').value;
  fetch('/encoderpid?kp=' + kp + '&ki=' + ki + '&kd=' + kd)
```

```
    .then(function(r) { return r.json(); })
    .then(function(d) {
      document.getElementById('ekp').value = d.kp;
      document.getElementById('eki').value = d.ki;
      document.getElementById('ekd').value = d.kd;
    });
}

function fetchSensor() {
  var t0 = Date.now();
  fetch('/sensor')
    .then(function(r) { return r.json(); })
    .then(function(d) {
      document.getElementById('voltage').textContent = d.v.toFixed(2);
      document.getElementById('current').textContent = d.c.toFixed(1);
      document.getElementById('power').textContent = d.p.toFixed(1);
      document.getElementById('lux1').textContent = d.l1;
      document.getElementById('lux2').textContent = d.l2;
      document.getElementById('lux3').textContent = d.l3;
      document.getElementById('lux4').textContent = d.l4;
      document.getElementById('encoder').textContent = d.enc;
      document.getElementById('encoder2').textContent = d.enc2;
      setAutoUI(d.auto === 1);

      document.getElementById('errX').textContent = d.ex.toFixed(0);
      document.getElementById('errY').textContent = d.ey.toFixed(0);
      document.getElementById('outX').textContent = d.ox.toFixed(1) + ' / ' + d.dx + 'us';
```

```
    document.getElementById('outY').textContent = d.oy.toFixed(1) + ' / ' + d.dy + 'us';
        document.getElementById('speed1').textContent = d.st1.toFixed(1) + ' / ' +
d.sa1.toFixed(1) + ' x/s';
        document.getElementById('speed2').textContent = d.st2.toFixed(1) + ' / ' +
d.sa2.toFixed(1) + ' x/s';

    if (typeof d.kp === 'number') {
     if (document.activeElement.id !== 'kp') document.getElementById('kp').value = d.kp;
     if (document.activeElement.id !== 'ki') document.getElementById('ki').value = d.ki;
     if (document.activeElement.id !== 'kd') document.getElementById('kd').value = d.kd;
    }
    if (typeof d.ekp === 'number') {
       if (document.activeElement.id !== 'ekp') document.getElementById('ekp').value =
d.ekp;
        if (document.activeElement.id !== 'eki') document.getElementById('eki').value =
d.eki;
       if (document.activeElement.id !== 'ekd') document.getElementById('ekd').value =
d.ekd;
    }

    if (d.auto === 1) {
     updateUI(1, d.ox > 25 ? 'F' : (d.ox < -25 ? 'B' : 'S'));
     updateUI(2, d.oy > 25 ? 'F' : (d.oy < -25 ? 'B' : 'S'));
    }

    pushHistory(d);
    drawChart();
```

```
    drawPidChart();
    document.getElementById('refreshInfo').textContent = 'Cập nhật ' + (Date.now() - t0)
+ 'ms';
  });
}

var MAXPTS = 60;
var hist = { power: [], voltage: [], current: [], light: [] };
var pidHist = { t1: [], a1: [], t2: [], a2: [] };
var pidAxis = 1;
var curMetric = 'power';
var MAXLOG = 3600;
var logData = [];

function pushHistory(d) {
 hist.power.push(d.p);
 hist.voltage.push(d.v);
 hist.current.push(d.c);
 hist.light.push((d.l1 + d.l2 + d.l3 + d.l4) / 4);
 for (var k in hist) { if (hist[k].length > MAXPTS) hist[k].shift(); }
 pidHist.t1.push(d.st1); pidHist.a1.push(d.sa1);
 pidHist.t2.push(d.st2); pidHist.a2.push(d.sa2);
 for (var p in pidHist) { if (pidHist[p].length > MAXPTS) pidHist[p].shift(); }

 var now = new Date();
 logData.push({
  t: now.toLocaleString('vi-VN'),
```

```
    v: d.v, c: d.c, p: d.p,
    l1: d.l1, l2: d.l2, l3: d.l3, l4: d.l4,
    enc: d.enc,
    enc2: d.enc2,
    ex: d.ex, ey: d.ey, ox: d.ox, oy: d.oy,
    auto: d.auto
  });
  if (logData.length > MAXLOG) logData.shift();
  updateLogInfo();
}

function exportCSV() {
  if (logData.length === 0) { alert('Chưa có dữ liệu để xuất'); return; }
      var lines = ['Thoi gian,Dien ap (V),Dong (mA),Cong suat (mW),AS_D34,AS_D35,AS_D32,AS_D33,Encoder1,Encoder2,ErrX,ErrY,OutX,OutY,Auto'];
  for (var i = 0; i < logData.length; i++) {
    var r = logData[i];
    lines.push('"' + r.t + '",' + r.v + ',' + r.c + ',' + r.p + ','
          + r.l1 + ',' + r.l2 + ',' + r.l3 + ',' + r.l4 + ',' + r.enc + ',' + r.enc2 + ','
          + r.ex + ',' + r.ey + ',' + r.ox + ',' + r.oy + ',' + r.auto);
  }
  var csv = '\uFEFF' + lines.join('\r\n');
  var blob = new Blob([csv], { type: 'text/csv;charset=utf-8;' });
  var url = URL.createObjectURL(blob);
  var a = document.createElement('a');
  var ts = new Date();
```

```
  var name = 'solar_' + ts.getFullYear()
          + ('0' + (ts.getMonth() + 1)).slice(-2)
          + ('0' + ts.getDate()).slice(-2) + '_'
          + ('0' + ts.getHours()).slice(-2)
          + ('0' + ts.getMinutes()).slice(-2)
          + ('0' + ts.getSeconds()).slice(-2) + '.csv';
  a.href = url; a.download = name;
  document.body.appendChild(a); a.click();
  document.body.removeChild(a);
  URL.revokeObjectURL(url);
}

function clearLog() {
  logData = [];
  for (var k in hist) hist[k] = [];
  updateLogInfo();
  drawChart();
}

function updateLogInfo() {
  document.getElementById('logInfo').textContent = logData.length + ' mẫu';
}

var CHART_CFG = {
  power:   { label: 'Công suất (mW)', color: '#38bdf8', dec: 1 },
  voltage: { label: 'Điện áp (V)',    color: '#22c55e', dec: 2 },
  current: { label: 'Dòng điện (mA)', color: '#f59e0b', dec: 1 },
```

```
  light:   { label: 'Ánh sáng TB',    color: '#a78bfa', dec: 0 }
};

function selectChart(btn) {
  curMetric = btn.getAttribute('data-metric');
  var tabs = document.querySelectorAll('.chart-tab');
  for (var i = 0; i < tabs.length; i++) tabs[i].classList.remove('active');
  btn.classList.add('active');
  drawChart();
}

function drawChart() {
  var cv = document.getElementById('chart');
  var ctx = cv.getContext('2d');
  var W = cv.width, H = cv.height;
  var padL = 46, padR = 10, padT = 12, padB = 20;
  var cfg = CHART_CFG[curMetric];
  var data = hist[curMetric];

  ctx.clearRect(0, 0, W, H);
  ctx.fillStyle = '#0f172a';
  ctx.fillRect(0, 0, W, H);

  var mn = Infinity, mx = -Infinity;
  for (var i = 0; i < data.length; i++) {
    if (data[i] < mn) mn = data[i];
    if (data[i] > mx) mx = data[i];
```

```
}
if (!isFinite(mn)) { mn = 0; mx = 1; }
if (mn === mx) { mn -= 1; mx += 1; }
var range = mx - mn;
mn -= range * 0.1; mx += range * 0.1;

var plotW = W - padL - padR, plotH = H - padT - padB;

ctx.strokeStyle = '#1e293b';
ctx.fillStyle = '#64748b';
ctx.font = '10px sans-serif';
ctx.lineWidth = 1;
ctx.textAlign = 'right';
for (var g = 0; g <= 4; g++) {
  var y = padT + plotH * g / 4;
  var val = mx - (mx - mn) * g / 4;
  ctx.beginPath(); ctx.moveTo(padL, y); ctx.lineTo(W - padR, y); ctx.stroke();
  ctx.fillText(val.toFixed(cfg.dec), padL - 5, y + 3);
}

if (data.length < 2) {
  ctx.fillStyle = '#475569'; ctx.textAlign = 'center';
  ctx.fillText('Đang thu thập dữ liệu...', W / 2, H / 2);
  return;
}

function px(i) { return padL + plotW * i / (MAXPTS - 1); }
```

```
  function py(v) { return padT + plotH * (1 - (v - mn) / (mx - mn)); }

  ctx.beginPath();
  ctx.moveTo(px(0), py(data[0]));
  for (var i = 1; i < data.length; i++) ctx.lineTo(px(i), py(data[i]));
  ctx.lineTo(px(data.length - 1), padT + plotH);
  ctx.lineTo(px(0), padT + plotH);
  ctx.closePath();
  ctx.fillStyle = cfg.color + '22';
  ctx.fill();

  ctx.beginPath();
  ctx.moveTo(px(0), py(data[0]));
  for (var i = 1; i < data.length; i++) ctx.lineTo(px(i), py(data[i]));
  ctx.strokeStyle = cfg.color;
  ctx.lineWidth = 2;
  ctx.stroke();

  var last = data[data.length - 1];
  ctx.fillStyle = cfg.color;
  ctx.beginPath(); ctx.arc(px(data.length - 1), py(last), 3, 0, 6.29); ctx.fill();
  ctx.textAlign = 'left';
  ctx.fillText(cfg.label + ': ' + last.toFixed(cfg.dec), padL + 4, padT + 10);
}

function selectPidAxis(btn) {
  pidAxis = Number(btn.getAttribute('data-axis'));
```

```
  var tabs = document.querySelectorAll('.pid-axis-tab');
  for (var i = 0; i < tabs.length; i++) tabs[i].classList.remove('active');
  btn.classList.add('active');
  drawPidChart();
}

function clearPidChart() {
  pidHist = { t1: [], a1: [], t2: [], a2: [] };
  drawPidChart();
}

function drawPidChart() {
  var cv = document.getElementById('pidChart');
  var ctx = cv.getContext('2d');
  var W = cv.width, H = cv.height;
  var padL = 42, padR = 12, padT = 25, padB = 22;
  var target = pidHist['t' + pidAxis];
  var actual = pidHist['a' + pidAxis];
  var maxVal = 10;

  for (var i = 0; i < target.length; i++) {
    if (target[i] > maxVal) maxVal = target[i];
    if (actual[i] > maxVal) maxVal = actual[i];
  }
  maxVal = Math.ceil(maxVal * 1.15 / 5) * 5;

  ctx.fillStyle = '#0f172a';
```

```
ctx.fillRect(0, 0, W, H);
var plotW = W - padL - padR, plotH = H - padT - padB;
ctx.font = '10px sans-serif';
ctx.textAlign = 'right';
ctx.lineWidth = 1;
for (var g = 0; g <= 5; g++) {
  var y = padT + plotH * g / 5;
  ctx.strokeStyle = '#1e293b';
  ctx.beginPath(); ctx.moveTo(padL, y); ctx.lineTo(W - padR, y); ctx.stroke();
  ctx.fillStyle = '#64748b';
  ctx.fillText((maxVal * (5 - g) / 5).toFixed(0), padL - 5, y + 3);
}

if (target.length < 2) {
  ctx.fillStyle = '#475569'; ctx.textAlign = 'center';
  ctx.fillText('Đang thu thập dữ liệu PID...', W / 2, H / 2);
  return;
}

function px(i) { return padL + plotW * i / (MAXPTS - 1); }
function py(v) { return padT + plotH * (1 - v / maxVal); }
function line(data, color) {
  ctx.beginPath(); ctx.moveTo(px(0), py(data[0]));
  for (var j = 1; j < data.length; j++) ctx.lineTo(px(j), py(data[j]));
  ctx.strokeStyle = color; ctx.lineWidth = 2; ctx.stroke();
}
line(target, '#38bdf8');
```

```
  line(actual, '#f59e0b');

  ctx.textAlign = 'left'; ctx.fillStyle = '#94a3b8';
  ctx.fillText('Trục ' + pidAxis + ' · mục tiêu ' + target[target.length - 1].toFixed(1)
    + ' · thực ' + actual[actual.length - 1].toFixed(1) + ' x/s', padL, 14);
}

window.addEventListener('load', function() {
  fetch('/state?m=1').then(function(r) { return r.text(); }).then(function(s) { updateUI(1, s);
});
  fetch('/state?m=2').then(function(r) { return r.text(); }).then(function(s) { updateUI(2, s);
});
  fetchSensor();
  setInterval(fetchSensor, 1000);
});
</script>
</body>
</html>)rawliteral";

void handleRoot() {
  server.send(200, "text/html; charset=utf-8", WEB_PAGE);
}

void setMotorState(int motor, char cmd) {
  MotorState ns = (cmd == 'F') ? FORWARD : (cmd == 'B') ? BACKWARD : STOP;
  const char* dirStr = (cmd == 'F') ? "Thuan" : (cmd == 'B') ? "Nguoc" : "Dung";
```

```
  // Dieu khien tay / home: dung toc do co dinh
  if (motor == 1) {
    state1 = ns;
    stepDelayUs1 = STEP_DELAY_MANUAL;
    if (cmd == 'S') digitalWrite(PIN_STEP1, LOW);
  } else if (motor == 2) {
    state2 = ns;
    stepDelayUs2 = STEP_DELAY_MANUAL;
    if (cmd == 'S') digitalWrite(PIN_STEP2, LOW);
  } else {
    state1 = state2 = ns;
    stepDelayUs1 = stepDelayUs2 = STEP_DELAY_MANUAL;
    if (cmd == 'S') {
      digitalWrite(PIN_STEP1, LOW);
      digitalWrite(PIN_STEP2, LOW);
    }
  }
  Serial.print(">> Dong co "); Serial.print(motor == 0 ? "ALL" : String(motor));
  Serial.print(": "); Serial.println(dirStr);
}

void handleCmd() {
  if (!server.hasArg("m") || !server.hasArg("c")) {
    server.send(400, "text/plain", "Thieu tham so");
    return;
  }
  int motor = server.arg("m").toInt();
```

```
  char cmd  = server.arg("c")[0];
  // Lenh tay -> huy home + tat auto (tranh PID ghi de)
  if (motor == 1 || motor == 0) homing1 = false;
  if (motor == 2 || motor == 0) homing2 = false;
  if (cmd != 'S') {
    autoTrack = false;
    pidX.reset();
    pidY.reset();
  }
  setMotorState(motor, cmd);
  server.send(200, "text/plain", String(cmd));
}

void handleState() {
  if (!server.hasArg("m")) { server.send(400, "text/plain", "Thieu tham so"); return; }
  int motor = server.arg("m").toInt();
  MotorState s = (motor == 2) ? state2 : state1;
  char c = (s == FORWARD) ? 'F' : (s == BACKWARD) ? 'B' : 'S';
  server.send(200, "text/plain", String(c));
}

void handleAuto() {
  if (server.hasArg("v")) {
    autoTrack = (server.arg("v").toInt() == 1);
    if (autoTrack) {
      homing1 = homing2 = false;
      pidX.reset();
```

```
      pidY.reset();
      axisXStopped = true;
      axisYStopped = true;
    } else {
      setMotorState(1, 'S');
      setMotorState(2, 'S');
      pidX.reset();
      pidY.reset();
      axisXStopped = true;
      axisYStopped = true;
      lastOutX = lastOutY = 0;
    }
  }
  server.send(200, "text/plain", autoTrack ? "1" : "0");
}

// GET /pid?kp=..&ki=..&kd=..  -> cap nhat gains (ap dung ca 2 truc)
// GET /pid                    -> tra ve gains hien tai (JSON)
void handlePid() {
  if (server.hasArg("kp") || server.hasArg("ki") || server.hasArg("kd")) {
    float kp = server.hasArg("kp") ? server.arg("kp").toFloat() : pidX.Kp;
    float ki = server.hasArg("ki") ? server.arg("ki").toFloat() : pidX.Ki;
    float kd = server.hasArg("kd") ? server.arg("kd").toFloat() : pidX.Kd;
    // Chan pham vi hop ly
    if (kp < 0) kp = 0; if (kp > 10) kp = 10;
    if (ki < 0) ki = 0; if (ki > 5)  ki = 5;
    if (kd < 0) kd = 0; if (kd > 5)  kd = 5;
```

```
    pidX.setGains(kp, ki, kd);
    pidY.setGains(kp, ki, kd);
    pidX.reset();
    pidY.reset();
    Serial.print(">> PID gains: Kp="); Serial.print(kp, 3);
    Serial.print(" Ki="); Serial.print(ki, 3);
    Serial.print(" Kd="); Serial.println(kd, 3);
  }
  String json = "{\"kp\":" + String(pidX.Kp, 3)
              + ",\"ki\":" + String(pidX.Ki, 3)
              + ",\"kd\":" + String(pidX.Kd, 3) + "}";
  server.send(200, "application/json", json);
}

void handleEncoderPid() {
  if (server.hasArg("kp") || server.hasArg("ki") || server.hasArg("kd")) {
    float kp = server.hasArg("kp") ? server.arg("kp").toFloat() : pidEncoder1.Kp;
    float ki = server.hasArg("ki") ? server.arg("ki").toFloat() : pidEncoder1.Ki;
    float kd = server.hasArg("kd") ? server.arg("kd").toFloat() : pidEncoder1.Kd;
    if (kp < 0) kp = 0; if (kp > 200) kp = 200;
    if (ki < 0) ki = 0; if (ki > 100) ki = 100;
    if (kd < 0) kd = 0; if (kd > 50)  kd = 50;
    pidEncoder1.setGains(kp, ki, kd);
    pidEncoder2.setGains(kp, ki, kd);
    resetEncoderSpeedPid();
    Serial.print(">> Encoder PID: Kp="); Serial.print(kp, 2);
    Serial.print(" Ki="); Serial.print(ki, 2);
```

```
    Serial.print(" Kd="); Serial.println(kd, 2);
  }
  String json = "{\"kp\":" + String(pidEncoder1.Kp, 2)
              + ",\"ki\":" + String(pidEncoder1.Ki, 2)
              + ",\"kd\":" + String(pidEncoder1.Kd, 2) + "}";
  server.send(200, "application/json", json);
}

bool atHome(int pin) {
  const int SAMPLES = 8;
  for (int i = 0; i < SAMPLES; i++) {
    if (digitalRead(pin) == HIGH) return false;
    delayMicroseconds(300);
  }
  return true;
}

void handleHome() {
  autoTrack = false;
  pidX.reset();
  pidY.reset();
  axisXStopped = true;
  axisYStopped = true;
  lastOutX = lastOutY = 0;
  homing1 = !atHome(PIN_HOME1);
  homing2 = !atHome(PIN_HOME2);
  if (homing1) {
```

```
    setMotorState(1, HOME_DIR1);
  } else {
    setMotorState(1, 'S');
    resetEncoderCount();
  }
  if (homing2) setMotorState(2, HOME_DIR2); else setMotorState(2, 'S');
  if (!homing2) resetEncoder2Count();
  Serial.println(">> HOME: bat dau ve vi tri goc");
  server.send(200, "text/plain", "OK");
}

void runHoming() {
  if (homing1 && atHome(PIN_HOME1)) {
    homing1 = false;
    setMotorState(1, 'S');
    resetEncoderCount();
    Serial.println(">> HOME: Dong co 1 da ve vi tri goc, Encoder=0");
  }
  if (homing2 && atHome(PIN_HOME2)) {
    homing2 = false;
    setMotorState(2, 'S');
    resetEncoder2Count();
    Serial.println(">> HOME: Dong co 2 da ve vi tri goc, Encoder2=0");
  }
}

void handleSensor() {
```

```
long encoderPosition = readEncoderCount();
long encoder2Position = readEncoder2Count();
String json = "{\"v\":" + String(busVoltage_V, 2)
          + ",\"c\":" + String(current_mA, 1)
          + ",\"p\":" + String(power_mW, 1)
          + ",\"l1\":" + String(light1)
           + ",\"l2\":" + String(light2)
           + ",\"l3\":" + String(light3)
           + ",\"l4\":" + String(light4)
           + ",\"enc\":" + String(encoderPosition)
           + ",\"enc2\":" + String(encoder2Position)
           + ",\"st1\":" + String(targetEncoderSpeed1, 1)
           + ",\"sa1\":" + String(actualEncoderSpeed1, 1)
           + ",\"st2\":" + String(targetEncoderSpeed2, 1)
           + ",\"sa2\":" + String(actualEncoderSpeed2, 1)
          + ",\"auto\":" + String(autoTrack ? 1 : 0)
          + ",\"ex\":" + String(lastErrorX, 1)
          + ",\"ey\":" + String(lastErrorY, 1)
          + ",\"ox\":" + String(lastOutX, 1)
          + ",\"oy\":" + String(lastOutY, 1)
          + ",\"dx\":" + String(stepDelayUs1)
          + ",\"dy\":" + String(stepDelayUs2)
          + ",\"kp\":" + String(pidX.Kp, 3)
          + ",\"ki\":" + String(pidX.Ki, 3)
           + ",\"kd\":" + String(pidX.Kd, 3) + "}";
json.remove(json.length() - 1);
json += ",\"ekp\":" + String(pidEncoder1.Kp, 2)
```

```
      +  ",\"eki\":" + String(pidEncoder1.Ki, 2)
      +  ",\"ekd\":" + String(pidEncoder1.Kd, 2) + "}";
  server.send(200, "application/json", json);
}

void readINA219() {
  if (!ina219Found) return;
  shuntVoltage_mV = ina219.getShuntVoltage_mV();
  busVoltage_V    = ina219.getBusVoltage_V();
  current_mA      = ina219.getCurrent_mA();
  power_mW        = ina219.getPower_mW();
}

void readLight() {
  const int N = 8;
  long s1 = 0, s2 = 0, s3 = 0, s4 = 0;
  for (int i = 0; i < N; i++) {
    s1 += analogRead(PIN_LDR1);
    s2 += analogRead(PIN_LDR2);
    s3 += analogRead(PIN_LDR3);
    s4 += analogRead(PIN_LDR4);
  }
  light1 = s1 / N;
  light2 = s2 / N;
  light3 = s3 / N;
  light4 = s4 / N;
}
```

```
// Dual-axis solar tracking dung PID
//   errorX = right - left  (Motor 1)
//   errorY = top - bottom  (Motor 2)
// Output PID > 0 -> FORWARD, < 0 -> BACKWARD
// |output| lon -> step delay nho (nhanh)
void autoTracking() {
  if (!autoTrack) return;
  if (homing1 || homing2) return;

  long avg = ((long)light1 + light2 + light3 + light4) / 4;
  if (avg < LIGHT_MIN) {
    if (state1 != STOP) setMotorState(1, 'S');
    if (state2 != STOP) setMotorState(2, 'S');
    pidX.reset();
    pidY.reset();
    resetEncoderSpeedPid();
    axisXStopped = true;
    axisYStopped = true;
    lastErrorX = lastErrorY = 0;
    lastOutX = lastOutY = 0;
    return;
  }

  int left   = light1 + light3;
  int right  = light2 + light4;
  int top    = light1 + light2;
```

```
int bottom = light3 + light4;

float errorX = (float)(right - left);
float errorY = (float)(top - bottom);
lastErrorX = errorX;
lastErrorY = errorY;

// Deadband hysteresis:
// - Neu dang chay: vao vung nho LIGHT_DEADBAND_IN thi dung
// - Neu da dung: chi chay lai khi vuot LIGHT_DEADBAND_OUT
if (!axisXStopped && fabsf(errorX) < LIGHT_DEADBAND_IN) {
  axisXStopped = true;
  lastOutX = 0;
  targetEncoderSpeed1 = 0;
  if (state1 != STOP) { state1 = STOP; digitalWrite(PIN_STEP1, LOW); }
} else if (axisXStopped && fabsf(errorX) < LIGHT_DEADBAND_OUT) {
  lastOutX = 0;
  targetEncoderSpeed1 = 0;
  if (state1 != STOP) { state1 = STOP; digitalWrite(PIN_STEP1, LOW); }
} else {
  lastOutX = pidX.compute(errorX);
  applyPidToMotor(1, lastOutX, axisXStopped);
}

if (!axisYStopped && fabsf(errorY) < LIGHT_DEADBAND_IN) {
  axisYStopped = true;
  lastOutY = 0;
```

```
    targetEncoderSpeed2 = 0;
    if (state2 != STOP) { state2 = STOP; digitalWrite(PIN_STEP2, LOW); }
  } else if (axisYStopped && fabsf(errorY) < LIGHT_DEADBAND_OUT) {
    lastOutY = 0;
    targetEncoderSpeed2 = 0;
    if (state2 != STOP) { state2 = STOP; digitalWrite(PIN_STEP2, LOW); }
  } else {
    lastOutY = pidY.compute(errorY);
    applyPidToMotor(2, lastOutY, axisYStopped);
  }
}

// Step non-blocking: moi motor co delay rieng (PID dieu chinh toc do)
void stepMotor(MotorState &state, int stepPin, int dirPin,
               MotorState &lastDir, unsigned long &lastStepUs,
               unsigned long delayUs) {
  if (state == STOP) {
    if (lastDir != STOP) {
      digitalWrite(stepPin, LOW);
      lastDir = STOP;
    }
    return;
  }

  unsigned long now = micros();
  if ((now - lastStepUs) < delayUs) return;
  lastStepUs = now;
```

```
  if (state == FORWARD) {
    if (lastDir != FORWARD) {
      digitalWrite(dirPin, HIGH);
      lastDir = FORWARD;
    }
  } else { // BACKWARD
    if (lastDir != BACKWARD) {
      digitalWrite(dirPin, LOW);
      lastDir = BACKWARD;
    }
  }

  digitalWrite(stepPin, HIGH);
  delayMicroseconds(5);
  digitalWrite(stepPin, LOW);
}

void setup() {
  pinMode(PIN_STEP1, OUTPUT); pinMode(PIN_DIR1, OUTPUT);
  pinMode(PIN_STEP2, OUTPUT); pinMode(PIN_DIR2, OUTPUT);
  digitalWrite(PIN_STEP1, LOW); digitalWrite(PIN_DIR1, LOW);
  digitalWrite(PIN_STEP2, LOW); digitalWrite(PIN_DIR2, LOW);

  pinMode(PIN_HOME1, INPUT_PULLUP);
  pinMode(PIN_HOME2, INPUT_PULLUP);
```

```
pinMode(PIN_ENCODER_A, INPUT_PULLUP);
pinMode(PIN_ENCODER_B, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(PIN_ENCODER_A), encoderISR, RISING);
pinMode(PIN_ENCODER2_A, INPUT_PULLUP);
pinMode(PIN_ENCODER2_B, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(PIN_ENCODER2_A), encoder2ISR, RISING);

Serial.begin(115200);
while (!Serial) delay(10);

WiFi.softAP(AP_SSID, AP_PASSWORD);
Serial.println("\n=== ESP32 Dual Motor + INA219 + PID ===");
Serial.print("WiFi AP: "); Serial.println(AP_SSID);
Serial.print("IP:      "); Serial.println(WiFi.softAPIP());
Serial.println("Mat khau:  12345678");
Serial.println("Web:       http://192.168.4.1");
Serial.print("PID default: Kp="); Serial.print(pidX.Kp, 3);
Serial.print(" Ki="); Serial.print(pidX.Ki, 3);
Serial.print(" Kd="); Serial.println(pidX.Kd, 3);

analogReadResolution(12);
analogSetPinAttenuation(PIN_LDR1, ADC_11db);
analogSetPinAttenuation(PIN_LDR2, ADC_11db);
analogSetPinAttenuation(PIN_LDR3, ADC_11db);
analogSetPinAttenuation(PIN_LDR4, ADC_11db);

Wire.begin(21, 22);
```

```
Serial.println("\n--- Quet I2C bus ---");
int foundCount = 0;
for (int addr = 1; addr < 127; addr++) {
  Wire.beginTransmission(addr);
  if (Wire.endTransmission() == 0) {
    Serial.print("  Tim thay: 0x"); Serial.print(addr, HEX);
    if (addr >= 0x40 && addr <= 0x43) {
      ina219Found = true;
      Serial.print(" [INA219]");
    }
    Serial.println();
    foundCount++;
  }
}
if (foundCount == 0) {
  Serial.println("  KHONG co thiet bi nao tren I2C!");
  Serial.println("  Kiem tra: SDA->D21, SCL->D22, VCC->3.3V, GND->GND");
} else {
  Serial.print("  Tong cong: "); Serial.print(foundCount); Serial.println(" thiet bi");
}

if (ina219Found && ina219.begin()) {
  Serial.println("INA219: OK - da doc duoc");
  readINA219();
} else {
  Serial.println("INA219: KHONG doc duoc! Kiem tra ket noi module");
}
```

```
  readLight();
  Serial.print("\nTEMT6000: D34="); Serial.print(light1);
  Serial.print("  D35="); Serial.print(light2);
  Serial.print("  D32="); Serial.print(light3);
  Serial.print("  D33="); Serial.println(light4);

   Serial.println("\nSerial:  1F=Thuan1  1B=Nguoc1  1S=Dung1   2F=Thuan2  2B=Nguoc2
2S=Dung2");
  Serial.println("Encoder truc duoi: A=D5, B=D23, vi tri ban dau=0");
  Serial.println("Encoder truc doc: A=D13, B=D14, vi tri ban dau=0");

  server.on("/",       handleRoot);
  server.on("/cmd",    handleCmd);
  server.on("/state",  handleState);
  server.on("/sensor", handleSensor);
  server.on("/auto",   handleAuto);
  server.on("/home",   handleHome);
  server.on("/pid",    handlePid);
  server.on("/encoderpid", handleEncoderPid);
  server.begin();
  Serial.println("Web Server da san sang!");
}

void loop() {
  static unsigned long lastServerCheck = 0;
  static unsigned long lastSensorRead  = 0;
```

```
unsigned long now = micros();

if (now - lastServerCheck >= 5000) {
  lastServerCheck = now;
  server.handleClient();

  while (Serial.available() >= 2) {
    int motor = Serial.read() - '0';
    char cmd  = Serial.read();
    if (motor == 1 || motor == 2) {
      autoTrack = false;
      pidX.reset();
      pidY.reset();
      if (motor == 1) homing1 = false;
      if (motor == 2) homing2 = false;
      setMotorState(motor, cmd);
    } else {
      Serial.println("Lenh khong hop le. VD: 1F, 1B, 1S, 2F, 2B, 2S");
    }
  }
  while (Serial.available() > 0) Serial.read();

  runHoming();
}

// Doc cam bien + chay PID ~ moi 100ms (nhanh hon de PID muot)
if (now - lastSensorRead >= 100000) {
```

```
  lastSensorRead = now;
  readINA219();
  readLight();
  autoTracking();
  updateEncoderSpeedPid();

  // Serial print cham va co the block, lam xung STEP bi ngat quang.
  // Chi in moi 1 giay de auto chay muot hon.
  static unsigned long lastSerialPrintMs = 0;
  unsigned long ms = millis();
  if (ms - lastSerialPrintMs >= 1000) {
    lastSerialPrintMs = ms;
    Serial.print("INA219 | V="); Serial.print(busVoltage_V, 2);
    Serial.print("V  I=");       Serial.print(current_mA, 1);
    Serial.print("mA  P=");      Serial.print(power_mW, 1);
    Serial.print("mW  | L1=");   Serial.print(light1);
    Serial.print("  L2=");       Serial.print(light2);
    Serial.print("  L3=");       Serial.print(light3);
    Serial.print("  L4=");       Serial.print(light4);
    Serial.print("  Encoder=");  Serial.print(readEncoderCount());
    Serial.print("  Encoder2="); Serial.print(readEncoder2Count());
    Serial.print("  Auto=");     Serial.print(autoTrack ? "ON" : "OFF");
    if (autoTrack) {
      Serial.print("  eX="); Serial.print(lastErrorX, 0);
      Serial.print(" oX=");  Serial.print(lastOutX, 1);
      Serial.print(" dX=");  Serial.print(stepDelayUs1);
      Serial.print("  eY="); Serial.print(lastErrorY, 0);
```

```
      Serial.print(" oY=");  Serial.print(lastOutY, 1);
      Serial.print(" dY=");  Serial.print(stepDelayUs2);
      Serial.print("  EncPID1="); Serial.print(targetEncoderSpeed1, 1);
      Serial.print("/"); Serial.print(actualEncoderSpeed1, 1);
      Serial.print("  EncPID2="); Serial.print(targetEncoderSpeed2, 1);
      Serial.print("/"); Serial.print(actualEncoderSpeed2, 1);
    }
    Serial.println();
  }
 }

 encoderCountingEnabled = (state1 != STOP);
 encoder2CountingEnabled = (state2 != STOP);
 stepMotor(state1, PIN_STEP1, PIN_DIR1, lastDir1, lastStepUs1, stepDelayUs1);
 stepMotor(state2, PIN_STEP2, PIN_DIR2, lastDir2, lastStepUs2, stepDelayUs2);
}
```

PHỤ LỤC B

BẢN VẼ